

Smarter Regular Expression Matching

From Theory to Tools

Ilian Kohl

September 18, 2026

Contents

1	Introduction	4
1.1	Motivation	4
1.2	Problem Statement	5
1.3	Contributions	5
1.4	Thesis Outline	6
2	Regular Expressions	7
2.1	Syntax	7
2.2	Semantics	8
2.3	Rust Implementation	9
3	Derivatives and Partial Derivatives	11
3.1	Deciding Membership	11
3.2	Brzozowski Derivatives	12
3.3	Antimirov Partial Derivatives	16
3.4	Matcher Selection	20
4	Matching versus Parsing	22
4.1	The Matching Problem	22
4.2	Parse Trees as Matching Evidence	24
4.3	Disambiguation Policies	28
4.3.1	Greedy Leftmost Matching	28
4.3.2	The Infinite Empty-Match Chain Problem	30
4.3.3	POSIX Longest-Leftmost Matching	31
5	Derivative-Based Parser	36
5.1	Reading the Implementation's Traces	36
5.2	Core Algorithm	37
5.3	Correctness Properties	40
5.4	Bit-Coded Optimization (Theory)	41
5.5	Simplification Rules	44
5.6	Rust Implementation: The <code>deriv_std</code> Parser	47
5.6.1	<code>ParseTree</code> and <code>flatten</code>	47
5.6.2	<code>mk_eps</code>	47
5.6.3	<code>inject</code>	47
5.6.4	<code>parse_deriv_std_rec</code> versus <code>parse_deriv_std_loop</code>	48
5.7	Rust Implementation: The Bit-Coded Version	48
5.7.1	<code>ARegex</code> , the Bit-Annotated Regex	49

5.7.2	<code>internalize</code> and <code>fuse</code>	49
5.7.3	<code>deriv_bc</code>	49
5.7.4	<code>mk_eps_bc</code>	50
5.7.5	<code>decode</code>	50
5.8	Comparing the Two Constructions	50
6	Partial-Derivative-Based Parser	53
6.1	Extending Derivatives to Partial Derivatives	53
6.2	Bit-Coded Variant	55
6.3	Rust Implementation: The <code>pderiv_bc</code> Parser	56
6.4	Rust Implementation: The <code>pderiv_std</code> Parser	58
6.5	Frontier Growth	61
6.6	Comparing the Two Policies	62
7	Frontend: Parsing External Regular Expression Syntax	64
7.1	Two ASTs, One Lowering Step	64
7.2	Parsing Surface Syntax	65
7.3	Translating <code>ExtPat</code> into <code>Regex</code>	67
7.4	Substring Search	69
7.5	The Cost of Translation	70
8	Evaluation	73
8.1	Compared Engines	73
8.1.1	Rust’s <code>regex</code> Crate	73
8.1.2	RE2	74
8.2	Testing Strategy	74
8.3	Dataset Preparation	74
8.4	Correctness Verification	76
8.4.1	Paper Examples Reproduced	76
8.4.2	Cross-Implementation Equivalence	76
8.5	Performance Analysis	77
8.5.1	Stack Depth: Recursive vs. Iterative Construction	77
8.5.2	Heap Allocation: <code>deriv_std</code> vs. <code>deriv_bc</code>	77
8.5.3	Derivative-Based vs. Partial-Derivative-Based	77
8.5.4	Pathological Inputs	77
8.5.5	Benchmark Results	78
8.6	Comparison with Existing Engines	78
8.6.1	Performance	78
8.6.2	Standardized Benchmarking via <code>rebar</code>	78
9	Discussion	79
9.1	Discussion of Evaluation Results	79
9.2	Rust versus Haskell	79
9.2.1	Language Characteristics of the Reference Implementation	79
9.2.2	What the Reference Provides vs. What It Leaves Open	80
9.2.3	Translation Strategy	80
9.2.4	Translation Insights	80
9.3	Mismatch Analysis via Derivative Tracing	80
9.3.1	The Gap	80

9.3.2	Deriving Diagnostics for Free from the Algorithm Itself	80
9.3.3	Design	80
9.3.4	Scope and Limits	81
9.4	Optimization Challenges and Insights	81
9.5	Future Work	81
9.5.1	Paper-Suggested Extensions	81
9.5.2	Further Performance Optimizations	81
9.5.3	A Full Mismatch Diagnostics System	81
10	Related Work and Conclusion	82
A	Code Listings	85
B	Full Benchmark Results	86
C	Test Case Catalogue	87
D	CLI and ENV Reference	88

Introduction

1.1 Motivation

Regular expressions are a fundamental formalism for describing regular languages and are among the most widely used abstractions in computer science. They underpin applications including lexical analysis, input validation, log processing, search tools, and network security. Across these applications, the problem is regular expression matching. Given a regular expression and an input string, determine whether the string belongs to the language denoted by the expression. Formally, this is the *membership problem*.

For example, the expression $(ab|a)^*$ matches "aba", as it decomposes into "ab" followed by "a", but does not match "abb", since no sequence of subexpressions can produce it.

For many applications, a Boolean answer is enough. An input validator may only need to accept or reject a password or URL. Others require more. For instance, a lexer must determine how input decomposes into tokens. Capturing groups, schema validation, and information extraction require identifying which input parts correspond to parts of the expression. Recognition alone is not enough; the matcher must also recover the *structure*.

Efficiently solving these problems remains an active research area. The choice of algorithm influences performance, memory consumption, and security. Once matching becomes parsing, the way in which match structure is recovered also influences correctness and raises the question of how to handle ambiguity when multiple matches are possible.

Existing approaches include backtracking and finite-automata-based techniques, such as nondeterministic (NFA) and deterministic (DFA) automata. These approaches have different trade-offs in implementation complexity, memory consumption, and worst-case performance. Backtracking engines are flexible and widely used, but may exhibit exponential running time on constructed inputs, leading to regular expression denial-of-service (ReDoS) vulnerabilities [Cox, 2007]. Automata-based approaches provide stronger worst-case guarantees but may require preprocessing or suffer from state-space explosion.

Derivative-based algorithms provide an alternative by transforming the regular expression as input symbols are consumed. Each derivative represents the language remaining after an input symbol has been consumed. By maintaining a correspondence between an expression and its successive residuals, derivative-based methods provide an elegant foundation for regular expression matching and, with suitable extensions, structured parsing.

However, derivative-based techniques are less common in practical regular expression tooling. Their use for practical matching, parsing, and diagnostic tools motivates this thesis: "From Theory to Tools".

1.2 Problem Statement

Derivative-based regular expression matching has been studied extensively from a theoretical perspective. Brzozowski introduced regular expression derivatives to construct a finite-state automaton directly from a regular expression [Brzozowski, 1964], while Antimirov introduced partial derivatives, which represent sets of residual expressions and are closely related to nondeterministic automata [Antimirov, 1996]. Building on these ideas, Sulzmann and Lu developed a derivative-based algorithm for POSIX-correct parsing and provided a Haskell reference implementation [Sulzmann and Lu, 2014].

Sulzmann and Lu’s work therefore provides a basis for structured regular expression matching, but leaves several practical questions open. In particular, these constructions can be investigated in a systems programming language with respect to execution time and memory consumption. Rust provides an appropriate setting through its ownership model and fine-grained control over data representation and memory allocation.

Additionally, the partial-derivative-based greedy parser provides an alternative for practical comparison with the derivative-based parser and the bit-coded parse-tree representation enables investigation of its effect on memory consumption.

The central problem addressed by this thesis is therefore to investigate the practical behaviour of these derivative-based parsing constructions: first, by implementing them in Rust and evaluating their performance and resource usage; second, by comparing the resulting parsers with two established regular expression engines. In addition, the thesis investigates whether the intermediate derivative computations can be exposed to provide useful explanations of successful and unsuccessful matches.

1.3 Contributions

This thesis makes the following contributions:

- **Rust implementation of derivative-based parsers.** The thesis implements the parsing constructions developed by Sulzmann and Lu [Sulzmann and Lu, 2014] in Rust, using their Haskell implementation as a reference. Both the derivative-based POSIX and partial-derivative-based greedy parsers are implemented in direct and bit-coded variants, resulting in four parser variants.
- **Empirical evaluation of the parser variants.** The four implementations are evaluated with respect to execution time and memory consumption using synthetic benchmarks and a corpus of real-world regular expressions from Snort/Suricata, SpamAssassin, and RegexLib, following the source categories investigated by Mamouras et al. [Mamouras et al., 2024]. The derivative-based implementations are additionally compared with Rust’s `regex` crate and Google’s RE2.
- **Derivative-tracing diagnostics.** The implementation records and exposes intermediate derivative computations during matching. These traces show how the regular expression evolves as input symbols are consumed and form the basis of a diagnostic system for explaining successful and unsuccessful matches.

1.4 Thesis Outline

The remainder of this thesis follows the development of the parsing constructions from their underlying formal concepts through their Rust implementation to their empirical evaluation. Rather than separating theoretical concepts and implementation into distinct chapters, the relevant concepts are introduced alongside their implementation. This structure follows the development of the constructions by Sulzmann and Lu while presenting their implementation and evaluation in Rust.

- **Chapter 2 - Regular Expressions.** This chapter introduces the syntax and semantics of regular expressions, together with the Rust representation of their syntax used throughout the implementation.
- **Chapter 3 - Derivatives.** This chapter introduces Brzozowski’s derivatives and Antimirov’s partial derivatives as algorithms for deciding language membership, together with their Rust implementation.
- **Chapter 4 - Matching vs. Parsing.** Before applying these algorithms to parsing, this chapter distinguishes matching from parsing and introduces the disambiguation problem that arises when a parse tree, rather than only a Boolean result, is required.
- **Chapter 5 - Derivative Parser.** This chapter presents Sulzmann and Lu’s derivative-based POSIX parser [Sulzmann and Lu, 2014], which provides the main algorithmic basis for this thesis, together with its Rust implementation in both direct and bit-coded forms.
- **Chapter 6 - Partial-Derivative Parser.** This chapter presents Sulzmann and Lu’s greedy parser based on Antimirov’s partial derivatives, together with its direct and bit-coded Rust implementations.
- **Chapter 7 - Front-End.** This chapter describes how a regular expression string is parsed into an external AST and translated into the internal representation used by the parsers.
- **Chapter 8 - Evaluation.** With all four parser variants in place, this chapter evaluates their correctness and performance with respect to execution time and memory consumption. The evaluation uses both synthetic benchmarks and a corpus of real-world regular expressions and compares the parser variants with Rust’s `regex` crate and Google’s RE2.
- **Chapter 9 - Discussion.** This chapter discusses and interprets the evaluation results, situates the work relative to related approaches, and reflects on the translation of the reference constructions from Haskell to Rust. It also discusses the derivative-tracing system, including its usefulness and limitations.
- **Chapter 10 - Conclusion.** Finally, this chapter summarizes the main findings and contributions in relation to the research questions introduced in this chapter. It also discusses directions for future work, including potential extensions and improvements to the parser implementations and diagnostic tooling.

Regular Expressions

2.1 Syntax

Over an alphabet Σ , regular expressions are defined inductively by the grammar

$$r, s ::= x \mid \varepsilon \mid \emptyset \mid r + s \mid r \cdot s \mid r^* \quad (x \in \Sigma). \quad (2.1)$$

Here x denotes a literal symbol, ε the expression matching only the empty word, and $\emptyset$ the expression matching no word. The operators $+$, $\cdot$, and $*$ denote alternation, concatenation, and Kleene star, respectively. Concatenation is often written implicitly, so rs denotes $r \cdot s$.

Unless parentheses are shown explicitly, Kleene star binds most tightly, followed by concatenation, followed by alternation. Thus, for example, ab^*c is parsed as $(a \cdot (b^*)) \cdot c$, and $a + bc^*$ is parsed as $a + (b \cdot (c^*))$, not $(a + b) \cdot (c^*)$.

Both $+$ and $\cdot$ are associative at the language level, meaning that different groupings denote the same language. For example, $L((r+s)+t) = L(r+(s+t))$ and $L((r \cdot s) \cdot t) = L(r \cdot (s \cdot t))$. The grouping determines the *syntax tree* on which the algorithms operate. Therefore, both operators associate to the *left*: $r + s + t$ means $(r + s) + t$, while rst means $(rs)t$.

The grammar deliberately omits convenience operators such as one-or-more, optional, and bounded repetition (r^+ , $r?$, $r^{\{m,n\}}$). These can be expressed using the six core constructs. For example, $r^+ = r \cdot r^*$, $r? = r + \varepsilon$, and

$$r^{\{m,n\}} = \underbrace{r \cdot \dots \cdot r}_{m \text{ times}} \cdot \underbrace{(r + \varepsilon) \cdot \dots \cdot (r + \varepsilon)}_{n-m \text{ times}}.$$

Thus, the omission does not restrict the expressiveness of the core syntax. Chapter 7 (Section 7.3) goes into detail of the complete desugarings.

As a running example, consider the problem of recognizing binary strings that end in "01". Over $\Sigma = \{0, 1\}$, this language is described by

$$r_{01} = (0 + 1)^* \cdot 0 \cdot 1. \quad (2.2)$$

Here $(0 + 1)$ is an alternation of the two literals 0 and 1, and $(0 + 1)^*$ permits any finite sequence of binary symbols. The expression then concatenates this sequence with 0 and 1, so the final two symbols must be "01". For example, it matches "01", "001", and "1101", but not "0" or "110". By the precedence and left-associativity rules above, its syntax tree is structured as $((0 + 1)^* \cdot 0) \cdot 1$. This tree, rather than the expression's textual representation, is the structure operated on by the algorithms.

2.2 Semantics

The syntax above describes the *shape* of an expression; the semantics assigns each expression the *language* it denotes. For a regular expression r , write $L(r)$ for the set of words it matches. The semantics is defined by structural recursion on r :

$$\begin{aligned}
 L(x) &= \{x\} \\
 L(\varepsilon) &= \{\varepsilon\} \\
 L(\emptyset) &= \{\} \\
 L(r + s) &= L(r) \cup L(s) \\
 L(r \cdot s) &= \{vw \mid v \in L(r), w \in L(s)\} \\
 L(r^*) &= \{\varepsilon\} \cup \{w_1 \cdots w_n \mid w_1, \dots, w_n \in L(r), n \geq 1\}
 \end{aligned} \tag{2.3}$$

Table 2.1 illustrates the distinction between an expression and the language it denotes, applying Equation (2.3) to six small cases.

Expression	Language
a	$\{a\}$
ε	$\{\varepsilon\}$
$\emptyset$	$\emptyset$
$a + b$	$\{a, b\}$
ab	$\{ab\}$
a^*	$\{\varepsilon, a, aa, aaa, \dots\}$

Table 2.1: Six expressions and the languages Equation (2.3) assigns them. Different expressions may denote related or identical sets of words, while the semantics assigns a precise language to each.

The distinction between an expression and its denoted language is important for the algorithms developed later. Derivative operations are defined syntactically over regular expressions, while their correctness is understood in terms of the languages those expressions denote.

A word w *matches* r exactly when $w \in L(r)$. This is the membership problem.

Unwinding the semantics for the running example, $L(0 + 1) = \{0, 1\}$, so $L((0 + 1)^*)$ is the set of all finite binary strings, including the empty string. Concatenating with $L(0) = \{0\}$ and then $L(1) = \{1\}$ restricts this to

$$L(r_{01}) = \{w \cdot 0 \cdot 1 \mid w \in \{0, 1\}^*\}, \tag{2.4}$$

i.e. exactly the binary strings ending in "01" like "01", "001", "1101", and so on. In particular, the empty string is not matched, since every word in $L(r_{01})$ contains the final literal 0 followed by the literal 1.

So an expression has both a syntactic structure and a meaning: the syntax determines how the expression is constructed, while its semantics, determines the language it denotes. Different syntax trees may therefore describe the same language. The distinction between an expression and its denoted language is important for the algorithms developed later.

The derivative algorithms transform regular expressions themselves, rather than their denoted languages. For example, consuming a from the expression ab produces the residual expression b , whose language is $\{b\}$. Thus, the algorithms operate on expressions, while $L(r)$ describes the language represented by each resulting expression.

2.3 Rust Implementation

The grammar above is represented directly as a Rust enum, mirroring each syntactic construct one to one. Rust enums define a type by enumerating its possible variants, and are commonly used together with pattern matching to distinguish between those variants [The Rust Project Developers, 2026c,i].

```
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
pub enum Regex {
    Phi,
    Eps,
    Lit(char),
    Seq(Box<Regex>, Box<Regex>),
    Alt(Box<Regex>, Box<Regex>),
    Star(Box<Regex>),
}
```

In this representation, each production of the grammar corresponds to one enum variant. For example, `Phi` represents $\emptyset$, `Eps` represents ε , `Lit` represents a literal symbol, and the remaining variants represent the three recursive constructs.

For example, the running example $r_{01} = (0 + 1)^* \cdot 0 \cdot 1$ is represented structurally by an expression tree whose root is a concatenation, with $(0 + 1)^* \cdot 0$ and 1 as its subexpressions, the same left-associated grouping fixed in Section 2.1. Its structure is Equation (2.5):

$$\text{Seq}(\text{Seq}(\text{Star}(\text{Alt}(0, 1)), 0), 1). \quad (2.5)$$

Thus, the Rust value is not merely a string representation of the regular expression; it is a tree that directly represents its syntactic structure. This structural representation allows operations to be defined recursively over the expression.

Rust’s `match` expressions are checked for exhaustiveness by the compiler, meaning that every possible pattern must be covered either explicitly or through a wildcard pattern [The Rust Project Developers, 2026d,j]. This mirrors the structural guarantee provided by the inductive definition of r in Section 2.1: a function defined by cases on r must account for every possible form of r .

The derived traits support operations used throughout the implementation. The `derive` attribute generates implementations of supported traits for the annotated type [The Rust Project Developers, 2026b]. In particular, `Debug` facilitates inspection of expressions, `Clone` allows expressions to be duplicated when required, and `PartialEq`, `Eq`, and `Hash` allow expressions to be compared and used as keys in hash-based collections. Rust’s `HashMap`, for example, requires its key type to implement both `Eq` and `Hash` [The Rust Project Developers, 2026g,f]. The variants `Phi` and `Eps` carry no data, corresponding directly to $\emptyset$ and ε , while `Lit` stores the literal symbol x . In the implementation, literal

symbols are represented using Rust’s `char` type and therefore correspond to Unicode scalar values [The Rust Project Developers, 2026e].

The recursive constructs, `Seq`, `Alt`, and `Star` require explicit indirection in Rust. A recursive type must have a finite size, so a definition such as `Seq(Regex, Regex)` would not have a finite size: each `Regex` would contain another `Regex`, and so on. `Box<Regex>` provides the necessary indirection by storing the recursive value separately and keeping a fixed-size pointer-like value within the enum [The Rust Project Developers, 2026a,k].

The use of `Box` is therefore one of the practical differences between this Rust representation and the Haskell reference implementation. It introduces explicit indirection and heap-based storage for recursive subexpressions, aspects that are relevant when considering the memory behaviour of the implementation. These implementation differences are revisited more generally in Section 9.2.

With this representation established, operations on regular expressions can be defined structurally over the expression tree.

Derivatives and Partial Derivatives

Chapter 2 introduced the syntax and semantics of regular expressions. The semantics provides a declarative definition of membership: a word w matches an expression r exactly when $w \in L(r)$. This chapter introduces three algorithms for deciding membership.

The first is a direct "naive" recursive matcher that follows the structure of the semantic definition. The second uses Brzozowski derivatives to transform an expression as input symbols are consumed. The third uses Antimirov's partial derivatives, representing the possible residual expressions as a collection rather than combining them into a single expression. All three are implemented in Rust with the common interface `fn(&str, &Regex) -> bool`.

The three approaches decide the same membership problem, but they differ in how they represent and explore the remaining matching problem. The derivative-based approaches introduced here form the basis of the parsers developed in Chapters 5 and 6.

3.1 Deciding Membership

The semantics of Section 2.2 defines membership declaratively: a word w matches a regular expression r exactly when $w \in L(r)$. A direct way to decide membership is therefore to interpret these semantic definitions as recursive matching rules.

Let w be represented as an indexable sequence of characters, and let $w[i : j]$ denote the substring beginning at position i and ending immediately before position j . A recursive procedure `match(w, i, j, r)` can then determine whether $w[i : j] \in L(r)$ by following the structure of r :

- ε matches $w[i : j]$ iff $i = j$.
- A literal x matches $w[i : j]$ iff $j = i + 1$ and the character at position i is x .
- $r + s$ matches $w[i : j]$ iff r matches $w[i : j]$ or s does.
- $r \cdot s$ matches $w[i : j]$ iff there is some split point $k \in [i, j]$ such that r matches $w[i : k]$ and s matches $w[k : j]$.
- r^* matches $w[i : j]$ iff $i = j$, or there is some $k \in (i, j]$ such that r matches $w[i : k]$ and r^* matches $w[k : j]$.

Membership of the whole word w corresponds to `match(w, 0, |w|, r)`. Each matching case is a direct operational restatement of the corresponding clause in the definition of $L(r)$, meaning the correctness follows directly from the regular-expression semantics.

It is also, in general, an expensive procedure. Concatenation and star both search over every split point in the current range rather than committing to one, and nothing remembers a range once it has already been explored while resolving a different call. Take $r = a \cdot a \cdot a$ against $w = \mathbf{aaa}$. Matching $w[0 : 3]$ against r tries every split point $k \in \{0, 1, 2, 3\}$ for the outer concatenation, even though only $k = 1$ can possibly succeed, since the left operand is a single literal and can only match a range of length exactly 1. The right operand at that point, $w[1 : 3]$ against $a \cdot a$, repeats the same search one level down. For a chain of n literals against an input of length n , the number of ranges examined this way grows exponentially in n rather than linearly, since each concatenation node re-explores the full range handed to it regardless of what a sibling call already tried.

`src/matchers/match_naive.rs` implements this procedure directly on a `Vec<char>` and a pair of `usize` bounds. Despite its cost, it remains useful: because it transcribes $L(r)$'s definition without any intervening transformation, it serves as an independent reference that the more efficient derivative-based matchers introduced below can be checked against.

The derivative approach takes a different view of the same problem. Rather than searching for a decomposition of w while keeping r fixed, it *transforms* r as each character of w is consumed. The resulting expression describes precisely the language that can still match the remaining suffix, allowing the matcher to continue with an adjusted expression rather than reconsidering the original r at every step.

3.2 Brzowski Derivatives

Brzowski's derivative of r with respect to a symbol x , written $r \backslash x$, is the expression describing exactly what remains of r 's language once a leading x has been removed [Brzowski, 1964]:

$$L(r \backslash x) = \{ w \mid xw \in L(r) \}. \quad (3.1)$$

An expression is *nullable* if it accepts the empty word ε . Let $n(r)$ denote the corresponding Boolean predicate, so that $n(r)$ is true exactly when $\varepsilon \in L(r)$. Deciding whether a word $x_1 x_2 \cdots x_n$ is in $L(r)$ then reduces to computing the chain $r \backslash x_1 \backslash x_2 \backslash \cdots \backslash x_n$ and checking whether the final expression is nullable. Both nullability and the derivative itself can be defined by structural recursion directly on the syntax of r , with no reference back to $L(r)$ needed once the base cases are fixed. Equation (3.2) gives nullability and Equation (3.3) the derivative:

$$\begin{array}{lll} n(\emptyset) = \text{false} & (\mathbf{N}_\emptyset) & n(r + s) = n(r) \vee n(s) \quad (\mathbf{N}_+) \\ n(\varepsilon) = \text{true} & (\mathbf{N}_\varepsilon) & n(r \cdot s) = n(r) \wedge n(s) \quad (\mathbf{N}_{\text{seq}}) \\ n(x) = \text{false} & (\mathbf{N}_{\text{lit}}) & n(r^*) = \text{true} \quad (\mathbf{N}_*) \end{array} \quad (3.2)$$

$$\begin{aligned}
\emptyset \backslash x &= \emptyset & (D_{\emptyset}) \\
\varepsilon \backslash x &= \emptyset & (D_{\varepsilon}) \\
y \backslash x &= \begin{cases} \varepsilon & x = y \\ \emptyset & x \neq y \end{cases} & (D_{\text{lit}}) \\
(r + s) \backslash x &= (r \backslash x) + (s \backslash x) & (D_{+}) \\
r^* \backslash x &= (r \backslash x) \cdot r^* & (D_{*}) \\
(r \cdot s) \backslash x &= \begin{cases} (r \backslash x) \cdot s + s \backslash x & n(r) = \text{true} \\ (r \backslash x) \cdot s & n(r) = \text{false} \end{cases} & \begin{matrix} (D_{\text{seq}}^n) \\ (D_{\text{seq}}) \end{matrix}
\end{aligned} \tag{3.3}$$

Each rule carries a name in the right-hand column, and every derivation in this chapter and in Chapters 5 and 6 cites those names step by step. Two of them, D_{+} and D_{seq} , are stated over metavariables r and s , so applying one to a concrete expression means first deciding which subexpression each metavariable stands for. Every application below states that binding explicitly before the rule is used, because for a left-associated expression such as $r_{01} = ((0 + 1)^* \cdot 0) \cdot 1$ the answer is not the one the written form suggests at a glance: the outer concatenation binds $r = (0 + 1)^* \cdot 0$ and $s = 1$, not $r = (0 + 1)^*$ and $s = 0 \cdot 1$.

The only case that needs justification beyond direct case analysis on $L(r \cdot s)$ is concatenation, the last line of Equation (3.3): if r is nullable, a word starting with x can either continue matching r (so the derivative goes into r while s is left untouched) or arise from r already being satisfied by ε , with x belonging to s instead. Hence the extra $s \backslash x$ summand appears exactly when $n(r)$ holds. If r is not nullable, this second possibility cannot arise, since $\varepsilon \notin L(r)$ rules out r having already been satisfied before any input was consumed.

Worked example: single-symbol matching. Consider x^* against the word $\mathbf{xx}$. Writing $r \xrightarrow{x} r \backslash x$ for one derivative step,

Step 1. The expression is a star, so D_{*} applies with body $r = x$:

$$\begin{aligned}
x^* \backslash \mathbf{x} &= (x \backslash \mathbf{x}) \cdot x^* & D_{*}, r = x \\
&= \varepsilon \cdot x^* & D_{\text{lit}}, \text{ since } \mathbf{x} = x
\end{aligned} \tag{3.4}$$

Step 2. The expression is now a concatenation, so the concatenation rule applies with $r = \varepsilon$ and $s = x^*$. Since $n(\varepsilon) = \text{true}$ by $\mathbf{N}_{\varepsilon}$, it is the nullable case D_{seq}^n that fires, and that is what introduces the $+$:

$$\begin{aligned}
(\varepsilon \cdot x^*) \backslash \mathbf{x} &= (\varepsilon \backslash \mathbf{x}) \cdot x^* + x^* \backslash \mathbf{x} & D_{\text{seq}}^n, r = \varepsilon, s = x^* \\
&= \emptyset \cdot x^* + x^* \backslash \mathbf{x} & D_{\varepsilon} \\
&= \emptyset \cdot x^* + (x \backslash \mathbf{x}) \cdot x^* & D_{*}, r = x \\
&= \emptyset \cdot x^* + \varepsilon \cdot x^* & D_{\text{lit}}
\end{aligned} \tag{3.5}$$

Nullability of the result, by structural recursion:

$$\begin{aligned}
n(\emptyset \cdot x^* + \varepsilon \cdot x^*) &= n(\emptyset \cdot x^*) \vee n(\varepsilon \cdot x^*) & \mathbf{N}_+, r = \emptyset \cdot x^*, s = \varepsilon \cdot x^* \\
&= (n(\emptyset) \wedge n(x^*)) \vee (n(\varepsilon) \wedge n(x^*)) & \mathbf{N}_{\text{seq}}, \text{ twice} \\
&= (\text{false} \wedge \text{true}) \vee (\text{true} \wedge \text{true}) & \mathbf{N}_\emptyset, \mathbf{N}_*, \mathbf{N}_\varepsilon \\
&= \text{true}
\end{aligned} \tag{3.6}$$

so x^* matches $\mathbf{xx}$, correctly. This short trace makes two things visible. First, the expression has grown from two nodes to nine across two steps, purely from the $+$ that $\mathbf{D}_{\text{seq}}^n$ introduces at each nullable-prefix step; nothing above bounds this growth, and a third derivative step would nest a further $+$ inside each existing summand, reaching fourteen nodes. Second, the identities listed below collapse the result back to x^* itself, the expression the trace started from, which is why applying them after every step keeps this particular sequence at a fixed size rather than letting it grow. The collapse takes three of the four identities in turn:

$$\begin{aligned}
\emptyset \cdot x^* + \varepsilon \cdot x^* &= \emptyset + \varepsilon \cdot x^* & \mathbf{S}_1, r = x^* \\
&= \varepsilon \cdot x^* & \mathbf{S}_3, r = \varepsilon \cdot x^* \\
&= x^* & \mathbf{S}_2, r = x^*
\end{aligned} \tag{3.7}$$

Four identities do this collapsing, Equation (3.8), each a direct consequence of the semantics of Section 2.2:

$$\begin{aligned}
\emptyset \cdot r = \emptyset & \quad (\mathbf{S}_1) & \emptyset + r = r & \quad (\mathbf{S}_3) \\
\varepsilon \cdot r = r & \quad (\mathbf{S}_2) & r + r = r & \quad (\mathbf{S}_4) \quad (\text{idempotent alternation})
\end{aligned} \tag{3.8}$$

`src/regex/simplify.rs` implements exactly these four rules, applied to every subexpression; `match_deriv (src/matchers/match_deriv.rs)` calls it after every derivative step, which is why neither worked example in this section ever shows an expression larger than what is written down. Each step in the trace below is annotated with the identity it uses.

This is not only a matter of efficiency: Antimirov observes that Brzozowski’s original formulation of the concatenation rule, phrased as multiplication by the nullability value rather than as the explicit case split used above, produces an infinite, non-collapsing sequence of derivatives for expressions such as $(x + y)^*$ unless simplification is applied at every step [Antimirov, 1996]. Eager simplification is therefore what keeps the finiteness property in practice, not merely what keeps expressions small.

Worked example: $r_{01} = (0 + 1)^* \cdot 0 \cdot 1$ Recall r_{01} from Section 2.1, left-associated as $((0 + 1)^* \cdot 0) \cdot 1$. Derive it against $w = 101$ one symbol at a time.

Because every rule below is applied to a concatenation, it is worth fixing the bindings once. r_{01} has three concatenation nodes, and matching each against the pattern $r \cdot s$ gives:

Node	r	s
$((0 + 1)^* \cdot 0) \cdot 1$ (the whole of r_{01})	$(0 + 1)^* \cdot 0$	1
$(0 + 1)^* \cdot 0$	$(0 + 1)^*$	0
inside $\mathbf{D}_*$: the star body	$0 + 1$	

The first row is the one that is easy to get wrong. r_{01} is left-associated (Section 2.1), so its outermost operator is the concatenation with the trailing literal 1, and the rule's s is that single literal rather than $0 \cdot 1$.

1. **Derive by 1.** At the outer node, $r = (0+1)^* \cdot 0$ and $s = 1$. Testing r for nullability, $n((0+1)^* \cdot 0) = n((0+1)^*) \wedge n(0) = \text{true} \wedge \text{false} = \text{false}$ by $\mathbf{N}_{\text{seq}}$, $\mathbf{N}_*$ and $\mathbf{N}_{\text{lit}}$: it does not match the empty word, since it still owes its trailing 0. So the non-nullable case applies and no second summand appears:

$$r_{01} \setminus 1 = (((0+1)^* \cdot 0) \setminus 1) \cdot 1 \quad \mathbf{D}_{\text{seq}}, \quad r = (0+1)^* \cdot 0, \quad s = 1 \quad (3.9)$$

The inner derivative is taken at the middle node, where $r = (0+1)^*$ and $s = 0$. That r is a star, so working it out needs $\mathbf{D}_*$ with star body $0+1$:

$$\begin{aligned} (0+1)^* \setminus 1 &= ((0+1) \setminus 1) \cdot (0+1)^* && \mathbf{D}_*, \quad r = 0+1 \\ &= ((0 \setminus 1) + (1 \setminus 1)) \cdot (0+1)^* && \mathbf{D}_+, \quad r = 0, \quad s = 1 \\ &= (\emptyset + \varepsilon) \cdot (0+1)^* && \mathbf{D}_{\text{lit}}, \text{ twice} \\ &= \varepsilon \cdot (0+1)^* && \mathbf{S}_3, \quad r = \varepsilon \\ &= (0+1)^* && \mathbf{S}_2, \quad r = (0+1)^* \end{aligned} \quad (3.10)$$

Substituting back into the middle node. Here $n((0+1)^*) = \text{true}$ by $\mathbf{N}_*$, so this time it *is* the nullable case, and a second summand does appear:

$$\begin{aligned} ((0+1)^* \cdot 0) \setminus 1 &= ((0+1)^* \setminus 1) \cdot 0 + (0 \setminus 1) \quad \mathbf{D}_{\text{seq}}^n, \quad r = (0+1)^*, \quad s = 0 \\ &= (0+1)^* \cdot 0 + \emptyset && \text{previous block; } \mathbf{D}_{\text{lit}} \\ &= (0+1)^* \cdot 0 && \mathbf{S}_3, \quad r = (0+1)^* \cdot 0 \end{aligned} \quad (3.11)$$

Note that $\mathbf{S}_3$ is stated as $\emptyset + r = r$ while the expression here is $r + \emptyset$; the identity is applied up to the commutativity of $+$, which **simplify** implements as a separate match arm for each side.

So, substituting once more into the outer node,

$$r_{01} \setminus 1 = ((0+1)^* \cdot 0) \cdot 1 = r_{01}. \quad (3.12)$$

A 1 that is not yet part of the final 01 loops back to the start, since $(0+1)^*$ can absorb it.

2. **Derive by 0.** The bindings are the same as in step 1, since the expression is unchanged. The computation is also the same with 0 and 1 exchanged, so $(0+1)^* \setminus 0$ again reduces to $(0+1)^*$, this time via $(\varepsilon + \emptyset)$ rather than $(\emptyset + \varepsilon)$. The one substantive difference is at the middle node's second summand:

$$\begin{aligned} ((0+1)^* \cdot 0) \setminus 0 &= ((0+1)^* \setminus 0) \cdot 0 + (0 \setminus 0) \quad \mathbf{D}_{\text{seq}}^n, \quad r = (0+1)^*, \quad s = 0 \\ &= (0+1)^* \cdot 0 + \varepsilon && \mathbf{D}_{\text{lit}}, \text{ since } 0 = 0 \end{aligned} \quad (3.13)$$

Here $\mathbf{D}_{\text{lit}}$ takes its $x = y$ branch and yields ε where step 1 got $\emptyset$, so no simplification applies: $\mathbf{S}_3$ needs an $\emptyset$ summand and $\mathbf{S}_4$ needs two identical summands, and neither holds. Substituting into the outer node, which is still the non-nullable case:

$$r_{01} \setminus 1 \setminus 0 = ((0+1)^* \cdot 0 + \varepsilon) \cdot 1. \quad (3.14)$$

It reads exactly as it should: after "10", either the trailing 01 is still owed in full (the $(0 + 1)^* \cdot 0$ summand), or the just-consumed 0 was already the first half of it (the ε summand); both possibilities stay open until the next symbol decides between them.

3. **Derive by 1.** The outer node's binding has changed: r is now $(0 + 1)^* \cdot 0 + \varepsilon$ and s is still 1. This r is nullable, since $n((0 + 1)^* \cdot 0 + \varepsilon) = n((0 + 1)^* \cdot 0) \vee n(\varepsilon) = \text{false} \vee \text{true} = \text{true}$ by $\mathbf{N}_+$ and $\mathbf{N}_\varepsilon$, so for the first time the outer node takes the nullable case:

$$(((0 + 1)^* \cdot 0 + \varepsilon) \cdot 1) \backslash 1 = (((0 + 1)^* \cdot 0 + \varepsilon) \backslash 1) \cdot 1 + (1 \backslash 1) \quad \mathbf{D}_{\text{seq}}^n \quad (3.15)$$

The inner derivative is now an alternation rather than a concatenation, so $\mathbf{D}_+$ applies, and its left operand is step 1's computation reused verbatim:

$$\begin{aligned} ((0 + 1)^* \cdot 0 + \varepsilon) \backslash 1 &= (((0 + 1)^* \cdot 0) \backslash 1) + (\varepsilon \backslash 1) \quad \mathbf{D}_+, \quad r = (0 + 1)^* \cdot 0, \quad s = \varepsilon \\ &= (0 + 1)^* \cdot 0 + \emptyset \quad \text{step 1; } \mathbf{D}_\varepsilon \\ &= (0 + 1)^* \cdot 0 \quad \mathbf{S}_3, \quad r = (0 + 1)^* \cdot 0 \end{aligned} \quad (3.16)$$

Substituting both parts back, and using $1 \backslash 1 = \varepsilon$ by $\mathbf{D}_{\text{lit}}$:

$$r_{01} \backslash 1 \backslash 0 \backslash 1 = ((0 + 1)^* \cdot 0) \cdot 1 + \varepsilon = r_{01} + \varepsilon. \quad (3.17)$$

$r_{01} + \varepsilon$ is nullable, since $n(r_{01} + \varepsilon) = n(r_{01}) \vee n(\varepsilon) = \text{false} \vee \text{true} = \text{true}$ by $\mathbf{N}_+$ and $\mathbf{N}_\varepsilon$, so $101 \in L(r_{01})$, correctly: 101 ends in 01. The final ε summand is exactly what witnesses this: it says the match was already complete the moment the third symbol was read, which is precisely what makes the whole expression nullable.

Now for only the word, $w = 10$. It shares its first two symbols with 101, so deriving r_{01} by 1 then by 0 produces the same intermediate expression as step 2 above, $((0 + 1)^* \cdot 0 + \varepsilon) \cdot 1$, but since 10 has only two symbols, this *is* the final result for this word, not an intermediate one. Checking its nullability directly: the expression is *not* nullable, since neither summand of $(0 + 1)^* \cdot 0 + \varepsilon$ makes the whole concatenation with 1 nullable: $(0 + 1)^* \cdot 0$ is not nullable, and $\varepsilon \cdot 1$ still needs the literal 1. So $10 \notin L(r_{01})$, correctly: 10 ends in 0, not 01.

`src/regex/nullable.rs` and `src/regex/deriv.rs` implement $n(r)$ and $r \backslash x$ case by case, one `Regex` variant at a time. `match_deriv` (`src/matchers/match_deriv.rs`) folds `deriv` over the input characters, applies `simplify` after every step exactly as annotated in the worked example above, and checks `nullable` on the result.

The sequence of derivatives $r_0 \xrightarrow{x_1} r_1 \xrightarrow{x_2} \dots \xrightarrow{x_n} r_n$ can equivalently be read as tracing a path through an automaton built lazily, on demand: each r_i is a state, and $\xrightarrow{x}$ is the transition relation. At every step there is exactly one current expression, playing the role of exactly one current automaton state, however large that state's internal syntax tree happens to be. This is a DFA-style view of the construction, one that Section 3.3 contrasts directly with an NFA-style one.

3.3 Antimirov Partial Derivatives

The derivative of Section 3.2 keeps the residual language as a single expression, combining every alternative reached so far with $+$. Antimirov's partial derivatives take a different

route to the same residual language: rather than combining the alternatives produced along the way, keep them apart as a *set* of expressions, one per strand of nondeterminism [Antimirov, 1996]. Writing $pd(r, x)$ for this set, Equation (3.18) defines it:

$$\begin{aligned}
pd(\emptyset, x) &= \{\} & (P_{\emptyset}) \\
pd(\varepsilon, x) &= \{\} & (P_{\varepsilon}) \\
pd(y, x) &= \begin{cases} \{\varepsilon\} & x = y \\ \{\} & x \neq y \end{cases} & (P_{\text{lit}}) \\
pd(r + s, x) &= pd(r, x) \cup pd(s, x) & (P_{+}) \\
pd(r^*, x) &= \{ smartC(r', r^*) \mid r' \in pd(r, x) \} & (P_{*}) \\
pd(r \cdot s, x) &= \begin{cases} \{ smartC(r', s) \mid r' \in pd(r, x) \} \cup pd(s, x) & n(r) = \text{true} \\ \{ smartC(r', s) \mid r' \in pd(r, x) \} & n(r) = \text{false} \end{cases} & \begin{matrix} (P_{\text{seq}}^n) \\ (P_{\text{seq}}) \end{matrix}
\end{aligned} \tag{3.18}$$

where the smart constructor, named SC when cited below, is

$$smartC(r', s) = \begin{cases} s & r' = \varepsilon \\ r' \cdot s & \text{otherwise} \end{cases} \tag{3.19}$$

and avoids introducing a spurious $\varepsilon \cdot s$ whenever a partial derivative of r is exactly ε . The rule names mirror Section 3.2's, so a P rule and the D rule with the same subscript are the two readings of the same case, and the derivations below cite them the same way, binding r and s explicitly at each application. This keeps every partial derivative a subexpression of r (up to the concatenations $smartC$ builds), rather than an expression that could grow without bound the way plain $r \setminus x$ can.

Worked example: single-symbol matching. Consider x^* against $\mathbf{xx}$ again, this time under pd . The base case gives $pd(x, \mathbf{x}) = \{\varepsilon\}$, and the star rule applies the smart constructor to it:

$$\begin{aligned}
pd(x^*, \mathbf{x}) &= \{ smartC(r', x^*) \mid r' \in pd(x, \mathbf{x}) \} & P_{*}, \text{ star body } r = x \\
&= \{ smartC(r', x^*) \mid r' \in \{\varepsilon\} \} & P_{\text{lit}}, \text{ since } \mathbf{x} = x \\
&= \{ smartC(\varepsilon, x^*) \} & \\
&= \{x^*\} & \text{SC, } r' = \varepsilon, s = x^*
\end{aligned} \tag{3.20}$$

Deriving again by $\mathbf{x}$ repeats exactly the same computation, because the set to derive is still $\{x^*\}$: $pd(x^*, \mathbf{x}, \mathbf{x}) = \{x^*\}$. This set contains x^* , which is nullable, so x^* matches $\mathbf{xx}$, correctly, which is the same conclusion as the derivative trace, reached here in two steps that never leave $\{x^*\}$. Unlike that trace, which passed through an expression carrying an $\emptyset$ and an ε neither one needed, before collapsing back down, pd never produces either kind of clutter to begin with: $pd(\emptyset, x) = \{\}$ discards empty alternatives the moment they arise, and $smartC$ discards a spurious $\varepsilon \cdot s$ the moment it would be built. No separate simplification pass is needed for this to hold, it follows directly from how pd and $smartC$ are defined, not from an additional rewriting step applied afterwards.

Worked example: $r_{01} = (0 + 1)^* \cdot 0 \cdot 1$ Derive r_{01} against $w = 101$ again, this time tracking a set of residuals instead of one expression.

1. **Derive by 1.** The bindings are the same three concatenation nodes as in Section 3.2's table, and the work proceeds innermost first. Start with the star body, where P_+ binds $r = 0$ and $s = 1$:

$$\begin{aligned} pd(0+1, 1) &= pd(0, 1) \cup pd(1, 1) && P_+, r = 0, s = 1 \\ &= \{\} \cup \{\varepsilon\} && P_{lit}, \text{ both branches} \\ &= \{\varepsilon\} \end{aligned} \quad (3.21)$$

Lifting that through the star, whose body is $r = 0 + 1$:

$$\begin{aligned} pd((0 + 1)^*, 1) &= \{ smartC(r', (0 + 1)^*) \mid r' \in \{\varepsilon\} \} && P_*, r = 0 + 1 \\ &= \{(0 + 1)^*\} && SC, r' = \varepsilon \end{aligned} \quad (3.22)$$

At the middle node $r = (0 + 1)^*$ and $s = 0$, and $n((0 + 1)^*) = \text{true}$ by N_* , so the nullable case applies and a union with $pd(s, x)$ appears:

$$\begin{aligned} pd((0 + 1)^* \cdot 0, 1) &= \{ smartC(r', 0) \mid r' \in pd((0 + 1)^*, 1) \} \cup pd(0, 1) && P_{seq}^n \\ &= \{ smartC((0 + 1)^*, 0) \} \cup \{\} && \text{above; } P_{lit} \\ &= \{(0 + 1)^* \cdot 0\} && SC, r' \neq \varepsilon \end{aligned} \quad (3.23)$$

Note that **SC** took its second branch here, building the concatenation, because $r' = (0 + 1)^*$ is not ε . Finally the outer node, where $r = (0 + 1)^* \cdot 0$ and $s = 1$. That r is not nullable, so no union is added:

$$\begin{aligned} pd(r_{01}, 1) &= \{ smartC(r', 1) \mid r' \in \{(0 + 1)^* \cdot 0\} \} && P_{seq}, r = (0 + 1)^* \cdot 0, s = 1 \\ &= \{(0 + 1)^* \cdot 0 \cdot 1\} = \{r_{01}\} && SC \end{aligned} \quad (3.24)$$

A single residual, identical to r_{01} itself: reading a 1 that is not yet part of the final 01 loops back to the start, exactly as in the derivative trace, but here as a one-element set, rather than as one expression equal to r_{01} .

2. **Derive by 0.** The set is still $\{r_{01}\}$, so derive r_{01} by 0 with the same three bindings. The star computation runs as before and again gives $pd((0 + 1)^*, 0) = \{(0 + 1)^*\}$. The difference is at the middle node's second operand, where P_{lit} now takes its $x = y$ branch:

$$\begin{aligned} pd((0 + 1)^* \cdot 0, 0) &= \{ smartC((0 + 1)^*, 0) \} \cup pd(0, 0) && P_{seq}^n, r = (0 + 1)^*, s = 0 \\ &= \{(0 + 1)^* \cdot 0\} \cup \{\varepsilon\} && P_{lit}, \text{ since } 0 = 0 \\ &= \{(0 + 1)^* \cdot 0, \varepsilon\} \end{aligned} \quad (3.25)$$

Write $P = \{(0 + 1)^* \cdot 0, \varepsilon\}$ for the set just computed. Substituting it into the outer node, **SC** now takes a different branch for each of its two elements:

$$\begin{aligned} pd(r_{01}, 1, 0) &= \{ smartC(r', 1) \mid r' \in P \} && P_{seq}, s = 1 \\ &= \{(0 + 1)^* \cdot 0 \cdot 1, 1\} && SC: \text{ second branch, then first} \\ &= \{r_{01}, 1\} \end{aligned} \quad (3.26)$$

The second element is the literal 1 rather than $\varepsilon \cdot 1$ precisely because SC recognized $r' = \varepsilon$ and returned s untouched.

This is the same information as $((0 + 1)^* \cdot 0 + \varepsilon) \cdot 1$ from the derivative trace. Both say "either the trailing 01 is still owed in full, or the just-consumed 0 was already its first half", but where the derivative folds the two possibilities into one expression with an internal $+$, pd keeps them apart as two separate residuals, r_{01} and the literal 1, with no $+$ at all. This is the concrete sense in which pd is NFA-shaped: the two-element set is exactly the state set an NFA for r_{01} would be in after reading 10, one state per strand, rather than one DFA state encoding both strands internally.

3. **Derive by 1.** Each element of $\{r_{01}, 1\}$ is derived separately and the results unioned. The first repeats step 1 exactly; the second is a bare literal, so only P_{lit} is needed:

$$\begin{aligned} pd(r_{01}, 1, 0, 1) &= pd(r_{01}, 1) \cup pd(1, 1) && \text{union over the frontier} \\ &= \{r_{01}\} \cup \{\varepsilon\} && \text{step 1; } P_{lit} \\ &= \{r_{01}, \varepsilon\} \end{aligned} \tag{3.27}$$

This set contains ε , so it contains a nullable residual, and $101 \in L(r_{01})$ the same conclusion as the derivative trace, reached here by checking whether any residual currently held is nullable, rather than by checking nullability of one combined expression.

Now for $w = 10$, the same way the derivative trace did: it shares its first two symbols with 101, so deriving by 1 then 0 (step 2 above) reaches the same set, $\{r_{01}, 1\}$ and since 10 has only two symbols, this *is* the final set for this word. Neither element is nullable: $n(r_{01}) = \text{false}$, and the literal 1 is nullable only once it has been derived into ε , not as itself. So $10 \notin L(r_{01})$, correctly.

The set of all partial derivatives reachable from a fixed r by derivation with respect to arbitrary words is always finite, bounded by the number of literal-symbol occurrences in r [Antimirov, 1996]. r_{01} has four such occurrences, the two inside $(0 + 1)^*$, and one each for the trailing 0 and 1, so at most four distinct partial derivatives (plus r_{01} itself) can ever appear, no matter how long a word r_{01} is derived with respect to. The worked example above already shows three of those residuals directly: every set encountered, $\{r_{01}\}$, $\{r_{01}, 1\}$, and $\{r_{01}, \varepsilon\}$, is built entirely from r_{01} , the literal 1, and ε . Ordinary derivatives are also finite up to language equivalence. This is Brzozowski [1964]’s own finiteness result, the one that makes the DFA reading at the end of Section 3.2 possible in the first place, but only once the simplification identities of Section 3.2 are applied to identify equivalent expressions; the partial-derivative bound holds already at the syntactic level, with no such identification needed and the same point the single-symbol example above made concretely for x^* . The two bounds also differ in kind, not just in what they need to hold: Brzozowski’s own bound on the number of derivative types is exponential in the structure of r [Brzozowski, 1964, Theorem 4.3], while Antimirov’s $\|r\| + 1$ bound on partial derivatives is linear in the number of literal occurrences [Antimirov, 1996, Theorem 3.4, p. 309].

pd is implemented inside `match_pderiv` (`src/matchers/match_pderiv.rs`), which mirrors the worked example above directly: rather than tracking one expression, it maintains a `HashSet<Regex>` [The Rust Project Developers, 2026h,f] of currently-live residuals, replacing that set at each character with the union of $pd(r', x)$ over every residual r' currently in it, and accepting once the input is exhausted if any surviving residual is nullable, which

is exactly the frontier $\{r_{01}\} \rightarrow \{r_{01}, 1\} \rightarrow \{r_{01}, \varepsilon\}$ traced by hand above. This is subset simulation of an NFA whose states are subexpressions of r_{01} , run directly on the syntax rather than on a separately constructed automaton.

3.4 Matcher Selection

`match_naive`, `match_deriv`, and `match_pderiv` share the signature `fn(&str, &Regex) -> bool` despite implementing three structurally different algorithms: exhaustive search over substring splits, a single evolving DFA-style expression, and an evolving NFA-style set of expressions. This uniform interface makes the three matchers directly interchangeable: the same regular expression and input can be run through any of them, and any disagreement between their answers points to a bug in one of the three independently-written implementations rather than to an ambiguity in what the correct answer is supposed to be. The same pattern of comparing independently-derived implementations is reused throughout this thesis for parsers rather than matchers, wherever multiple constructions are expected to agree.

`MatcherType` (`src/matchers/selection.rs`) is an enum with one variant per matcher and a `matcher()` method returning the corresponding function pointer. Its purpose is purely to give the `regex-engine` command-line tool (`src/cli/matcher.rs`) a single place that maps a `--matcher` flag value (`naive`, `deriv`, `pderiv`, or `all`, defaulting to `deriv`) on the `match` subcommand to the matcher it names, rather than every call site duplicating that mapping itself. All three matchers can be compared side by side on an arbitrary input through the CLI directly, which is how the two worked examples of this chapter can be checked end to end rather than only by hand. Running each of x^* against `xx`, and r_{01} against `101` and `10`, with `--matcher all` gives:

Example	Word	naive	deriv	pderiv
x^*	<code>xx</code>	true	true	true
r_{01}	<code>101</code>	true	true	true
r_{01}	<code>10</code>	false	false	false

```
cargo run -- match "(0|1)*01" "101" --matcher all
```

prints `true` for all three matchers and reproduces the second row directly. This agrees with the hand-derived conclusions of Sections 3.2 and 3.3 in every cell: x^* accepts `xx`, r_{01} accepts `101`, and r_{01} rejects `10` and all three independently-written matchers reach each conclusion the same way. The same table-plus-CLI-invocation pattern is used again in Chapters 5 and 6 to check the parsers' worked examples against the `parse` subcommand instead of `match`.

Running these through the CLI does, however, lean on more than the three matchers themselves. `match_naive`, `match_deriv`, and `match_pderiv` take only an already-built `Regex` value and their shared signature is `fn(&str, &Regex) -> bool`, with no path from a string like `"(0|1)*01"` to a `Regex` anywhere in them. That translation is entirely the frontend's purpose: the `match` subcommand (`src/cli/matcher.rs`) calls `frontend::parse_pattern` *before* it calls whichever matcher `--matcher` selected, turning the concrete syntax into the `Regex` AST of Section 2.3 first. Chapter 7 covers that parsing and translation in detail.

The POSIX parsers built in Chapters 5 and 6 follow the same pattern one level up:

rather than a Boolean, each parser returns a parse tree, but all of them remain selectable through a single uniform interface and are expected to agree with one another wherever they overlap.

Matching versus Parsing

Chapter 3 established matching with a Boolean answer: whether $w \in L(r)$, the word is either accepted or rejected. Parsing refines this question by asking for the derivation that witnesses the match. Parse trees provide a representation of such derivations, recording the choices made by the structure of r while consuming w .

This refinement introduces ambiguity: a fixed r and w may admit multiple parse trees. A parser must therefore combine the semantics of matching with a *disambiguation policy* that selects one of them. This chapter introduces parse trees, examines Greedy and POSIX disambiguation, and establishes the ordering that the parsers developed later must compute.

4.1 The Matching Problem

The three matchers of Chapter 3 all answer the same Boolean question: given r and w , is $w \in L(r)$? Several applications need more than that answer. For example knowing *which alternative* of r matched, and knowing *which part* of w a given subexpression matched. Boolean recognition discards both kinds of information: it confirms that some derivation of the match exists, but nothing about what that derivation actually was [Frisch and Cardelli, 2004].

Which alternative: lexing and schema validation. Consider a lexer for a programming language, using the pattern `if|[a-z]+` to recognize a single token that could be the keyword `if`, marking the start of a conditional statement, or an ordinary lowercase identifier, naming a variable or function. Against the input `"if"`, both readings apply: the string is the literal keyword, but it is also a valid identifier, since it consists only of lowercase letters. A Boolean matcher confirms only that the pattern matches `"if"`; it does not say which of the two readings was taken, and the two readings are not interchangeable downstream. In a statement such as `if (x > 0) foo();`, the parser needs an `IF` token to recognize a conditional. If the lexer instead reported an identifier, `IDENT("if")`, the parser would try to read `if` as an expression, and the conditional would never be recognized.

In practice, this particular ambiguity is resolved before parsing ever begins, not during it. Most languages forbid identifiers from being spelled the same as a keyword, and lexer generators enforce this directly: flex always matches the longest possible token, and when two rules match input of the same length, the rule listed earlier in the specification wins [The Flex Project, 2026]. Placing the keyword pattern for `if` before the general identifier pattern is therefore enough to guarantee the keyword reading whenever both would

otherwise apply, and "if" can never be tokenized as an identifier at all. The underlying problem, that a Boolean matcher cannot report which alternative explains a match, has not gone away; it has been settled in advance by a rule the tool applies uniformly, rather than recovered from any particular match. That rule is already a disambiguation policy in miniature, of exactly the kind Section 4.3.1 and Section 4.3.3 formalize later in this chapter.

Schema validation avoids the same problem whenever a schema declares a field's type up front: checking a value declared as an integer against the pattern for integers never raises the question of whether it might also be a Boolean, since no other pattern is ever consulted. The problem reappears in formats that infer a value's type from its text instead of from a declared schema. YAML is the clearest documented case. A plain, unquoted scalar is resolved to a type by checking it against an ordered set of patterns, one for null, one for booleans, one for integers, one for floats, and so on, with the type belonging to the first pattern that matches [Ben-Kiki et al., 2005]. In YAML 1.1, the boolean pattern accepts not only `true` and `false` but also `yes`, `no`, `on`, `off`, and several capitalizations of each, so the two-letter ISO country code for Norway, `NO`, is resolved to the Boolean value `false` rather than the string `"NO"` in any YAML document that does not quote it explicitly. This became widely known as the "Norway problem," and it is exactly the failure this section describes: the text `NO` matches both the boolean pattern and, trivially, the fallback string pattern, and only the fixed order in which YAML checks its candidate types, not anything about the text `NO` itself, decides which one wins [HitchDev, 2026]. YAML 1.2 removed the `yes/no/on/off` boolean forms specifically to shrink this overlap, though widely used implementations such as PyYAML still default to the 1.1 rules.

A pattern where the two readings genuinely compete. The "if" example is resolved by a tie-breaking convention. Both alternatives of `if|[a-z]+` match the same two-character input, so a longest-match rule cannot distinguish them. The lexical specification therefore gives the keyword alternative priority. What appears to be an obvious choice is thus the result of a specific disambiguation rule: the two alternatives consume equally long substrings, and the earlier alternative wins the tie.

The pattern $(a+ab)(b+\varepsilon)$ against `"ab"` presents a different situation. The two alternatives of $a + ab$ consume different amounts of input. The `ab` alternative can consume both characters, after which $(b + \varepsilon)$ matches ε . Alternatively, the `a` alternative consumes only the first character, leaving `b` to be matched by the `b` alternative of $(b + \varepsilon)$. Both choices produce a complete and valid match of the same input.

Here, two natural disambiguation policies give different results. A policy that prefers the longer match selects the `ab` alternative, whereas a policy that gives priority to the earlier alternative selects `a`. Unlike the "if" example, neither choice can be dismissed as merely resolving a tie. The expression admits both parses, and selecting between them requires an explicit policy.

Before such choices can be compared, however, the possible parses must be made explicit. A Boolean result records only that a match exists; it does not record the structural choices through which the match was obtained. Section 4.2 represents these choices as *parse trees* for a structured *witness* to a match: it records how the structure of r was used to consume w . When multiple parse trees witness the same match, the parser must determine which one to return.

4.2 Parse Trees as Matching Evidence

Sulzmann and Lu formalize such a witness by viewing regular expressions as types and parse trees as the values inhabiting them [Sulzmann and Lu, 2014]. A parse tree v is built from the same constructors used informally in Section 2.2’s definition of $L(r)$: a pair for concatenation, a tagged left or right injection for alternation, and a list for repetition. A judgment $\vdash v : r$, defined by structural recursion on both v and r , states that v is a valid parse tree for r , by the rules of Equation (4.1):

$$\begin{array}{c}
\vdash () : \varepsilon \quad (\mathsf{T}_\varepsilon) \qquad \frac{l \in \Sigma}{\vdash l : l} \quad (\mathsf{T}_{\text{lit}}) \\
\\
\frac{\vdash v_1 : r_1 \quad \vdash v_2 : r_2}{\vdash (v_1, v_2) : r_1 r_2} \quad (\mathsf{T}_{\text{seq}}) \quad \frac{\vdash v_1 : r_1}{\vdash \text{Left } v_1 : r_1 + r_2} \quad (\mathsf{T}_+^L) \quad \frac{\vdash v_2 : r_2}{\vdash \text{Right } v_2 : r_1 + r_2} \quad (\mathsf{T}_+^R) \\
\\
\frac{}{\vdash [] : r^*} \quad (\mathsf{T}_*^0) \qquad \frac{\vdash v : r \quad \vdash vs : r^*}{\vdash (v : vs) : r^*} \quad (\mathsf{T}_*)
\end{array} \tag{4.1}$$

As with the derivative rules of Section 3.2, each rule carries a name, and the derivations in this chapter cite those names together with the binding of each metavariable, so that it is always visible which subexpression of r a given r_1 or r_2 refers to.

A *flattening* function $|v|$ reads the word represented by a parse tree by structural recursion:

$$\begin{aligned}
|()| &= \varepsilon, & |l| &= l, & |[]| &= \varepsilon, \\
|(v_1, v_2)| &= |v_1||v_2|, & |\text{Left } v| &= |v|, & |\text{Right } v| &= |v|, \\
|v : vs| &= |v||vs|.
\end{aligned} \tag{4.2}$$

The key property is that the parse trees characterize the language:

$$L(r) = \{ |v| \mid \vdash v : r \}. \tag{4.3}$$

Thus, a parse tree together with its derivation provides a structured *witness* that $w \in L(r)$: it records not only that the match exists, but the choices made by the regular expression while producing w .

Unambiguous example. For an unambiguous match, this witness is unique. For $r = a \cdot b$ and $w = \mathbf{ab}$, only one derivation is possible. Matching r against T_{seq} ’s conclusion binds $r_1 = a$ and $r_2 = b$, so the rule requires $\vdash v_1 : a$ and $\vdash v_2 : b$. Each of those is a literal, and T_{lit} is the only rule whose conclusion matches a literal, so $v_1 = a$ and $v_2 = b$ with no choice anywhere:

$$\frac{\frac{a \in \Sigma}{\vdash a : a} (\mathsf{T}_{\text{lit}}) \quad \frac{b \in \Sigma}{\vdash b : b} (\mathsf{T}_{\text{lit}})}{\vdash (a, b) : ab} (\mathsf{T}_{\text{seq}}) \tag{4.4}$$

This is the only possible derivation, yielding $v = (a, b)$, and flattening it recovers the input:

$$\begin{aligned}
|(a, b)| &= |a| |b| && \text{flattening of a pair, } v_1 = a, v_2 = b \\
&= \mathbf{ab} && \text{flattening of a literal, twice}
\end{aligned} \tag{4.5}$$

When the structure of the expression permits different choices, the witness itself can be ambiguous. The first two examples below are adapted from the running examples of [Sulzmann and Lu \[2014\]](#) and are reproduced in this project's test suite as `paper_r1` and `paper_r2`. They are included here because the same cases are used later to illustrate and evaluate the parser implementations developed in Chapters 5 and 6. The derivations given here provide a reference for that later discussion. Two further examples, drawn from this project's own test suite, follow.

Ambiguous example 1: concatenation. Consider $r_1 = (a + ab)(b + \varepsilon)$ against $w = \mathbf{ab}$. One valid derivation chooses the second alternative of $a + ab$, which consumes both characters, and the empty alternative of $b + \varepsilon$:

$$\begin{array}{c}
\frac{\frac{\vdash a : a \quad \vdash b : b}{\vdash (a, b) : ab}}{\vdash \text{Right}(a, b) : a + ab} \quad \frac{\vdash () : \varepsilon}{\vdash \text{Right}() : b + \varepsilon}
\end{array} \tag{4.6}$$

and therefore

$$\vdash (\text{Right}(a, b), \text{Right}()) : (a + ab)(b + \varepsilon). \tag{4.7}$$

The corresponding parse tree is

$$v = (\text{Right}(a, b), \text{Right}()), \tag{4.8}$$

with

$$|v| = |(a, b)| |()| = \mathbf{ab} \varepsilon = \mathbf{ab}. \tag{4.9}$$

This is not the only witness. The first alternative of $a + ab$ can instead consume **a**, leaving **b** for the first alternative of $b + \varepsilon$:

$$\frac{\vdash a : a}{\vdash \text{Left } a : a + ab} \quad \frac{\vdash b : b}{\vdash \text{Left } b : b + \varepsilon}
\end{array} \tag{4.10}$$

and therefore

$$\vdash (\text{Left } a, \text{Left } b) : (a + ab)(b + \varepsilon). \tag{4.11}$$

The corresponding parse tree is

$$v' = (\text{Left } a, \text{Left } b), \tag{4.12}$$

with

$$|v'| = |a| |b| = \mathbf{ab}. \quad (4.13)$$

Thus the same expression r_1 and word w have two distinct parse trees, v and v' , satisfying

$$|v| = |v'| = \mathbf{ab}. \quad (4.14)$$

Ambiguous example 2: repetition. The same phenomenon can arise from repetition rather than concatenation. Consider $r_2 = (a + b + ab)^*$ against $w = \mathbf{ab}$, nesting the alternation as $a + (b + ab)$ (`paper_r2`¹ in the test suite). One derivation uses a single iteration, taking the $\mathbf{ab}$ alternative directly:

$$\frac{\frac{\frac{\vdash a : a \quad \vdash b : b}{\vdash (a, b) : ab}}{\vdash \text{Right}(a, b) : b + ab}}{\vdash \text{Right } \text{Right}(a, b) : a + (b + ab)} \quad (4.15)$$

which becomes a one-element list via the star's constructor rule:

$$\vdash [\text{Right } \text{Right}(a, b)] : (a + (b + ab))^*. \quad (4.16)$$

The corresponding parse tree is

$$v_2 = [\text{Right } \text{Right}(a, b)], \quad |v_2| = |(a, b)| = \mathbf{ab}. \quad (4.17)$$

A second derivation instead uses two iterations, matching $\mathbf{a}$ and $\mathbf{b}$ through separate alternatives of the star's body:

$$\frac{\vdash a : a}{\vdash \text{Left } a : a + (b + ab)} \quad \frac{\frac{\vdash b : b}{\vdash \text{Left } b : b + ab}}{\vdash \text{Right } \text{Left } b : a + (b + ab)} \quad (4.18)$$

which combine, one iteration at a time, into

$$\vdash [\text{Left } a, \text{Right } \text{Left } b] : (a + (b + ab))^*. \quad (4.19)$$

The corresponding parse tree, with two iterations of the star, is

$$v'_2 = [\text{Left } a, \text{Right } \text{Left } b], \quad |v'_2| = |a| |b| = \mathbf{ab}. \quad (4.20)$$

As with r_1 , both v_2 and v'_2 witness the same match: ambiguity here comes from how many iterations the star takes, not from which top-level alternative fires.

¹The associativity of $+$ is immaterial to $L(r)$ and to this example.

Ambiguous example 3: duplicate alternatives. Concatenation and repetition are not the only sources of ambiguity. Consider $r_3 = (a + a)^*$ against $w = \mathbf{aa}$. Here the star’s body contains two syntactically distinct but semantically identical alternatives for the same literal. This is an instance of `ambiguous_star_dedup_matches_bitcoded` in `src/parsers/pderiv_std/parse.rs`².

Each of the two positions in w can independently be matched by either the left or the right copy of $\mathbf{a}$, giving four distinct parse trees for this two-character input alone:

$$[Left\ a, Left\ a], \quad [Left\ a, Right\ a], \quad [Right\ a, Left\ a], \quad [Right\ a, Right\ a], \quad (4.21)$$

all flattening to $\mathbf{aa}$. Nothing about *Left a* and *Right a* distinguishes them semantically; the ambiguity is entirely an artifact of writing the same alternative twice. This kind of duplication is particularly relevant to the partial-derivative parser, where distinct syntactic paths can produce equivalent residual matches. The deduplication mechanisms discussed in Chapter 6 prevent such redundant paths from accumulating in the parse frontier.

Ambiguous example 4: an IPv4 pattern. The previous examples isolate particular sources of ambiguity. The same phenomenon also occurs in a pattern taken from practice: a regular expression for dotted-quad IPv4 addresses from the Snort intrusion-detection dataset studied by Mamouras et al. [2024]. The pattern is included in this project’s test suite as `snort_ip_regex`³.

$$((\backslash\mathbf{d}[1-9]\backslash\mathbf{d}|1\backslash\mathbf{d}\backslash\mathbf{d}|2[0-4]\backslash\mathbf{d}|25[0-5])\backslash.)\{3\}$$

$$(\backslash\mathbf{d}[1-9]\backslash\mathbf{d}|1\backslash\mathbf{d}\backslash\mathbf{d}|2[0-4]\backslash\mathbf{d}|25[0-5])$$

using the class and repetition shorthands of Section 7.3, under which $\backslash\mathbf{d}$ desugars to a ten-way alternation of the literals 0 through 9 and $\{3\}$ desugars to three concatenated copies of its operand. The pattern concatenates four copies of a shared three-character octet pattern, separated by literal dots. Write $O = A + B + C + D$ for this shared four-way alternation, with

$$A = \backslash\mathbf{d}[1-9]\backslash\mathbf{d} \quad B = 1\backslash\mathbf{d}\backslash\mathbf{d} \quad C = 2[0-4]\backslash\mathbf{d} \quad D = 25[0-5],$$

left-associated as $((A + B) + C) + D$, so that the full pattern is $(O.)^3O$ with the final dot omitted. Already at the level of a single octet, O is ambiguous on some inputs in exactly the sense of r_1 : against $w = 239$, alternative A matches, reading the digits as *any digit, a digit from 1 to 9, any digit*; but so does alternative C , reading 2 as the literal 2, 3 as a digit from 0 to 4, and 9 as any digit (B and D both fail, since w does not begin with 1 or 25). Both A and C consume all three characters of w . Thus, unlike r_3 , where the alternatives are semantically identical, the ambiguity here is between two genuinely different alternatives that happen to match the same substring. Mamouras et al. [2024] use this same pattern to illustrate a different symptom of the same underlying policy difference: under unanchored substring search, Greedy’s leftmost-first matching can return a shorter, truncated address where POSIX returns the complete one, a distinction in match *span* rather than, as here, in which of two total-match parse trees is selected.

²The test uses a four-character input; the two-character case shown here exhibits the same phenomenon.

³`tests/test_frontend.rs`.

4.3 Disambiguation Policies

The parse-tree judgment identifies every valid witness, but it does not say which witness a parser should return. A *disambiguation policy* adds this missing choice. Given a regular expression r and word w , the policy selects one parse tree v satisfying

$$\vdash v : r \quad \text{and} \quad |v| = w. \quad (4.22)$$

The four ambiguous examples above illustrate why the policy matters, and in more than one way. For r_1 and **ab**, one witness chooses **a** before **b**, while the other assigns the whole input to **ab**. For r_2 and **ab**, one witness uses two iterations and the other uses one. For r_3 and **aa**, four witnesses differ only in which of two identical alternatives each position is attributed to. For O and **239**, two alternatives with different content, not merely different labels, both happen to consume the whole octet. Different matching conventions select different witnesses, and, as r_3 and O show, a policy also has to say something about witnesses that agree on every character consumed and differ only in which alternative gets the credit.

The two policies considered here resolve these choices differently. Greedy matching follows the structural priority of the expression, whereas POSIX matching gives priority to longer leftmost submatches. Their difference becomes clear when they are applied to the same ambiguous expression.

4.3.1 Greedy Leftmost Matching

The policy associated with conventional backtracking matchers is *Greedy*. Alternatives are considered from left to right, and the first alternative that leads to an overall match is selected. Within a Kleene star, another iteration is preferred to stopping whenever continuing the repetition can still lead to an overall match [Frisch and Cardelli, 2004].

For alternatives, the defining preference is structural: the left alternative is preferred regardless of how much input the two alternatives consume. In an ordering notation, the essential rule is

$$Left\ v \succ_{r_1+r_2} Right\ v'. \quad (4.23)$$

Every other comparison rule: (C1) and (C2) for concatenation, (A3) and (A4) for continuing a comparison once both alternatives already agree on a branch, and (K1)–(K4) for repetition is shared, unchanged, between Greedy and POSIX; the two orderings differ only in how they compare two alternatives that choose *different* branches, where POSIX replaces the rule above with the length-comparing pair (A1) and (A2) of Section 4.3.3. Sulzmann and Lu construct their POSIX ordering explicitly as this one substitution into Frisch and Cardelli’s Greedy ordering [Frisch and Cardelli, 2004, Sulzmann and Lu, 2014], leaving every other rule untouched.

Applied to r_1 . Greedy first tries the **a** alternative. It consumes the leading **a**, and the remaining **b** is successfully matched by the first alternative of $b + \varepsilon$. Since the complete match succeeds, there is no need to try the **ab** alternative. The Greedy result is therefore

$$v' = (\textit{Left } a, \textit{Left } b). \quad (4.24)$$

Applied to r_2 . Greedy similarly begins with the first alternative, matching **a**. The star then attempts another iteration, in which the first alternative fails on **b** and the second alternative succeeds. The resulting parse uses two iterations:

$$v'_2 = [\textit{Left } a, \textit{Right } \textit{Left } b]. \quad (4.25)$$

The Greedy result therefore depends directly on the order in which the regular expression presents its alternatives.

Applied to r_3 . At each of the two positions, $\textit{Left } v \succ_{a+a} \textit{Right } v'$ applies directly, regardless of length, since both alternatives consume the same single character. Greedy therefore selects the left alternative every time:

$$[\textit{Left } a, \textit{Left } a]. \quad (4.26)$$

Unlike r_1 and r_2 , this result is not decided by which alternative matches more of the input; every alternative here matches exactly one character. It is decided purely by position in the pattern, the same kind of tie-breaking that Section 4.1 showed resolving "if" against `if|[a-z]+`.

Applied to O . Against $w = 239$, Greedy tries O 's alternatives left to right: A , then B , then C , then D . Alternative A matches the leading digit 2 against `\d`, the digit 3 against `[1-9]`, and the digit 9 against `\d`, consuming all three characters. Since this already produces a complete match, Greedy never tries B , C , or D , even though C would also have matched. The Greedy result selects A , exactly as it selected the left alternative of r_1 and of r_3 : once an alternative leads to a full match, later alternatives are never even attempted.

Procedural versus declarative view. The description above gives a procedural intuition for Greedy matching: alternatives are considered from left to right, and a choice is retained when it can be extended to a complete match. The relation $\succ_{r_1+r_2}$ used throughout this subsection is a different kind of thing: a declarative ordering on the parse trees admitted by r and w , generalized to arbitrary r as $\succ_r$ in Section 4.3.3. Frisch and Cardelli [2004], whose ordering this one adapts, are explicit that these two views are not the same thing computationally: a backtracking procedure computing their ordering's maximum is possible, but they describe it as "quite inefficient because of backtracking," and spend the rest of their paper building a two-pass, backtracking-free algorithm instead, with a proof that it computes the same value. Neither `pderiv_std` nor `pderiv_bc` (Chapter 6) backtracks either, for the same reason: both compute the Greedy answer through a forward partial-derivative frontier with first-occurrence-wins deduplication. The distinction matters concretely in Section 4.3.2, where the ordering can have no maximal element at all, even though a concrete, non-backtracking parser can still compute the intended Greedy result.

4.3.2 The Infinite Empty-Match Chain Problem

Greedy’s preference for continuing a repetition creates a problem when the repeated expression can match the empty word. Consider ε^* against the empty input. The following parse trees are all valid:

$$v_0 = [], \quad v_1 = [()], \quad v_2 = [(), ()], \quad \dots \quad (4.27)$$

Each additional iteration still produces the same flattened word ε . Under the Greedy ordering, every additional iteration is preferred:

$$v_0 \prec_{\varepsilon^*} v_1 \prec_{\varepsilon^*} v_2 \prec_{\varepsilon^*} \dots \quad (4.28)$$

There is therefore no maximal parse tree. The problem is not that the language is ill-defined; ε is clearly a valid match. The problem is that the Greedy preference does not identify a largest witness when empty iterations can be added indefinitely.

A slightly less degenerate example is $(\varepsilon + a)^*$ against **a**. The parse trees

$$\begin{aligned} v_0 &= [Right\ a], \\ v_1 &= [Left\ (), Right\ a], \\ v_2 &= [Left\ (), Left\ (), Right\ a], \\ &\vdots \end{aligned} \quad (4.29)$$

all flatten to **a**. Greedy prefers each additional empty iteration, producing another infinite ascending chain.

The same pathology reappears whenever a star’s body is itself nullable, not just when the body is ε outright. Take $(a^*)^*$ against **aaa**: since the inner a^* can always match ε , an outer iteration contributing nothing is always available, and Greedy always prefers taking it:

$$[[a, a, a]] \prec_{(a^*)^*} [[a, a, a], []] \prec_{(a^*)^*} [[a, a, a], [], []] \prec_{(a^*)^*} \dots, \quad (4.30)$$

another infinite ascending chain, this time built on top of an otherwise unambiguous inner match rather than on ε itself. Nested, mutually nullable stars of this shape are also the classic trigger for catastrophic backtracking in engines that search for a Greedy-maximal match directly [Cox, 2007]; the ordering-theoretic failure identified here is the same failure that shows up operationally as exponential blowup in those engines. The frontier-based Greedy parsers of Chapter 6 do not search for an ordering maximum at all, so this particular failure mode does not resurface there; Section 4.3.3 returns to $(a^*)^*$ once POSIX’s rule (K1) is available, to show why POSIX does not have this problem either.

This exposes a limitation of the Greedy ordering: without additional restrictions or special handling of empty matches, a maximal parse tree need not exist. The POSIX ordering avoids this problem by treating empty matches explicitly.

4.3.3 POSIX Longest-Leftmost Matching

POSIX uses a different disambiguation policy. Subpatterns are compared according to the length of the substrings they match, with subpatterns that occur earlier in the regular expression taking priority. The standard states that, consistent with the whole match being the longest of the leftmost matches, each subpattern, from left to right, shall match the longest possible string [IEEE and The Open Group, 2017].

Sulzmann and Lu formalize this policy as an ordering on parse trees [Sulzmann and Lu, 2014]. Let $\text{len}(w)$ denote the number of characters in w . The ordering is defined by the rules of Equation (4.31):

$$\begin{aligned}
\text{(C1)} \quad & \frac{v_1 = v'_1 \quad v_2 \succ_{r_2} v'_2}{(v_1, v_2) \succ_{r_1 r_2} (v'_1, v'_2)} \\
\text{(C2)} \quad & \frac{v_1 \succ_{r_1} v'_1}{(v_1, v_2) \succ_{r_1 r_2} (v'_1, v'_2)} \\
\text{(A1)} \quad & \frac{\text{len}(|v_2|) > \text{len}(|v_1|)}{\text{Right } v_2 \succ_{r_1+r_2} \text{Left } v_1} \\
\text{(A2)} \quad & \frac{\text{len}(|v_1|) \geq \text{len}(|v_2|)}{\text{Left } v_1 \succ_{r_1+r_2} \text{Right } v_2} \\
\text{(A3)} \quad & \frac{v_2 \succ_{r_2} v'_2}{\text{Right } v_2 \succ_{r_1+r_2} \text{Right } v'_2} \\
\text{(A4)} \quad & \frac{v_1 \succ_{r_1} v'_1}{\text{Left } v_1 \succ_{r_1+r_2} \text{Left } v'_1} \\
\text{(K1)} \quad & \frac{|v : vs| = \varepsilon}{[] \succ_{r^*} v : vs} \\
\text{(K2)} \quad & \frac{|v : vs| \neq \varepsilon}{v : vs \succ_{r^*} []} \\
\text{(K3)} \quad & \frac{v_1 \succ_r v_2}{v_1 : vs_1 \succ_{r^*} v_2 : vs_2} \\
\text{(K4)} \quad & \frac{v_1 = v_2 \quad vs_1 \succ_{r^*} vs_2}{v_1 : vs_1 \succ_{r^*} v_2 : vs_2}
\end{aligned} \tag{4.31}$$

The corresponding non-strict relation $\succeq_r$ is defined by

$$v_1 \succeq_r v_2 \iff v_1 = v_2 \quad \text{or} \quad v_1 \succ_r v_2. \tag{4.32}$$

Rules (C1) and (C2) propagate comparisons through concatenation. Rule (C1) compares the second component when the first components are equal, while (C2) gives priority to the first component whenever it differs. Thus, earlier subexpressions have priority over later ones.

Rules (A1)–(A4) define the crucial difference from Greedy matching. When two alternatives compete, the alternative producing the longer substring is preferred. If the two substrings have the same length, (A2) gives preference to the left alternative. Rules (A3)

and (A4) continue the comparison when both candidates have already chosen the same alternative.

Rules (K1)–(K4) apply the same idea to repetition. In particular, (K1) prefers the empty list when a non-empty sequence of iterations also flattens to ε , while (K2) prefers a non-empty match over the empty list. Rules (K3) and (K4) compare the iterations structurally, first comparing their heads and then their tails.

Applied to r_1 . Return to $r_1 = (a + ab)(b + \varepsilon)$ against ab , with

$$v = (Right(a, b), Right()) \quad (4.33)$$

and

$$v' = (Left\ a, Left\ b). \quad (4.34)$$

Both trees are pairs, so the comparison starts at (C2), whose conclusion $(v_1, v_2) \succ_{r_1 r_2} (v'_1, v'_2)$ matches with

$$r_1 = a + ab, \quad r_2 = b + \varepsilon, \quad v_1 = Right(a, b), \quad v'_1 = Left\ a. \quad (4.35)$$

(C2) needs only its first components to differ, which they do, so it reduces the whole comparison to $v_1 \succ_{r_1} v'_1$, that is to $Right(a, b)$ against $Left\ a$ within $a + ab$. Both sides are now tagged injections of an alternation, so an A rule applies, with

$$r_1 = a, \quad r_2 = ab, \quad v_1 = a, \quad v_2 = (a, b). \quad (4.36)$$

Under that binding (A1)'s premise holds, since the right alternative consumes strictly more of the input:

$$\begin{aligned} \text{len}(|v_2|) &= \text{len}(|(a, b)|) = 2 \\ \text{len}(|v_1|) &= \text{len}(|a|) = 1, \quad \text{so } 2 > 1. \end{aligned} \quad (4.37)$$

(A1) is the only rule that can fire here: (A2) would need $\text{len}(|v_1|) \geq \text{len}(|v_2|)$, which fails at $1 \geq 2$, and (A3) and (A4) both require the two sides to carry the *same* tag, which they do not. So (A1) gives

$$Right(a, b) \succ_{a+ab} Left\ a, \quad (4.38)$$

and discharging (C2)'s premise with it gives $v \succeq_{r_1} v'$. POSIX consequently selects

$$v = (Right(a, b), Right()), \quad (4.39)$$

which is the opposite of the Greedy result.

Applied to r_2 . For $r_2 = (a + b + ab)^*$ against **ab**, consider

$$v_2 = [Right\ Right(a, b)] \quad (4.40)$$

and

$$v'_2 = [Left\ a, Right\ Left\ b]. \quad (4.41)$$

Both trees are non-empty iteration lists, so (K3) applies. Matching its conclusion, which is $v_1:vs_1 \succ_{r^*} v_2:vs_2$, gives

$$r = a + (b + ab), \quad v_1 = Right\ Right(a, b), \quad v_2 = Left\ a, \quad (4.42)$$

and $vs_1 = []$, $vs_2 = [Right\ Left\ b]$. (K3) compares only the first iterations and ignores the tails entirely, which is what reduces the question to $v_1 \succ_r v_2$ inside the star body. Within that body both sides are tagged injections, binding

$$r_1 = a, \quad r_2 = b + ab, \quad v_1 = a, \quad v_2 = Right(a, b), \quad (4.43)$$

and (A1) fires again because the right alternative consumes more:

$$\text{len}(|Right(a, b)|) = 2 > 1 = \text{len}(|a|). \quad (4.44)$$

Thus

$$Right\ Right(a, b) \succ_{a+(b+ab)} Left\ a, \quad (4.45)$$

and discharging (K3)'s premise with that lifts the comparison to the star itself:

$$v_2 \succeq_{r_2} v'_2. \quad (4.46)$$

POSIX therefore selects the single two-character iteration, whereas Greedy selects the two one-character iterations.

Applied to r_3 . For $r_3 = (a + a)^*$ against **aa**, the star body binds $r_1 = a$ and $r_2 = a$, so both alternatives are the same literal and every candidate at every position matches exactly one character. (A1)'s premise $\text{len}(|v_2|) > \text{len}(|v_1|)$ therefore reads $1 > 1$ and never holds, so every comparison falls through to (A2), whose premise is the non-strict one:

$$\text{len}(|a|) \geq \text{len}(|a|) \implies Left\ a \succ_{a+a} Right\ a. \quad (4.47)$$

(K3) and (K4) lift this position by position, giving the same result as Greedy:

$$[Left\ a, Left\ a]. \quad (4.48)$$

Unlike r_1 and r_2 , Greedy and POSIX agree here. This is not a coincidence: (A2) only ever overrules (A1) when both alternatives tie in length, and a tie is exactly the condition under which Greedy's unconditional left-preference and POSIX's tie-broken left-preference compute the same thing, the same connection already observed for "if" in Section 4.1.

Applied to O . For $O = ((A + B) + C) + D$ against $w = 239$, only A and C match at all, so the comparison is between a tree choosing A and a tree choosing C . Both trees agree on the outer alternation: neither chooses D , so the comparison reduces, by (A4), to a comparison within $(A + B) + C$ of a tree choosing A (further inside $A + B$) against one choosing C directly. Since A and C both consume the full input,

$$\text{len}(|v_A|) = 3 = \text{len}(|v_C|), \quad (4.49)$$

so (A1)'s strict inequality fails and (A2) resolves the tie in favor of the left-hand candidate, the one going through $A + B$ and ultimately choosing A . POSIX therefore also selects A . As with r_3 , this is not a coincidence: A and C tie in length, so (A2)'s tie-break coincides with Greedy's unconditional left-preference, even though A and C , unlike r_3 's two copies of $\mathbf{a}$, are not otherwise interchangeable.

Applied to the infinite-chain examples. Rule (K1) resolves Section 4.3.2's pathology directly: for ε^* against ε , since

$$|() : []| = |()| |[]| = \varepsilon, \quad (4.50)$$

(K1) gives

$$[] \succ_{\varepsilon^*} [()]. \quad (4.51)$$

The same rule applies to every longer list of empty iterations. Thus $[]$ is preferred to every parse tree obtained by repeatedly matching ε .

For $(\varepsilon + a)^*$ against $\mathbf{a}$, the relevant comparison is different. Consider

$$v_0 = [\textit{Right } a] \quad (4.52)$$

and

$$v_1 = [\textit{Left } (), \textit{Right } a]. \quad (4.53)$$

Rule (K3) compares their first iterations, reducing the comparison to

$$\textit{Right } a \quad \text{versus} \quad \textit{Left } () \quad (4.54)$$

within $\varepsilon + a$. Rule (A1) applies because

$$\text{len}(|a|) = 1 > 0 = \text{len}(|()|). \quad (4.55)$$

Therefore

$$\textit{Right } a \succ_{\varepsilon+a} \textit{Left } (), \quad (4.56)$$

and hence

$$v_0 \succ_{(\varepsilon+a)^*} v_1. \quad (4.57)$$

The same comparison applies to every parse tree obtained by inserting additional empty iterations before the iteration matching **a**. Thus the POSIX ordering selects v_0 , while the Greedy ordering admits the infinite ascending chain.

Finally, for $(a^*)^*$ against **aaa**, compare

$$[[a, a, a]] \quad \text{against} \quad [[a, a, a], []]. \quad (4.58)$$

Since $[[] : [] = \varepsilon$, (K1) applies to the tail once the first (identical) iteration is matched by (K4), giving

$$[[a, a, a]] \succ_{(a^*)^*} [[a, a, a], []], \quad (4.59)$$

and the same argument disposes of every list padded with further empty iterations after the first. POSIX therefore selects the single outer iteration whose inner star consumes all three characters directly,

$$[[a, a, a]], \quad (4.60)$$

exactly the same tree the running implementation returns for $(a^*)^*$ against **aaa** under `deriv_std_rec` and `deriv_bc`. Unlike the abstract Greedy ordering, which has no maximal element for this expression, the frontier-based Greedy implementations of Chapter 6 do not compute an ordering maximum at all, and return this same tree without difficulty; the ordering-theoretic failure identified in Section 4.3.2 is a property of the abstract Greedy comparison, not of every construction that happens to compute a Greedy answer.

The contrast between the two policies is now explicit. Greedy resolves ambiguity according to the structural priority encoded by the regular expression, whereas POSIX compares the substrings assigned to subexpressions and gives priority to longer leftmost matches. Sulzmann and Lu prove that, unlike the Greedy ordering, the POSIX ordering is total and has a maximal element for every expression and input [Sulzmann and Lu, 2014]. The derivative parser developed in Chapter 5 computes that candidate without enumerating all possible parse trees.

Derivative-Based Parser

Chapter 4 fixed what is being computed, the $\geq_r$ -maximal parse tree of Section 4.3.3, without saying how. This chapter presents Sulzmann and Lu’s answer [Sulzmann and Lu, 2014]: a parser built directly on the Brzozowski derivative of Section 3.2, in two forms. The first, direct construction computes a derivative sequence and then explicitly rebuilds a parse tree from it in a second, backward pass. The second, bit-coded construction fuses parse-tree bookkeeping into the derivative step itself, needing only one forward pass. Both are POSIX, computing the identical $\geq_r$ -maximal tree, and differ only in how that tree is produced.

Both are developed here as theory first (Sections 5.2 to 5.5), then as the two Rust implementations that realize them, `deriv_std` (Section 5.6) and `deriv_bc` (Section 5.7), and are finally set against each other in Section 5.8. Throughout, the implementation’s own execution traces are used as the primary evidence for what each construction does, so Section 5.1 introduces the tracing facility first. Chapter 6 then repeats the exercise once more, replacing the Brzozowski derivative throughout with Antimirov’s partial derivative from Section 3.3, and, as that chapter shows, changing more than just which derivative operator is used.

5.1 Reading the Implementation’s Traces

Each of the four parsers in this thesis exists in two forms: a plain one and a *traced* one that records every step it takes. The traced form is not a debugging afterthought bolted on afterwards. It computes the same result by the same code path and additionally emits, at each step, the rule it applied and the intermediate value it produced, which makes it possible to check a construction against its defining equations by running it rather than by reading it.

The facility is exposed through `-diag`, which selects one of four verbosity levels:

Level	Adds	Destination
<code>-diag 0</code>	<code>true/false</code> only	<code>stdout</code>
<code>-diag 1</code>	regex, input, tree, error caret	<code>stdout</code>
<code>-diag 2</code>	timing, step count, construction steps	<code>stdout</code>
<code>-diag 3</code>	full derivation trace	<code>reports/report.txt</code>

Level 3 writes to a file rather than the terminal because its output grows with both the input length and the size of the intermediate expressions, and Section 5.5 shows how quickly the latter can grow. `-diag-report` overrides the destination; the environment variables `REGEX_DIAG` and `REGEX_DIAG_REPORT` set the same two values for library use.

How traces are used in this chapter. Rather than reproducing whole reports, the sections that follow quote *excerpts*, each placed next to the construction step it illustrates. A level-3 report is organized into labelled sections, and those sections map onto the algorithm's own phases closely enough that individual fragments stand on their own:

Report section	Construction phase
FORWARD PASS	the derivative sequence
NULLABILITY CHECK	the accept/reject decision
BACKWARD PASS	<i>mkEps</i> and <i>inj</i> , direct construction
INTERNALIZE, MKEEPSBC + DECODE	the bit-coded construction's ends
PARTIAL RECOVERY, ERROR SUMMARY	failure diagnostics

All eight reports quoted from in this chapter and the next are reproduced in full in Appendix ??, so that any excerpt can be read back in its original context. They all use the same running example, $r_1 = (a + ab)(b + \varepsilon)$ from Section 4.3, against the matching input "ab" and the non-matching input "b". In the frontend's surface syntax (Section 7.2) that expression is written "(a|ab)(b|)", with ε spelled as an empty right alternative, so the command producing every trace in this chapter is

```
cargo run -- parse "(a|ab)(b|)" "ab" --parser <PARSER> --diag 3
```

Excerpts are shown exactly as the tool writes them, except that the tool's Unicode operators are transliterated into ASCII (∂ as **d**, $\cdot$ as **.**, $\oplus$ as **(+)**, ε as **epsilon**, ϕ as **phi**, and $\checkmark$ as **[OK]**), and long lines are wrapped.

5.2 Core Algorithm

The direct construction follows the same two-phase shape informally sketched for Brzozowski's derivative in Section 3.2: consume the input by taking successive derivatives, then decide whether the result is nullable. What is new here is what happens *between* those two phases. Instead of a single Boolean, a parse tree is built. The construction is best understood in two halves, shown in Figure 5.1: a forward pass that is nothing more than the derivative sequence already introduced, and a backward pass that walks that sequence in reverse to assemble a tree one consumed character at a time.

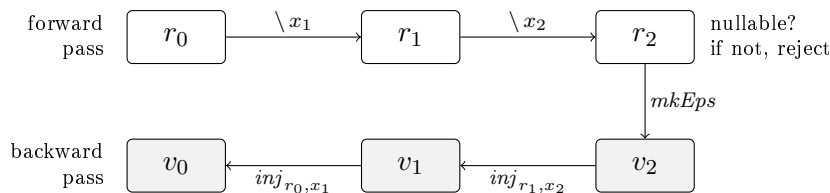


Figure 5.1: The direct construction on a two-character input. The forward pass takes one derivative per character; the backward pass turns the empty-word tree v_2 back into a tree of the original r_0 , undoing one derivative per step. The whole forward sequence must be retained until the backward pass consumes it.

Forward pass. For input $x_1x_2 \cdots x_n$, compute $r_0 \xrightarrow{x_1} r_1 \xrightarrow{x_2} \cdots \xrightarrow{x_n} r_n$ exactly as before, where $r_0 = r$ and $r_i = r_{i-1} \setminus x_i$. If r_n is not nullable, no parse tree exists and parsing fails. This half of the construction is identical to `match_deriv` (Section 3.2) with one difference:

no simplification is applied between steps. `simplify` is safe for a Boolean matcher, since it only ever normalizes r_i to a language-equivalent expression, but the backward pass below needs to walk back through the *exact* sequence of derivative transformations that produced r_n from r_0 , and a simplified r_i may no longer be the expression `inject` expects to see at that step.

The trace makes both halves of that claim visible at once. Each forward step names the derivative rule it applied and prints the resulting expression in full:

```
Step 1: d(r0, 'a')
  rule: d(r1.r2, 'a') [nullable(r1)=false]
        = d(r1, 'a').r2
r1 = Seq(Alt(Eps, Seq(Eps, Lit('b'))), Alt(Lit('b'), Eps))
  nullable(r1) = true [OK] position 1 matched
```

The rule line records which branch of the $(r_1 r_2) \setminus x$ case split was taken and why, and `r1` is printed unsimplified: the `Eps` inside `Seq(Eps, Lit('b'))` is exactly the kind of node `simplify` would have removed, and it is retained because `inject` will need it.

Backward pass. Once r_n is known to be nullable, the parse tree v_n for the empty remainder is built directly, by the function `mkEps` of Equation (5.1), defined by structural recursion on a nullable expression:

$$\begin{aligned}
mkEps_\varepsilon &= () & (E_\varepsilon) \\
mkEps_{r^*} &= [] & (E_*) \\
mkEps_{r_1 r_2} &= (mkEps_{r_1}, mkEps_{r_2}) & (E_{\text{seq}}) \\
mkEps_{r_1 + r_2} &= \begin{cases} \text{Left } mkEps_{r_1} & \varepsilon \in L(r_1) & (E_+^L) \\ \text{Right } mkEps_{r_2} & \text{otherwise} & (E_+^R) \end{cases} & (5.1)
\end{aligned}$$

The rule names continue the scheme of Section 3.2, and derivations below cite them with the binding of r_1 and r_2 stated, since which subexpression each stands for is what decides the case taken. `mkEps` has no case for a literal x or for $\emptyset$, since neither is ever nullable. The function is only ever called on an expression already known to satisfy $n(r)$. Its only interesting case is alternation, and exactly one of rules (A1)/(A2) of Section 4.3.3 is relevant there: both sides of an empty match have length 0, so (A2)'s tie-break, "prefer left when lengths are equal", degenerates at the empty word to "prefer left whenever left is even possible", which is exactly the $\varepsilon \in L(r_1)$ guard above.

That guard is the single most consequential line in the whole backward pass, and the trace prints it firing:

```
mkEps(r2):
  mkEps(r1 + r2): nullable(r1)=true -> Left(mkEps(r1))
v2 = Left (Right Right (), Right ())
```

Matching `r2` against `mkEpsr1+r2` binds r_1 to its long `Seq(...)` disjunct and r_2 to `Alt(Eps, Phi)`, and at this point *both* are nullable. E_+^L and E_+^R are therefore not both applicable: the case split tests only $\varepsilon \in L(r_1)$, so E_+^L fires and r_2 is never examined. `mkEps` does not compare the two; it takes the left one because it is nullable at all. Everything the backward pass produces downstream follows from that choice, and Section 5.3's Lemma 1 is the claim that making it this way is POSIX-correct.

With v_n in hand, the remaining task is to turn it back into a parse tree of the *original* r_0 , one derivative step in reverse at a time. This is the job of *inj*, an injection function that takes a parse tree of a derivative $r \setminus x$ and re-inserts the consumed character x to produce a parse tree of r itself, Equation (5.2):

$$\begin{aligned}
inj_{r^* \setminus x}(v, vs) &= (inj_{r \setminus x} v) : vs & (I_*) \\
inj_{(r_1 r_2) \setminus x}(v) &= \begin{cases} (inj_{r_1 \setminus x} v_1, v_2) & v = (v_1, v_2), \text{ not nullable}(r_1) & (I_{\text{seq}}) \\ (inj_{r_1 \setminus x} v_1, v_2) & v = \text{Left}(v_1, v_2), \text{ nullable}(r_1) & (I_{\text{seq}}^L) \\ (mkEps_{r_1}, inj_{r_2 \setminus x} v_2) & v = \text{Right } v_2, \text{ nullable}(r_1) & (I_{\text{seq}}^R) \end{cases} & (5.2) \\
inj_{(r_1 + r_2) \setminus x}(v) &= \begin{cases} \text{Left } (inj_{r_1 \setminus x} v_1) & v = \text{Left } v_1 & (I_+^L) \\ \text{Right } (inj_{r_2 \setminus x} v_2) & v = \text{Right } v_2 & (I_+^R) \end{cases} \\
inj_{x \setminus x}() &= x & (I_{\text{lit}})
\end{aligned}$$

The three-way $(r_1 r_2)$ case is worth reading against the derivative rule it inverts (Section 3.2): $(r_1 r_2) \setminus x$ produced either $(r_1 \setminus x) r_2$ alone (when r_1 is not nullable) or that same term joined with $r_2 \setminus x$ via $+$ (when it is), tagged *Left/Right* respectively by the *Alt* this new $+$ introduces. *inj* undoes exactly that tagging. A bare pair means r_1 was not nullable and the derivative went straight into it; a *Left*-tagged pair means it was nullable but the derivative still chose to continue matching r_1 ; a *Right*-tagged value means the derivative instead jumped straight into r_2 , so r_1 's own contribution to the reconstructed pair has to be synthesized fresh via $mkEps_{r_1}$, since no part of it was consumed by x at all.

Parsing is then this two-phase process packaged into one function, applying *mkEps* once at the end of the forward pass and *inj* once per character on the way back:

$$\begin{aligned}
parse(r, \varepsilon) &= mkEps_r \quad (\text{if } \varepsilon \in L(r)) \\
parse(r, xw) &= inj_{r \setminus x}(parse(r \setminus x, w))
\end{aligned} \tag{5.3}$$

The worked example, one step at a time. Return to $r_1 = (a + ab)(b + \varepsilon)$ against **ab**, whose two candidate parse trees were already established in Section 4.3. The forward pass computes

$$\begin{aligned}
r_1 &\xrightarrow{a} (\varepsilon + \varepsilon b)(b + \varepsilon) & D_{\text{seq}}, r = a + ab, s = b + \varepsilon, n(r) = \text{false} \\
&\xrightarrow{b} (\phi + (\phi b + \varepsilon))(b + \varepsilon) + (\varepsilon + \phi) & D_{\text{seq}}^n, r = \varepsilon + \varepsilon b, n(r) = \text{true}
\end{aligned} \tag{5.4}$$

The two steps take different cases of the same rule, and the reason is the binding: after one character the left operand has become $\varepsilon + \varepsilon b$, which is nullable by N_+ and N_ε , where the original $a + ab$ was not. That is what adds the second summand $(\varepsilon + \phi)$, and it is that summand which makes the whole expression nullable. The trace's second forward step confirms the rule taken and shows the expression that the hand derivation abbreviates:

```

Step 2: d(r1, 'b')
  rule: d(r1.r2, 'b') [nullable(r1)=true]
        = d(r1, 'b').r2 + d(r2, 'b')
r2 = Alt(Seq(Alt(Phi, Alt(Seq(Phi, Lit('b'))), Eps)), Alt(Lit('b'),
  Eps)), Alt(Eps, Phi))
nullable(r2) = true [OK] position 2 matched

```

This is the nullable- r_1 branch, the one that introduces a fresh *Alt*, and it is that new *Alt* whose two branches *mkEps* then chooses between. From v_2 the backward pass runs two injections, each printed with the expression it injects into and the value it produces:

```
inject(r1, 'b', v2):
  -> (Right (), b), Right ()
v1 = (Right (), b), Right ()

inject(r0, 'a', v1):
  -> (Right (a, b), Right ())
v0 = (Right (a, b), Right ())
```

Reading the two lines together shows *inj* doing its characteristic work. At the first step the tree still contains $()$, an empty-word placeholder where the literal *a* has not yet been reinserted; at the second, the *Lit* case replaces it with *a*, and the result is a complete tree of r_1 itself. The answer, $(\text{Right}(a, b), \text{Right}())$, selects the *longer ab* branch, matching the POSIX result claimed in Section 4.3.3 rather than Greedy’s $(\text{Left } a, \text{Left } b)$.

The report’s header quantifies the cost of the two-pass shape in one line:

```
Expressions stored:    3 (r0 -> r1 -> r2)
```

one expression per input character plus the original, all of them live until the backward pass has consumed them. That is exactly the requirement Figure 5.1 draws, and exactly the one Section 5.4 removes.

5.3 Correctness Properties

Sulzmann and Lu prove three results establishing that *parse* computes exactly the $\geq_r$ -maximal tree of Section 4.3.3, not merely a valid one [Sulzmann and Lu, 2014]. This section states them; full proofs are in the reference and are not reproduced here, but the intuition behind each is worth making explicit, since it is what justifies trusting the Rust implementation of Section 5.6 beyond its agreement with the reference on the test cases actually tried. Chapter 8 verifies that agreement empirically; the proofs are the reason agreement on finitely many test cases is expected to generalize to inputs never tested.

Lemma 1 (Empty POSIX Parse Tree). For r with $\varepsilon \in L(r)$, $mkEps_r$ is the POSIX parse tree of r for the empty word. This grounds the base case of the induction the other two results build on: at *mkEps*’s only interesting case, alternation, both sides can only ever match length 0, since there is no remaining input to consume, so (A2)’s length comparison always ties and reduces to left-preference, exactly the $\varepsilon \in L(r_1)$ test *mkEps* performs. The `nullable(r1)=true -> Left(mkEps(r1))` line quoted in Section 5.2 is this lemma applied once, at the one point in the example where the choice is not forced.

Lemma 2 (POSIX Preservation under Injection). If v is the POSIX parse tree of $r \setminus x$, then $inj_{r \setminus x}(v)$ is the POSIX parse tree of r . This is the result doing the real work, and its proof is correspondingly the most involved of the three. The informal argument: the derivative operation always tries r_1 before falling through to r_2 at a concatenation or alternation node (Section 3.2’s $(r_1 r_2) \setminus x$ case only reaches $r_2 \setminus x$ when r_1 is nullable, and even then keeps the r_1 -continuation branch as the left

disjunct of the resulting $+$). *inj* reconstructing that exact same left-first bias at every backward step, rather than an arbitrary one, is what carries POSIX-maximality from the smaller expression $r \setminus x$ up to the original r .

Theorem 1 (POSIX Parsing). *parse* computes POSIX parse trees. This follows from Lemmas 1 and 2 by induction on the input word: the base case is Lemma 1 applied to r_n , and each inductive step is one application of Lemma 2, carrying POSIX-maximality from $r \setminus x_1 \cdots x_n$ back up through n injections to r itself.

The induction in Theorem 1 has the same shape as the backward pass itself, which is why a level-3 trace reads as an instance of the proof: the **mkEps** line is the base case, and each **inject** line is one inductive step. Theorem 1 is nonetheless a statement about the construction, not about any particular implementation of it, so it transfers to the Rust code only to the extent that the code really is a transcription of the construction. Section 5.6 therefore reads each Rust function back against the equation it implements, and Section 5.8 checks the end-to-end result against the tree derived by hand in Section 4.3.

5.4 Bit-Coded Optimization (Theory)

The direct construction of Section 5.2 must retain the whole forward-pass sequence, every r_i from r_0 to r_n , or in an implementation an equivalent call stack, so that *inj* has each $r_{i-1} \setminus x_i$ available to reconstruct against on the way back: the backward pass cannot begin until the forward pass has fully finished. Sulzmann and Lu's bit-coded construction (their Section 3.2, "Incremental Bit-Coded Forward Parse Tree Construction") [Sulzmann and Lu, 2014] removes this requirement by incrementally building the parse tree *during* the forward pass itself, so that by the time the last character is consumed, the winning tree's information is already sitting in the final expression, with no second pass required. Figure 5.2 gives the resulting shape, which is Figure 5.1 with the return journey removed.

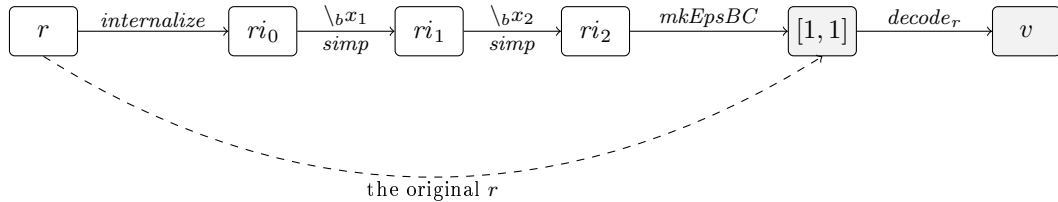


Figure 5.2: The bit-coded construction on the same two-character input. Every arrow points forward: no intermediate ri_i is needed once its successor exists. The dashed arrow marks the one backward dependency that remains, namely that *decode* must be given the original, un-annotated r to know what shape the bits describe.

Bit-code representation. Rather than a `ParseTree` value, a parse tree is represented as a flat sequence of bits: 0 marks a *Left* choice, 1 a *Right* choice, and inside a Kleene star, 0 marks "one more iteration" and 1 marks "stop". A pair of mutually recursive functions *encode_r*/*decode_r* convert between a $\vdash v : r$ parse tree and its bit-string representation, structurally straightforward given the tagging already present in v itself.

The star convention is worth spelling out on a concrete case, since it is the one place where a bit does not correspond to an alternation. Under $r = a^*$, the word **aaa** encodes

as $[0, 0, 0, 1]$: three 0s, one per iteration taken, then a final 1 ending the star. The empty word under the same r encodes as $[1]$ alone, a star that stops immediately. So the bit-string length grows with the number of iterations rather than with the size of r , and reading it back requires knowing r , which is what $decode_r$ is given.

Annotated regular expressions. The construction's core idea is to annotate the derivative operation's own *input*, not just its output, with partial bit-code information, so that by the time a sub-expression becomes ε its accumulated annotation already *is* the relevant slice of the final bit-string. An annotated regular expression ri extends plain r with a bit-list bs attached at every node except $\emptyset$, which denotes no language and so has no parse tree to annotate:

$$ri ::= \emptyset \mid (bs@ \varepsilon) \mid (bs@x) \mid (bs@ri_1 \oplus ri_2) \mid (bs@ri_1 ri_2) \mid (bs@ri^*) \quad (5.5)$$

Two operations connect plain r to this annotated world. *internalize* converts r into ri with every bit-list initially empty, except at *Alt*: the left operand of $r_1 + r_2$ is tagged with a leading 0 and the right with a leading 1 at construction time, so that which alternative a sub-tree came from can always be recovered from its own annotation without re-inspecting the surrounding structure. *fuse*(bs, ri) prepends a bit-list onto whatever annotation ri 's own top-most node already carries, and is the one primitive every other operation in this construction is built from.

The trace's INTERNALIZE section shows the result for r_1 , with the alternation tags already in place before any input is read:

```
ri0 = []@([]@([]@([false]@'a' (+) [true]@([]@'a' . []@'b')) .
  []@([false]@'b' (+) [true]@epsilon))
```

Reading the four tags off: `[false]` on a and `[true]` on ab record the two alternatives of $a + ab$, and the same pair appears again on b and ε . Every other bit-list is empty. Those four tags are the only bits the construction will ever emit for this expression; everything the derivative does from here is a matter of moving them, never of inventing new ones.

The refined derivative. The ordinary derivative operation is lifted to annotated expressions, written $ri \setminus_b l$, following the same case structure as Equation (3.3)'s $r \setminus l$ but threading bits through instead of discarding the annotation:

$$\begin{aligned} (bs@ri_1 \oplus ri_2) \setminus_b l &= (bs@(ri_1 \setminus_b l) \oplus (ri_2 \setminus_b l)) & (B_+) \\ (bs@ri_1 ri_2) \setminus_b l &= \begin{cases} (bs@(ri_1 \setminus_b l) ri_2) \oplus fuse(mkEpsBC_{ri_1}, ri_2 \setminus_b l) & \varepsilon \in L(ri_1) & (B_{seq}^n) \\ (bs@(ri_1 \setminus_b l) ri_2) & \text{otherwise} & (B_{seq}) \end{cases} \\ (bs@ri^*) \setminus_b l &= (bs@fuse([0], ri \setminus_b l) ([@ri^*]) & (B_*) \end{aligned} \quad (5.6)$$

The concatenation case mirrors $(r_1 r_2) \setminus x$'s two-way split exactly, with one addition: on the branch where r_1 's derivative is abandoned in favor of jumping into r_2 , the bits r_1 would have contributed to an empty match ($mkEpsBC_{ri_1}$, below) are fused directly onto r_2 's derivative rather than discarded. This is the incremental step that replaces the direct construction's *later* call to $mkEps_{r_1}$ inside *inj* (Section 5.2) with work done immediately, during the forward pass. The star case attaches a leading 0 bit ("one more iteration") each time the body is derived once more; the outer bs stays where it is, on the new alternation or concatenation node the derivative rule produces, exactly as it does for plain *Alt*.

mkEpsBC. Mirroring Equation (5.1)’s *mkEps* but returning accumulated bits instead of a tree:

$$\begin{aligned}
mkEpsBC_{(bs@_\varepsilon)} &= bs & (EB_\varepsilon) \\
mkEpsBC_{(bs@ri_1 \oplus ri_2)} &= bs \mathrel{++} \begin{cases} mkEpsBC_{ri_1} & \varepsilon \in L(ri_1) \\ mkEpsBC_{ri_2} & \text{otherwise} \end{cases} & \begin{matrix} (EB_+^L) \\ (EB_+^R) \end{matrix} & (5.7) \\
mkEpsBC_{(bs@ri_1 ri_2)} &= bs \mathrel{++} mkEpsBC_{ri_1} \mathrel{++} mkEpsBC_{ri_2} & (EB_{seq}) \\
mkEpsBC_{(bs@ri^*)} &= bs \mathrel{++} [1] & (EB_*)
\end{aligned}$$

Once the forward pass ends on a nullable annotated expression, *mkEpsBC* extracts its complete bit-string directly, and *decode_r* turns that bit-string into an ordinary parse tree. *decode* uses the *original*, un-annotated *r*, because it needs to know the shape it is decoding bits *for*, which the annotated *ri* alone no longer determines once its own annotations have been consumed. That is the dashed dependency in Figure 5.2, and the only thing the construction still has to remember from before the forward pass began.

The worked example, one step at a time. Run Section 5.2’s *r₁* against *ab* through this construction instead, to see the same POSIX answer assembled incrementally rather than reconstructed afterward. Starting from the *ri₀* quoted above, deriving with respect to *a* takes the Seq case’s second branch, since the left operand *[]@ (0@a ⊕ 1@ab)* is not nullable:

```

Step 1 (char: 'a', position 1)
  rule: (bs@ri1.ri2) \ 'a' [nullable_bc(ri1)=false]
        = bs@(ri1\'a'.ri2)
  simp -> []@([]@([false]@epsilon (+) [true]@'b')) . []@([false]@'b'
        (+) [true]@epsilon))

```

Comparing this against *ri₀*, the literal *'a'* has become *epsilon* and *[]@'a'* . *[]@'b'* has become *'b'*, while both alternation tags stay exactly where they were. Deriving one character has consumed structure and moved no bits.

ri₁’s left operand is now nullable, via *[false]@epsilon*, so the second character takes the other branch of the Seq case, and this is where bits finally move:

```

Step 2 (char: 'b', position 2)
  rule: (bs@ri1.ri2) \ 'b' [nullable_bc(ri1)=true]
        = bs@((ri1\'b'.ri2) (+) (fuse (mkEpsBC ri1) (ri2\'b')))
  simp -> []@([true]@([false]@'b' (+) [true]@epsilon) (+) [false,
        false]@epsilon)

```

Two things happened in that one step. The rule’s *fuse (mkEpsBC ri1)* produced the *[false, false]* prefix on the right disjunct: the first *false* is *ri₁*’s own empty-match bit, the second is *(b+ε)*’s left-alternative tag, fused in front of it. And a *[true]* appeared on the left disjunct, tagging "the derivative continued into the left operand" as the *Right* alternative of the freshly introduced node. Where *inj* discovers that tagging only on the way back, *_b* must commit to it immediately, as part of the forward pass.

Both disjuncts of *ri₂* are nullable, so *mkEpsBC* faces the same choice *mkEps* faced in Section 5.2, and resolves it the same way, by preferring the left disjunct whenever it is

nullable at all:

$$\begin{aligned}
mkEpsBC_{ri_2} &= [] ++ mkEpsBC_{1@([] @ (0@b \oplus 1@ \epsilon))} & EB_+^L, bs = [] \\
&= [1] ++ ([] ++ mkEpsBC_{1@ \epsilon}) & EB_+^R, bs = [1] \\
&= [1] ++ [1] = [1, 1] & EB_\epsilon, bs = [1]
\end{aligned} \tag{5.8}$$

The outer step takes EB_+^L and the inner one EB_+^R , for the same reason in both cases: the rule tests only whether its own ri_1 is nullable. At the outer node ri_1 is the $1@(\dots)$ disjunct, which is; at the inner node ri_1 is $0@b$, a bare literal, which is not, so the right branch is taken and contributes the second 1. which the trace confirms, together with the decode that follows:

`mkEpsBC(ri2) = [11]`

`decode((a|ab)(b|), [11]):`
`= (Right (a, b), Right ())`

Both constructions therefore encode "prefer continuing the match already in progress over falling back to the shorter alternative", just applied at different points: in *inj*'s backward pass there, and directly inside *mkEpsBC*'s forward-accumulated bits here. The two bits select *Right* at both alternation points of r_1 , which is the tree Section 5.2's worked example reached via its explicit backward pass, read this time straight off the forward pass alone.

On simplification between steps. Section 5.2 noted that the plain construction's forward pass must *not* simplify between steps, since the backward pass needs to walk back through the exact sequence of derivatives that produced it. The bit-coded construction has no such restriction, and both entry points in `src/parsers/deriv_bc/parse.rs` do call `simp` after every step, which is why every trace excerpt above shows a `simp ->` line rather than a raw derivative. This is safe here for the same reason `fuse` is: the information a backward pass would otherwise need to walk back for is already carried *on* the annotated expression itself, as bits attached to its nodes, so simplifying ri_i down to a smaller but bit-equivalent expression loses nothing *mkEpsBC* needs later. In the plain construction, by contrast, simplifying r_i would destroy the very structure *inject* needs to reconstruct against.

5.5 Simplification Rules

Section 3.2's worked example already showed plain derivatives growing an extra $+$ at every nullable-prefix step with nothing to bound that growth, and the bit-coded construction inherits the same problem. [Sulzmann and Lu \[2014\]](#) give the concrete case

$$\begin{aligned}
a^* &\xrightarrow{a} \epsilon a^* \xrightarrow{a} \phi a^* + \epsilon a^* \\
&\xrightarrow{a} (\phi a^* + \epsilon a^*) + (\phi a^* + \epsilon a^*) \xrightarrow{a} \dots
\end{aligned} \tag{5.9}$$

where every step after the first is language-equivalent to ϵa^* but syntactically doubles in size. Left unchecked, this defeats the whole point of the bit-coded construction's incremental, single-pass promise: an unboundedly growing ri makes each subsequent $\setminus_b$ step correspondingly more expensive.

A simplification function *simp*, applied after every derivative step, keeps the annotated expression's size bounded. Its rules are Phi-propagation through *Seq* and *Alt*, epsilon-elimination $((bs@(bs'@ \epsilon) ri) \rightarrow fuse(bs ++ bs', ri))$, folding the eliminated node's own bits

forward rather than discarding them), and flattening and deduplicating nested alternations, keeping the first occurrence of a syntactic duplicate. All of these are instances of the familiar algebraic identities from Section 3.2 ($\emptyset \cdot r = \emptyset$, $\varepsilon \cdot r = r$, $\emptyset + r = r$, $r + r = r$), applied here to the annotated grammar and combined with an explicit *isPhi* test that recognizes "empty language" expressions built up from these rules rather than only the literal $\emptyset$ constructor.

***simp* on the paper's own example.** Applied to a^* against **aaa**, *simp* collapses every step back down to a bit-annotated form of a^* itself. The trace shows one *simp* result per character, each a single star node whose accumulated bit-prefix has grown by one 0:

```
Step 1 ... simp -> [false]@([@'a')*
Step 2 ... simp -> [false, false]@([@'a')*
Step 3 ... simp -> [false, false, false]@([@'a')*
```

```
mkEpsBC(ri3) = [0001]
decode(a*, [0001]):
  = [a, a, a]
```

The expression stays at two nodes throughout while the bit-prefix grows linearly, and the three 0s plus *mkEpsBC*'s closing 1 are exactly the $[0, 0, 0, 1]$ encoding of "three iterations, then stop" from Section 5.4. This is the clearest single illustration of what the bit-coded construction is for: all of the parse-tree information is in the prefix, and none of it is in the expression.

Where *simp* does not bound the growth. This result does not generalize to every expression, and it is worth being precise about that rather than letting the a^* case stand for the general one. Figure 5.3 measures node counts per derivative step for two patterns, each with and without *simp*.

The two simplified series behave completely differently. For a^* the line is flat. For $(a + (b + ab))^*$ it is straight on a logarithmic axis, which is to say exponential. Every two characters take the size s to $2s + 1$ exactly, so $s + 1$ doubles: from 35 at step 4 the series runs 71, 143, 287, 575, 1151, 2303 at step 16, and 36 863 at step 24. The growth is therefore $\Theta(2^{n/2})$ in the input length, not the bounded size the a^* case suggests. The figures are unchanged if the three-way alternation is grouped the other way, as $((a + b) + ab)^*$, which is what the CLI spelling "**(a|b|ab)***" parses to (Section 2.1); the growth depends on the pattern's shape rather than on that grouping. Applying *simp* repeatedly to a genuine fixpoint changes nothing either: a single pass already *is* the fixpoint here, producing byte-identical expressions at every step, so this is not a case of the implementation stopping too early.

The mechanism is specific and easy to state. *simp*'s deduplication rule discards an alternation branch only when it is *syntactically identical* to an earlier one, and an annotated branch carries its bit-list as part of its syntax. Counting the top-level alternation branches of *simp*(ri_i) at each step, and counting them again after erasing every annotation, separates the two:

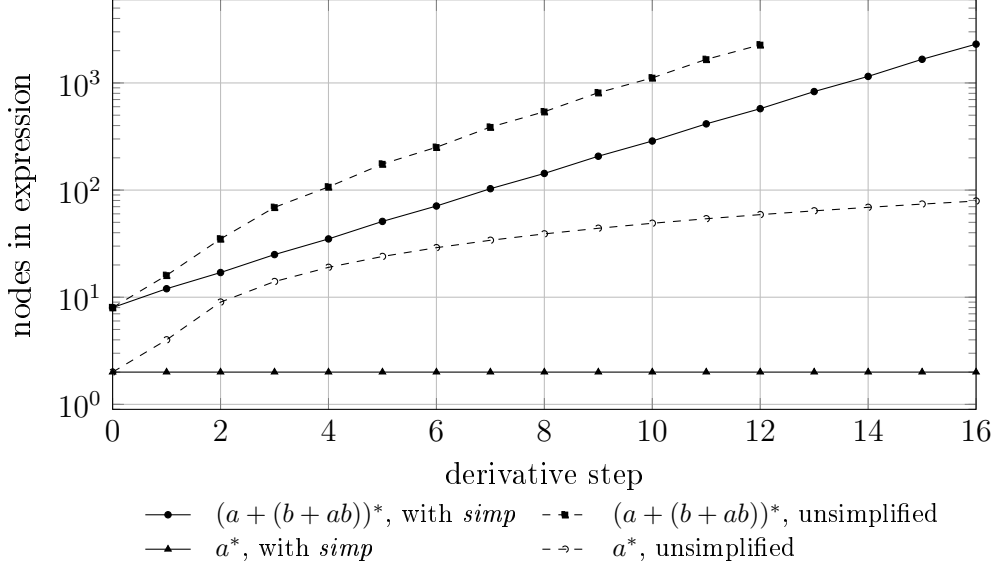


Figure 5.3: Expression size per derivative step, logarithmic vertical scale. For a^* , *simp* holds the expression at two nodes while the unsimplified derivative grows linearly. For $(a + (b + ab))^*$ against $(ab)^n$, *simp* reduces the expression substantially at every step and still leaves it growing exponentially: a straight line on this scale is exponential growth. Regenerate with `cargo run --release --example thesis_figures -- growth; tests/test_thesis_figures.rs` pins these values.

Step	2	4	6	8	10	12
Branches	2	4	8	16	32	64
Distinct branches	2	4	8	16	32	64
Distinct ignoring bits	1	1	1	1	1	1

At every step, all of the branches are structurally the same expression, differing only in the bits recording which route through the alternation reached it, so the last row stays at 1 while the first two double. The $r + r = r$ rule never fires, because on annotated expressions the two sides are not equal.

The bits are not redundant: each branch’s annotation is a different candidate parse tree, and *mkEpsBC*’s left-preference is what eventually selects among them, so dropping duplicates on shape alone would change which tree the parser returns rather than merely shrinking the expression. That is why the rule is stated on full syntactic equality in the first place, and why the growth here is a property of the construction rather than a defect in this transcription of it.

Sulzmann and Lu claim, but do not fully prove, that the resulting construction runs in time linear in the input, on the grounds that each of $\backslash_b$, *simp*, *fuse*, *mkEpsBC*, and *isPhi* only ever recurses over sub-calls bounded by the now-simplified expression’s size, and they note explicitly that detailed space-complexity estimates are left as future work [Sulzmann and Lu, 2014]. Figure 5.3 is one concrete reason that gap matters: the argument’s premise, that *simp* keeps the expression small, holds for a^* and fails for $(a + (b + ab))^*$, and the linear-time claim rests on it. Chapter 8 measures the resulting wall-clock behaviour on both kinds of pattern, and Section 8.5.4 returns to this family specifically.

5.6 Rust Implementation: The `deriv_std` Parser

`deriv_std` (`src/parsers/deriv_std/`) is the Rust realization of Section 5.2's direct construction. It exposes two entry points computing the identical result by two different means, `parse_deriv_std_rec` and `parse_deriv_std_loop`, discussed together in Section 5.6.4 below, after the shared machinery both call into.

5.6.1 `ParseTree` and `flatten`

The `ParseTree` type and its `flatten` function were already introduced in Section 4.2 (`src/types/tree.rs`), with `Empty`, `Char`, `Pair`, `Left`, `Right`, `Star` corresponding one to one with `()`, a literal, a pair, the two tagged injections, and a Kleene-star list. Nothing about the type itself is specific to `deriv_std`: it is the one representation every parser in this and the following chapter ultimately produces, precisely so that the agreement tests of Chapter 8 can compare their outputs directly rather than through some intermediate translation.

Keeping `flatten` separate from tree construction is what makes the round-trip check in those tests possible at all. `flatten(v)` discards every *Left/Right* tag and every pair boundary, returning only the concatenated literals, so for any successful parse of *w* it must return *w* exactly. That is a property independent of disambiguation policy: POSIX and Greedy select different trees for *r*₁ against `ab`, but both trees flatten back to `"ab"`. Every diagnostics report prints the flattened result next to the tree for precisely this reason, as a visible check that the tree returned really is a tree for the input given:

```
Parse tree:      (Right (a, b), Right ())
Flattened:      "ab"
```

5.6.2 `mk_eps`

`mk_eps` (`src/regex/mk_eps.rs`) transcribes *mkEps* case for case: `Eps` to `Empty`, `Star` to an empty `Vec`, `Seq` to a `Pair` of the two recursive results, and `Alt` to `Left/Right` depending on `nullable(r1)` exactly as the theory prescribes. Called on `Lit` or `Phi`, which *mkEps* has no case for since neither is ever nullable, it panics rather than silently returning a meaningless value. Every call site only ever reaches `mk_eps` once nullability of its argument has already been established, so the panic is an assertion of an invariant the surrounding code maintains, not a reachable runtime error.

This function lives in `regex/`, flat and outside `deriv_std/` specifically, because Chapter 6's `pderiv_std` needs the identical function for the identical reason: both build a `ParseTree` directly rather than bit-codes, and both need the same "parse tree for a nullable expression matched against nothing" primitive to do it.

5.6.3 `inject`

`inject` (`src/parsers/deriv_std/inject.rs`) is the direct transcription of *inj*. Its `Seq` case is the one worth reading closely against the theory: when *r*₁ is not nullable, only a `Pair` is ever handed in, and injection recurses into its first component; when *r*₁ is nullable, the incoming value is either `Left(Pair(v1, v2))`, the "continue *r*₁" branch the derivative rule produces, or `Right(v2)`, the "*r*₁ already matched empty" branch, and

only the second case calls `mk_eps(r1)` to synthesize the otherwise-absent left component. This is exactly the three-way split of Section 5.2's $inj_{(r_1 r_2) \setminus x}$.

The trace prints each call with the expression being injected into, which is what makes it possible to check that the case taken really is the one the theory predicts:

```
inject(r1, 'b', v2):
  inject(Seq(Alt(Eps, Seq(Eps, Lit('b')))), Alt(Lit('b'), Eps)),
    'b', Left (Right Right (), Right ()))
  -> (Right ((), b), Right ())
```

The expression is a `Seq`, so r_1 binds to `Alt(Eps, Seq(Eps, Lit('b')))` and r_2 to `Alt(Lit('b'), Eps)`. That r_1 is nullable, and the incoming value is `Left(...)`, so the case that fires is l_{seq}^L : continue into r_1 , keep v_2 untouched. Had the value been `Right(...)` instead, the same call would have taken l_{seq}^R and synthesized a left component through `mk_eps`. Had r_1 not been nullable, neither tag would have been present at all and l_{seq} would have applied to a bare `Pair`.

Every case that receives a value shaped differently from what the corresponding derivative rule could have produced panics rather than guessing. A bare `Pair` where a nullable `r1` demands a tagged `Left/Right` is the clearest example: such a mismatch can only mean the caller passed a tree that did not actually come from `r`'s own derivative sequence, which is the one precondition the whole backward pass depends on.

5.6.4 parse_deriv_std_rec versus parse_deriv_std_loop

Both functions (`src/parsers/deriv_std/parse.rs`) compute the identical result, shown empirically and not merely by construction (Chapter 8), by two different means of implementing the same two-phase algorithm.

`parse_deriv_std_rec` recurses to the end of the input first, then calls `inject` on the way back up the call stack as the recursion unwinds. `deriv`'s "leading character removed" and `inject`'s "leading character re-inserted" mirror each other directly across the recursion, needing no explicit intermediate storage for the forward-pass sequence at all, since the call stack *is* that sequence. `parse_deriv_std_loop` makes the same sequence explicit as a `Vec<Regex>`, built by an ordinary forward `for` loop, then walks that vector backward with a second loop calling `inject` at each step.

Both are Figure 5.1 realized; they differ only in where the top row is kept while the bottom row consumes it. That makes different resource trade-offs unrelated to the algorithm itself. `parse_deriv_std_rec`'s call stack depth grows with the input length and can overflow it on sufficiently long input, where `parse_deriv_std_loop`'s explicit `Vec` instead grows on the heap. Section 8.5.1 measures exactly where that difference starts to matter.

5.7 Rust Implementation: The Bit-Coded Version

`deriv_bc` (`src/parsers/deriv_bc/`) realizes Section 5.4's construction, one module per box in Figure 5.2.

5.7.1 ARegex, the Bit-Annotated Regex

`ARegex` (`src/types/aregex.rs`) mirrors the annotated grammar *ri* directly: every variant except `Phi` carries its own `Vec<bool>` bit-prefix alongside the same structural payload `Regex` already carries, a `char` for `Lit` and boxed sub-expressions for the recursive cases. This is the one point in the whole implementation where a plain `Regex` is not enough: `deriv_bc`'s entire purpose is threading partial parse-tree information *through* the tree as it is derived, and that requires a place on every node to put it, not only at the leaves.

One derived trait is deliberately absent. `Regex` derives `Hash` (Section 2.3) so that it can be used as a `HashSet` key, which Chapter 6's deduplication depends on; `ARegex` derives only `Debug`, `Clone`, `PartialEq` and `Eq`. Nothing in `deriv_bc` ever needs to hash an annotated expression, because its own deduplication, inside `simp`, compares alternation branches pairwise rather than through a hash set. That is also why the quadratic `dedup_first` in `simplify.rs` is acceptable: the branch lists it runs over are the ones counted in Section 5.5, and on the patterns where those lists grow large, the growth itself is the dominant cost rather than the comparison strategy.

5.7.2 internalize and fuse

Both live in `src/parsers/deriv_bc/internalize.rs`. `internalize` converts a plain `Regex` into a freshly bit-annotated `ARegex` with every list empty, except `Alt`: the left branch is fused with a leading `false` and the right with a leading `true` at construction time, exactly as the theory's *internalize* does, and exactly as the `INTERNALIZE` excerpt in Section 5.4 shows. `fuse(bs, ri)` prepends `bs` onto `ri`'s own top-level annotation and is the one primitive the rest of this section's functions are all built from, including the degenerate case `fuse(bs, Phi) = Phi`, since there is no annotation slot on `Phi` to prepend anything to.

`fuse` also short-circuits on an empty prefix, returning `ri` untouched rather than rebuilding it. This matters more than it looks: `fuse` is called on every `Seq` and `Star` step of every derivative, and the overwhelming majority of those calls carry an empty `bs`, so the check turns what would be a node rebuild into a move.

5.7.3 deriv_bc

`src/parsers/deriv_bc/deriv.rs` transcribes $\backslash_b$ case for case. Its `Alt` case keeps the outer bit-list on the resulting `Alt` node itself, rather than pushing it into either recursive call. This looks like an arbitrary implementation choice here, and for `deriv_bc` it largely is, since there is always a single combined node available to carry it. Chapter 6 shows what happens to the same decision in a construction that has no such node, where the bits must instead be carried separately per residual.

Its `Seq` case implements the nullable- r_1 branch's two strands exactly as written above, continuing r_1 's own derivative or jumping into r_2 's with r_1 's empty-match bits fused on, which is the step the `[false, false]` prefix in Section 5.4's step-2 excerpt records. Its `Star` case attaches the leading "one more iteration" bit via `fuse(&[false], ...)` before rewrapping the untouched star body, which is the step the growing `[false, ...]` prefix in Section 5.5's a^* excerpt records.

Both entry points in that module's `parse.rs` call `deriv_bc` in a loop, once per input

character, applying `simp` after every step. They are named `parse_bitcoded_recursive` and `parse_bitcoded_loop`. `simp`, in the same directory's `simplify.rs`, is the Rust realization of Section 5.5's simplification, and its cases match *simp*'s directly: Phi propagation via the `is_phi` test, epsilon-elimination via `fuse`, and list-flattening with first-occurrence deduplication for nested alternations.

Unlike `deriv_std`'s two entry points, which differ in how the forward sequence is stored, these two differ only in recursion versus iteration over the input: neither retains the sequence, because the bit-coded construction never revisits it.

5.7.4 `mk_eps_bc`

`src/parsers/deriv_bc/mk_eps_bc.rs` extracts the accumulated bit-string from a nullable `ARegex`, structurally identical to `mk_eps` but returning bits instead of a tree. `Alt` prefers whichever branch is nullable, checking left first to match `mk_eps`'s own left-preference; `Seq` concatenates both children's bits after its own prefix; and `Star` appends a final `true`, marking "no further iterations", onto whatever bits the completed iterations already accumulated. Like `mk_eps`, it panics on `Phi` and `Lit`, for the same reason and under the same established precondition.

5.7.5 `decode`

`src/regex/decode.rs` turns the plain, *original* `Regex` plus a flat `Vec<bool>` bit-string back into an ordinary `ParseTree`. `Alt` consumes one leading bit to choose `Left/Right`, `Seq` decodes both children in sequence, and `Star` loops, consuming a `false` per iteration until a `true` ends it: the direct inverse of `encode`, which this implementation never needs to run itself, since nothing here starts from an already-built `ParseTree` needing to be re-encoded.

`decode` takes only a plain `Regex`, not an `ARegex`, because by the time it is called the bit-string alone carries everything needed, decoded against the shape of the un-annotated original expression. That is the dashed dependency in Figure 5.2. It is also why `decode` lives in flat `regex/` rather than inside `deriv_bc/` specifically: Chapter 6's `pderiv_bc` produces the same `(Regex, bits)` pair by an entirely different route and needs the identical final step to turn it into a tree.

The decoder is also where a bit-string that does not correspond to the expression it is decoded against is caught. `decode` asserts that no bits remain once the whole expression has been walked, and its `Alt` and `Star` cases panic on an unexpected end of bits. Both conditions are unreachable when the bits come from `mk_eps_bc` applied to a derivative of that same expression, which is the only way either parser calls it, so they serve the same role as the panics in `inject`: they turn a silent wrong answer into an immediate failure if the invariant is ever broken.

5.8 Comparing the Two Constructions

Sections 5.2 and 5.4 have now run the same expression against the same input through both constructions, quoting each trace at the point where it illustrated a rule. Setting

the two reports against each other as wholes makes the structural difference visible in one view. The full reports are Appendix ?? and Appendix ??.

	<code>deriv_std</code>	<code>deriv_bc</code>
Report sections	forward, nullability, backward	internalize, forward, nullability, decode
Named intermediates	<code>r0 -> r1 -> r2</code>	<code>ri0 -> ri1 -> ri2</code>
Retained at the end	all three	the last one only
Tree-building calls	1 <code>mkEps</code> + 2 <code>inject</code>	1 <code>mkEpsBC</code> + 1 <code>decode</code>
Simplification	none, by necessity	<code>simp</code> after every step
Final answer	<code>(Right (a, b), Right ())</code>	

The number of tree-building calls is the row that does not scale with the input. `deriv_std` needs one `inject` per character and so grows with the input length; `deriv_bc` needs exactly one `mkEpsBC` and one `decode` no matter how long the input is. That is the whole content of the optimization, and it is visible in the reports as the difference between a `BACKWARD PASS` section whose length depends on the input and a fixed-size `MKEEPSBC + DECODE` section.

The final bits `[11]` and the final tree `(Right (a, b), Right ())` encode the same answer in the two representations, and the correspondence is exact: each `Right` in the tree is a 1 in the bit-string, read in the order `decode` walks the expression.

What happens on a failure. Running the same r_1 against `b`, a string not in $L(r_1)$ since every derivation of r_1 must begin by matching `a`, both parsers stop after one derivative step and neither reaches a tree-building phase at all. What each has in hand at that point differs, and the difference is instructive:

```
deriv_std:  r1 = Seq(Alt(Phi, Seq(Phi, Lit('b'))), Alt(Lit('b'), Eps))
            nullable(r1) = false  [FAIL] position 1
```

```
deriv_bc:   simp -> phi
            ri1 = phi
            nullable_bc(ri1) = false  [FAIL]
```

`deriv_std`'s expression denotes the empty language but is not syntactically `Phi`; it is a `Seq` whose left operand has both alternatives dead. `deriv_bc`'s `simp` collapses exactly that expression all the way down to `phi`. This is the single clearest illustration of what Section 5.5's *isPhi* test adds over plain `simplify`: recognizing an empty-language expression that is not the literal $\emptyset$ constructor.

Both constructions then report the identical failure, character for character:

```
Failure at position:  1 (1-indexed)
Found:                'b'
Expected:              'a'
  b
  ^
```

The diagnostics layer's expected set (Section 9.3.3) is read directly off the first-literal set

of the expression at the point of failure, so it does not depend on which construction produced that expression or on how much simplification was applied to it. This is the general pattern for failures across all four parsers in this thesis: the constructions differ in what they build on a successful match, and agree on what they report when there is nothing to build. The two full failure reports are Appendix ?? and Appendix ??.

Agreement between the three POSIX entry points. The two parsers of this chapter, together with `deriv_std`'s loop variant, are three separate routes to the tree Theorem 1 specifies, so their outputs are directly comparable. Running all three over 300 randomly generated expressions against 39 inputs each, 11 700 cases in total, they agree on every one, both on whether the input matched and on the exact `ParseTree` returned when it did. Section 8.4.2 describes the generator and the full test setup; what matters here is that the bit-coded route and the inject route, which share no code beyond `Regex` itself and `nullable`, are not observed to diverge on any of them.

Partial-Derivative-Based Parser

Chapter 5 built two parsers around Brzozowski’s derivative, both computing the identical POSIX $\geq_r$ -maximal tree of Section 4.3.3. This chapter repeats the exercise with Antimirov’s partial derivative from Section 3.3 in place of $r \backslash x$ throughout. As Section 6.1 shows first, repeating the construction with a different derivative operator does not repeat its disambiguation guarantee.

Neither flops14-extended [Sulzmann and Lu, 2014] nor Sulzmann’s teaching-page Haskell reference [Sulzmann, 2026] defines a POSIX partial-derivative parser at all. The paper’s bit-coded construction (Chapter 5) is Brzozowski-derivative-only, and the teaching page’s own bit-coded partial-derivative parser, `pDerivBC`, is built around a deduplication rule this chapter shows to be fundamentally Greedy rather than POSIX. This chapter therefore builds two *Greedy* parsers: `pderiv_bc` (Section 6.3), following `pDerivBC` directly, and `pderiv_std` (Section 6.4), this project’s own construction, built to give `pderiv_bc` the same kind of matched non-bit-coded counterpart Chapter 5’s `deriv_bc` has in `deriv_std`. Both are verified to agree with each other exactly, and with the two POSIX parsers on membership alone, in Chapter 8 and again in Section 6.6.

Trace excerpts are used here exactly as in Chapter 5, under the conventions Section 5.1 sets out, and against the same running example. The two parsers’ level-3 reports have their own section structure, since a frontier is not a single expression:

Report section	Construction phase
INITIAL FRONTIER	the one-strand starting state
FORWARD PASS	one block per character: before, rule, after
SELECTION	the first-nullable-strand decision
MKEEPSBC + DECODE	bit-coded only; <code>pderiv_std</code> injects in place

The four full reports are Appendices ??, ??, ?? and ??.

6.1 Extending Derivatives to Partial Derivatives

Section 3.3 already defined $pd(r, x)$ as a *set* of residual expressions rather than one combined expression, and noted the NFA reading: each element of $pd(r, x)$ is one state of an implicit NFA the original r denotes, alive after reading x . Extending this to parsing means pairing each surviving element of that set with enough information to reconstruct a parse tree once a particular strand turns out to be the winning one at the end of the input. Concretely, that is either an accumulated bit-string per strand (Section 6.2) or, in this project’s own extension, an accumulated tree-injection closure per strand (Section 6.4).

Either way the algorithm shape is the same, and Figure 6.1 draws it: maintain a *frontier* of (**residual**, **extra**) pairs, one per live strand, replace it at each character with the union of *pd* applied to every current strand, each contributing its own possibly-empty set of successor strands, and at the end of the input select the first strand in the frontier whose residual is nullable.

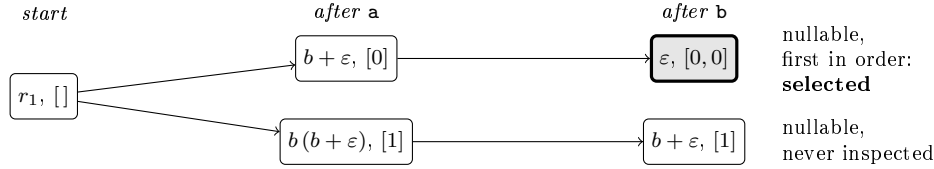


Figure 6.1: The frontier for $r_1 = (a+ab)(b+\varepsilon)$ against "ab", with `pderiv_bc`'s bits shown as the per-strand extra. One character advances every strand independently. At the end both strands are nullable, and selection takes the first in list order without comparing them, which is what makes the construction Greedy.

Why the result is Greedy, not POSIX. That last step, "the *first* nullable strand in the frontier", is where this construction's disambiguation policy is actually decided, and it is decided implicitly, as a consequence of how the frontier's growth is kept finite rather than by any explicit ordering rule analogous to Section 4.3.3's (A1) to (A4) and (K1) to (K4).

Antimirov's finiteness theorem (Section 3.3) guarantees the set of *distinct* partial derivatives is bounded, but nothing about the extended, tree-or-bit-carrying construction guarantees strands stay distinct as they accumulate: two different derivation paths can easily reach the identical residual *shape* by two different routes, each carrying its own bits or injection. Kept without pruning, this set would multiply extra copies of an already-explored residual instead of staying within the bounded, syntactic set *pd* itself guarantees. Pruned instead by comparing the accumulated bits or trees for POSIX-style "longest match wins", the way Section 4.3.3's ordering does, the comparison would need to look inside possibly-incomplete, still-growing accumulated state rather than a finished `ParseTree`, which is not what the ordering relation of Section 4.3.3 is defined over.

The Haskell reference's actual choice is `nub2`: deduplicate purely on residual *shape*, keeping whichever strand reached that shape *first*. This sidesteps the problem entirely by discarding the comparison altogether. Once a residual shape has been claimed by some strand, every later strand reaching the identical shape is simply dropped, regardless of what either one's accumulated bits or tree would eventually have produced. "First" here means first in the list-ordered traversal *pd*'s own case structure produces, which explores the left operand of $+$ before the right at every step, so "first claim wins" is exactly Greedy's leftmost-commits-immediately rule (Section 4.3.1) and not POSIX's longest-wins-with-left-as-tiebreak rule.

The trace states the consequence in one line, at the point where the decision is made:

```

SELECTION (first nullable residual, priority/list order)
any nullable residual in final frontier = true -> proceed to
  mkEpsBC + decode
  
```

There is no comparison here, because there is nothing to compare against: the moment

a nullable residual is found, scanning stops. That is the shaded node in Figure 6.1, and the strand below it is never examined even though it is nullable too.

This reading is this thesis’s own account of why the construction lands on Greedy rather than POSIX; neither primary source states it in these terms. Both parsers built in this chapter are verified empirically in Chapter 8 to compute exactly the Greedy answer wherever it is known to differ from POSIX, including the `paper_r1` and `paper_r2` examples of Section 4.3, which is the strongest evidence available for it in the absence of a reference construction to check against directly.

A family of witnesses. The two paper examples are not the only place the divergence is visible, and it is worth having a case where the pattern is simple enough that both answers can be read off by inspection. Take $r = (b + bb)^*$ against a run of `bs`. POSIX’s rule (A1) prefers the longer alternative at each iteration, while Greedy commits to the leftmost alternative that works, so the two policies partition the same input differently:

Input	POSIX	Greedy
"bb"	[Right (b, b)]	[Left b, Left b]
"bbb"	[Right (b, b), Left b]	[Left b, Left b, Left b]
"bbbbbb"	[Right (b, b) × 3]	[Left b] × 6

POSIX uses as few iterations as the longest-alternative rule allows, padding with a single `Left b` only when the length is odd; Greedy uses one iteration per character throughout, because b is the left alternative and it always matches. Both flatten back to the same input, and both are valid parse trees for it. Only the policy differs. These rows are the output of `cargo run - parse "(b|bb)*" "bb" -parser all`, which runs all five entry points on one input and prints the trees side by side.

6.2 Bit-Coded Variant

Sulzmann’s teaching-page Haskell transcript gives a bit-coded partial-derivative construction, `pDerivBC`, that this thesis’s `pderiv_bc` follows directly [Sulzmann, 2026]. It extends `pd` with per-strand bits the same way Section 5.4’s `\b` extends the Brzozowski derivative, but, notably, keeps those bits *external* to the regular-expression tree, as a plain list carried alongside a plain, un-annotated r , rather than folding them into an `ARegex`-shaped tree the way `\b` does.

Writing D_r for $pDerivBC(x, r)$ throughout, to keep the set comprehensions readable, the base and non-sequence cases are

$$\begin{aligned}
D_\varepsilon &= \{\} & D_\emptyset &= \{\} & & (\text{PB}_\varepsilon), (\text{PB}_\emptyset) \\
D_y &= \begin{cases} \{(\varepsilon, [])\} & x = y \\ \{\} & x \neq y \end{cases} & & & & (\text{PB}_{\text{lit}}) \\
D_{r+s} &= \text{nub2}(\{(r', 0:bs) \mid (r', bs) \in D_r\} \\
&\quad \cup \{(s', 1:bs) \mid (s', bs) \in D_s\}) & & & (\text{PB}_+) \\
D_{r^*} &= \text{nub2}(\{(smartC(r', r^*), 0:bs) \mid (r', bs) \in D_r\}) & & & (\text{PB}_*)
\end{aligned} \tag{6.1}$$

and the sequence case, Equation (6.2), splits on the nullability of the left factor exactly

as Equation (3.3) does:

$$\begin{aligned}
D_{rs} &= \{(smartC(r', s), bs) \mid (r', bs) \in D_r\} & (PB_{seq}), \text{ if } n(r) = \text{false} \\
D_{rs} &= nub2(\{(r's, bs) \mid (r', bs) \in D_r\} \\
&\quad \cup \{(s', mkEpsBC_r ++ bs) \mid (s', bs) \in D_s\}) & (PB_{seq}^n), \text{ if } n(r) = \text{true}
\end{aligned} \tag{6.2}$$

The names extend Section 3.3's P scheme, one PB rule per P rule, and the worked example below cites them with the binding of r and s given at each application. The two sequence cases differ in exactly one respect worth keeping in view: PB_{seq} builds its residual with $smartC$, while PB_{seq}^n builds the first of its two strands with a plain $r's$. Here $mkEpsBC_r$ is the plain- r analogue of Section 5.7.4's $mkEpsBC$, identical in structure but walking an un-annotated tree instead of an **ARegex**, and $nub2$ deduplicates a list of (**residual**, **bits**) pairs purely by comparing the residual, keeping the first occurrence. $smartC$ is the smart concatenation constructor of Section 3.3, collapsing $\varepsilon \cdot s$ to s .

The concatenation case's first strand deliberately keeps r' wrapped in a literal, non-smart-constructed $r's$ rather than $smartC$. This asymmetry is present in the reference itself and is preserved here rather than "corrected", since correctness of this construction is judged against literal agreement with that reference (Chapter 8) and not against an independently-derived ideal form. Section 6.4 returns to the same question from the other side, where the choice between the two constructors is not free at all.

6.3 Rust Implementation: The `pderiv_bc` Parser

`src/parsers/pderiv_bc/pderiv.rs` transcribes $pDerivBC$ case for case. Each D_r becomes a `Vec<(Regex, Vec<bool>)>` rather than a set. Order matters here, encoding Greedy priority directly in list position, so a `HashSet` would lose exactly the information the construction depends on. `mk_eps_bits` implements $mkEpsBC_r$; `nub2` deduplicates on the residual `Regex` alone, via a `HashSet<Regex>` used only to test membership, keeping the first-seen pair for each distinct residual and discarding the rest. This is the one place in the codebase where `Regex`'s derived `Hash` implementation (Section 2.3) is load-bearing. `pderiv_bc`'s own `Seq` case matches the reference literally, down to the same asymmetric use of plain `Regex::seq` rather than `smart_seq` for its first strand.

`step_frontier` and `select_bits` (`src/parsers/pderiv_bc/parse.rs`) implement the frontier-driving loop of Section 6.1. At each character, `step_frontier` replaces every (**residual**, **bits**) pair in the current frontier with the union of `pderiv_bc` applied to it, concatenating each result's bits onto that strand's already-accumulated prefix. At the end of the input, `select_bits` scans the frontier in list order and returns the first nullable strand's `bits ++ mk_eps_bits(residual)`. `parse_pderiv_bc` drives this loop to completion and passes the winning bit-string, together with the *original* `Regex` and never the derived residual, to `decode` (Section 5.7.5). That is the same shared decoder Chapter 5's `deriv_bc` uses, since both constructions end up at the identical (**Regex**, **bits**) shape by the time decoding is needed, whatever route each took to get there.

The worked example, one step at a time. Run Section 5.4's $r_1 = (a + ab)(b + \varepsilon)$ against `ab` through `pderiv_bc` instead of `deriv_bc`: same regex, same input, different parser, so that Greedy and POSIX can be seen disagreeing on the same worked example rather than only being asserted to. The frontier starts as a single strand, the whole regex

with no bits yet, which is the trace's INITIAL FRONTIER section and the leftmost node of Figure 6.1.

Deriving with respect to **a**, the outer node binds $r = a + ab$ and $s = b + \varepsilon$. That r is not nullable, by N_+ over two non-nullable literals and a concatenation, so PB_{seq} applies and no second strand is contributed by s . The strands that do appear come from PB_+ one level down, binding $r = a$ and $s = ab$ and tagging the two alternatives 0 and 1. Where the plain bit-coded construction kept one expression, there are now two strands:

```
Step 1 (char: 'a', position 1)
  rule: pDerivBC('a', r1.r2) [nullable(r1)=false]
      = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
  frontier after (2 residual(s)):
    [0] Alt(Lit('b'), Eps)    bits = [0]
    [1] Seq(Lit('b'), Alt(Lit('b'), Eps)) bits = [1]
```

Strand [0] came from a 's own derivative, ε , which SC then absorbed against $(b + \varepsilon)$ by its first branch, leaving $Alt(Lit('b'), Eps)$ alone; it carries [0] for $(a + ab)$'s left disjunct. Strand [1] came from ab 's derivative, b , which SC kept as a concatenation by its second branch since $b \neq \varepsilon$; it carries [1] for the right disjunct. Both bits were emitted by PB_+ one level down, and PB_{seq} shown in the rule line carried them outward unchanged, which is why neither strand's bits grew at this step.

Deriving with respect to **b** steps *both* strands independently, and the trace prints a rule line per strand, which is where the frontier's per-strand character becomes explicit:

```
Step 2 (char: 'b', position 2)
  from [0]:
    rule: pDerivBC('b', r1+r2) = tag(0, pDerivBC('b',r1)) ++
      tag(1, pDerivBC('b',r2)), nub2
    -> (Eps, [0])
  from [1]:
    rule: pDerivBC('b', r1.r2) [nullable(r1)=false]
      = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
    -> (Alt(Lit('b'), Eps), [])
  frontier after (2 residual(s)):
    [0] Eps    bits = [00]
    [1] Alt(Lit('b'), Eps) bits = [1]
```

Two different rules fired on the same character, because the two strands are different expressions and each is matched against the rule set independently. Strand [0] is an Alt , so PB_+ applies with $r = b$ and $s = \varepsilon$: the left disjunct contributes ε tagged 0, and the right contributes nothing, since PB_ε returns the empty set. Strand [1] is a Seq binding $r = b$ and $s = b + \varepsilon$, and that r is a bare literal and so not nullable, so PB_{seq} applies and derives only the left factor. The accumulated bits differ correspondingly: strand [0] gains a 0 and reaches [00], while strand [1] gains nothing and stays at [1].

Selection then scans this final frontier in list order and takes the *first* nullable strand. Strand [0], $(Eps, [00])$, is checked and accepted before strand [1] is even inspected, regardless of strand [1] also being nullable:

```
selected residual's accumulated bits ++ mkEpsBC(residual) = [00]
```

```
decode((a|ab)(b|), [00]):
  = (Left a, Left b)
```

This is Greedy’s leftmost-commits rule made mechanical. Strand [0] is exactly the "matched **a**, then **b** separately" reading, the first alternative *pDerivBC* generated back at the first derivative step, and it wins here purely because it appears first in the frontier, not because any length comparison favors it. The decoded tree is the *shorter* reading, matching **a** and then **b** separately, and it is the opposite answer from Section 5.4’s (*Right(a, b)*, *Right()*) on the identical regex and input. Nothing about r_1 or "ab" changed between the two traces; only the disambiguation policy governing which nullable strand or branch wins did, exactly the distinction Section 6.1 argues is built into nub2’s shape-only, first-occurrence-wins deduplication.

Where nub2 is and is not applied. nub2 appears inside `pderiv_bc`, once per case that can produce more than one strand, and nowhere else. In particular `step_frontier` concatenates the per-strand results into the next frontier without deduplicating across them, so the guarantee nub2 provides is local to a single strand’s expansion and does not extend to the frontier as a whole. Two strands that were distinct at step i can converge on the same residual at step $i + 1$ and both survive. Section 6.5 measures what that costs.

6.4 Rust Implementation: The `pderiv_std` Parser

`pderiv_std` (`src/parsers/pderiv_std/`) is this project’s own construction, with no counterpart in either reference. Its purpose is to stand to `pderiv_bc` as `deriv_std` stands to `deriv_bc`: the same traversal, the same disambiguation policy, and the same answers, but building a `ParseTree` directly instead of a bit-string that has to be decoded afterward. Having both means the Greedy answer is computed twice by routes that share almost no code, which is what makes the agreement check of Section 6.6 worth anything.

What each strand carries. Where `pderiv_bc` pairs each residual with a `Vec<bool>`, `pderiv_std` pairs it with an *injection closure* instead, defined in this module’s `inject.rs`:

```
pub(crate) type Inj = Rc<dyn Fn(ParseTree) -> ParseTree>;
```

A strand’s `Inj` is a function that, given the parse tree of that strand’s *residual*, returns the parse tree of the *original* expression. The frontier starts with one strand carrying `identity_inj()`, and each step composes the step’s own injection inside the strand’s accumulated one. Figure 6.2 contrasts the two representations after two steps.

This is the same idea as Chapter 5’s `inject`, with one difference that matters: `deriv_std` applies `inject` once per character during an explicit backward pass, re-deriving the expression sequence it needs as it unwinds, whereas `pderiv_std` never runs a backward pass at all. It builds the composition forward, one layer per step, and applies the finished composition exactly once at the end. In that sense `pderiv_std` is as single-pass as `pderiv_bc` is; the closure chain plays the role the bit-string plays there.

The `Rc` is required rather than incidental. A single step can expand one strand into several, and every one of those successors needs the same parent injection, so the parent has to be shared rather than cloned or moved. `Rc<dyn Fn(...)>` gives shared ownership of a



Figure 6.2: What one strand holds after two derivative steps. The bit-coded strand accumulates into a flat buffer; the tree-building strand accumulates a chain of heap-allocated closures, one link per step, each holding the next. Both are applied only once, to the winning strand, at the end of the input.

heap-allocated closure at the cost of a reference-count bump per successor. Section 8.5.2 measures what this costs against `pderiv_bc`'s flat `Vec<bool>`; the structural point here is the one Figure 6.2 draws, that a strand's accumulated state is a chain whose length grows with the input where `pderiv_bc`'s is a contiguous buffer.

`pderiv_tree`. `src/parsers/pderiv_std/pderiv_tree.rs` is the (Regex, Inj) analogue of `pderiv_bc`, case for case, with each case's bit-tagging replaced by the tree constructor that bit would have decoded to:

Case	<code>pderiv_bc</code> attaches	<code>pderiv_tree</code> attaches
$\varepsilon, \emptyset$	no strands	no strands
x (matching)	<code>[]</code>	<code> _ Char(x)</code>
$r + s$, left	<code>0:bs</code>	<code> v Left(inj1(v))</code>
$r + s$, right	<code>1:bs</code>	<code> v Right(inj2(v))</code>
rs , continue r	<code>bs</code>	<code> Pair(v1,v2) Pair(inj1(v1), v2)</code>
rs , jump to s	<code>mkEpsBC_r ++ bs</code>	<code> v2 Pair(mk_eps(r), inj2(v2))</code>
r^*	<code>0:bs</code>	<code> Pair(v1,vs) Star(inj1(v1) : vs)</code>

Every row is the same decision, expressed once as a bit and once as a constructor. The correspondence is exact enough that the `Seq` case's "jump to s " strand calls the shared `mk_eps` (Section 5.6.2) at precisely the point `pderiv_bc` calls `mk_eps_bits`, and for the same reason: the left factor matched empty, so its contribution to the pair has to be synthesized rather than derived. `nub_tree` is the `Inj`-carrying `nub2`, deduplicating on the residual alone and keeping the first occurrence, so the Greedy priority order is preserved identically.

Why the smart constructor cannot be used here. `pderiv_bc` is free to apply `smartC` in its non-nullable `Seq` case and in its `Star` case, so as to collapse $\varepsilon \cdot s$ down to s . `pderiv_tree` is not, and uses plain `Regex::seq` in every position where its sibling uses the smart one. The reason is a direct consequence of what an `Inj` is applied to.

Selection ends by calling `inj(mk_eps(r))` on the winning strand, so each accumulated closure is applied to a tree whose *shape is determined by its own residual*. The `Seq` strands' closures pattern-match on `ParseTree::Pair`, which is what `mk_eps` produces for a `Seq` residual. Collapsing `Seq(Eps, s)` to s would make `mk_eps` produce s 's own tree shape instead, and the closure expecting a `Pair` would receive something else. The residual and the injection are two halves of one invariant, and a smart constructor rewrites one half

without the other.

This is not a speculative concern. Substitute `smart_seq` for `Regex::seq` in the two `Seq` strands of `pderiv_tree`, and the test suite fails immediately, on the simplest possible sequence pattern, with the panic that case’s own guard raises:

```
---- parsers::pderiv_std::parse::tests::seq_matches stdout ----
thread '...::seq_matches' panicked at
  src/parsers/pderiv_std/pderiv_tree.rs:71:38:
pderiv_tree: Seq(non-nullable) strand expected Pair, got Char('b')
```

The panics in `pderiv_tree` therefore serve the same purpose as those in `inject` (Section 5.6.3) and `decode` (Section 5.7.5): they encode a precondition that holds by construction, and they convert a violation into an immediate, located failure rather than a silently wrong parse tree.

This is also why the two parsers’ frontiers look different in the traces even though they compute the same answer. After one step on r_1 , `pderiv_bc`’s first residual is `Alt(Lit('b'), Eps)` and `pderiv_std`’s is `Seq(Eps, Alt(Lit('b'), Eps))`: the same language, the same strand, one of them still carrying the `Eps` its sibling discarded. Because `nub_tree` deduplicates on residual syntax, the two parsers can in principle prune at different points, which makes their agreement on every tested case a stronger check than it would be if they shared a residual representation.

Driving the frontier. The driver in `src/parsers/pderiv_std/parse.rs` mirrors its `pderiv_bc` counterpart exactly. `step_frontier` expands every strand and composes injections; `select` scans the final frontier in list order and returns `inj(mk_eps(r))` for the first nullable residual; `parse_pderiv_std` runs the loop, returning `None` as soon as the frontier becomes empty, since a frontier with no strands can never regain one. As with `pderiv_bc`, no deduplication is applied across the flattened frontier, only within each strand’s own expansion.

The same worked example, and what the trace cannot show. Running r_1 against `ab` through `pderiv_std` produces the same two-strand fork at step 1, and the same winner at step 2. The one visible difference from `pderiv_bc`’s trace is a recurring annotation:

```
-> Seq(Eps, Alt(Lit('b'), Eps)) (+ injection closure, not
  printable)
-> Seq(Seq(Eps, Lit('b')), Alt(Lit('b'), Eps)) (+ injection
  closure, not printable)
```

This is the one place where this parser’s state is less inspectable than its bit-coded sibling’s. A `Vec<bool>` can be printed; an `Rc<dyn Fn>` cannot. The trace can therefore show which residuals are alive and which rule produced each, but not what tree each strand would build if it won, which is precisely the information `pderiv_bc`’s `bits = [00]` annotations do carry. That limitation is a direct cost of the representation choice drawn in Figure 6.2, and Section 9.3.4 returns to it as a general point about what a tracing layer can and cannot recover.

The residuals themselves also differ from `pderiv_bc`’s, in exactly the way the smart-constructor argument above predicts. Where that parser lists `Alt(Lit('b'), Eps)`, this

one lists `Seq(Eps, Alt(Lit('b'), Eps))`; where that one lists `Seq(Lit('b'), ...)`, this one lists `Seq(Seq(Eps, Lit('b')), ...)`. The two parsers are not running over the same residuals at all, and still select the same tree.

6.5 Frontier Growth

Both parsers of this chapter inherit Antimirov’s finiteness result for the set of distinct partial derivatives (Section 3.3), and neither inherits a bound on the size of the frontier, for the reason Section 6.1 identified: the frontier holds (`residual`, `extra`) pairs, and only the residual component is subject to *pd*’s bound. This section measures the consequence.

Take $(a + (b + ab))^*$, the `paper_r2` expression, against $(ab)^n$, and record both the number of strands in the frontier after each character and the number of *distinct* residuals among them.

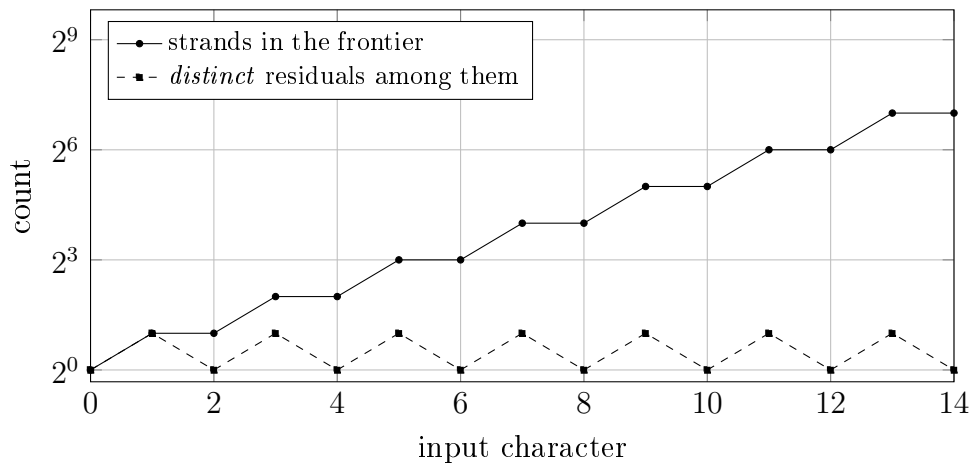


Figure 6.3: Frontier size against distinct residual count for $(a + (b + ab))^*$ on $(ab)^n$, logarithmic vertical scale. The frontier doubles every two characters while the number of distinct residuals never exceeds two: every strand beyond the second is a duplicate carrying different accumulated state. Regenerate with `cargo run --release --example thesis_figures -- frontier; tests/test_thesis_figures.rs` pins these values.

Every strand beyond the second is a duplicate of one already present, differing only in the bits or injection it carries. `nub2` does not remove them because it never sees them together: it runs inside `pderiv_bc`, on the successors of one strand, and the duplicates here arise *between* strands, at the point `step_frontier` concatenates the per-strand results.

This is the partial-derivative counterpart of Section 5.5’s finding for `deriv_bc`, and the two have the same root cause seen from different sides. There, alternation branches that were structurally identical survived deduplication because their bit annotations made them syntactically distinct. Here, residuals that are syntactically identical survive because the deduplication is never applied at the level where they meet. In both cases the bounded object is the set of residual *shapes*, and in both cases what actually grows is the set of shapes paired with accumulated parse-tree state.

Deduplicating across the frontier. The obvious question is whether the duplicates can simply be dropped. Moving `nub2` out of `pderiv_bc` and applying it across the whole

flattened frontier, keeping the first occurrence, is a change of one deduplication site, and it preserves the Greedy priority order for the same reason `nub2` does inside a single expansion: the frontier is built in priority order, so the first occurrence of a residual is the highest-priority strand that reaches it.

Whether that argument is enough to preserve the selected *tree* is a different question, and this thesis does not prove it. It was tested instead, by running both variants over the 300 generated expressions and 39 inputs of Section 8.4.2, 11 700 cases in total. The two agree on every case, on membership and on the exact tree returned, and the peak frontier over the whole run falls from 170 strands to 4.

The shipped `pderiv_bc` nonetheless keeps `nub2` where the reference puts it. The reason is the one stated in Section 6.2: this parser's correctness criterion is literal agreement with `pDerivBC`, and a change that is verified on 11 700 cases but not proved is a change to the thing being verified against. Section 9.5.2 records it as an available optimization rather than an applied one, with the measurement above as the evidence for its value.

6.6 Comparing the Two Policies

Section 4.3 constructed, by hand, the *second* valid derivation of $r_1 = (a+ab)(b+\varepsilon)$ against `ab` (`paper_r1`) that Chapter 5 deliberately left aside: $v' = (\text{Left } a, \text{Left } b)$, matching the *first* alternative of $a + ab$ against `a` alone rather than the second alternative against `ab` in one step. That is the Greedy result, and the two preceding sections have shown both of this chapter's parsers computing it. Figure 6.4 sets the two answers side by side.

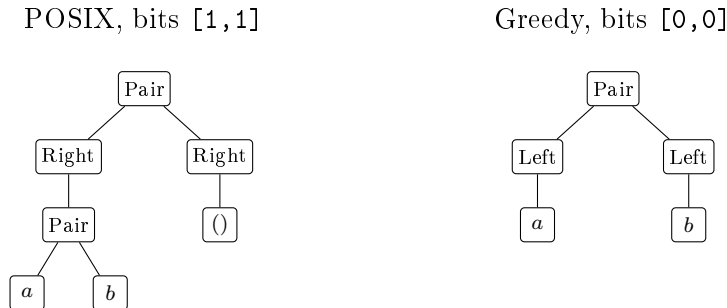


Figure 6.4: The two parse trees for $(a+ab)(b+\varepsilon)$ against `"ab"`. Both flatten to `"ab"` and both are valid. POSIX takes the second alternative of $a + ab$, consuming both characters at once and leaving $b + \varepsilon$ to match empty; Greedy takes the first, leaving $b + \varepsilon$ to consume the `b`.

The bit-strings underneath make the comparison exact. Four alternation choices exist in r_1 , two of which are reached on any given parse, and the two parsers disagree on both: `[1,1]` against `[0,0]`. Nothing else differs. The same expression, the same input, the same membership answer, and the entire distinction between the two chapters compressed into two bits.

Failure behaviour is shared. Against `b` instead of `ab`, the first partial derivative of r_1 produces no residual at all, since neither alternative of $a + ab$ begins with `b`:

```
rule: pDerivBC('b', r1.r2) [nullable(r1)=false]
```

```

    = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
-> []
frontier after (0 residual(s)):
no nullable residual in this frontier [FAIL]

```

An empty frontier is the partial-derivative counterpart of `deriv_bc` simplifying to `phi` (Section 5.8), and it is detected the same way but earlier: both `parse_pderiv_std` and `parse_pderiv_bc` return as soon as the frontier empties, without consuming the rest of the input, because a frontier with no strands has no way to acquire one. For a long input whose first character already fails, that is the difference between one step and n .

Both parsers then emit the same error summary as Chapter 5's two, character for character:

```

Failure at position:  1 (1-indexed)
Found:                'b'
Expected:              'a'

```

The two chapters' constructions disagree, deliberately, on which tree a successful match produces, Greedy against POSIX, but agree exactly on the Boolean question of membership and on the diagnostics reported for a non-match. Neither disambiguation policy nor the choice between bits and closures is ever reached until after the underlying derivative or partial-derivative walk has already found at least one live residual. The full reports are Appendix ?? and Appendix ??.

All five entry points at once. The `-parser all` mode runs every parser on one input and checks the three invariants this and the previous chapter claim, without any diagnostics level being selected:

```

$ cargo run -- parse "(a|ab)(b|)" "ab" --parser all
Regex: (a|ab)(b|)
Input: "ab"

```

Parser	Policy	Result
DERIV_REC	POSIX	(Right (a, b), Right ()) -> "ab"
DERIV_LOOP	POSIX	(Right (a, b), Right ()) -> "ab"
DERIV_BC	POSIX	(Right (a, b), Right ()) -> "ab"
PDERIV_STD	GREEDY	(Left a, Left b) -> "ab"
PDERIV_BC	GREEDY	(Left a, Left b) -> "ab"

```

POSIX parsers agree
PDERIV parsers agree
GREEDY agrees with POSIX on membership

```

The three closing lines are the invariants themselves, checked on this input: the three POSIX entry points must return identical trees, the two Greedy ones must return identical trees, and all five must agree on whether the input matched. Nothing requires the two *policies* to agree, and on this input they do not, which is the entire subject of both chapters. Section 8.4 runs the same three checks across the full corpus rather than a single input.

Frontend: Parsing External Regular Expression Syntax

Every example so far in this thesis has been written directly in `Regex` terms, the six-constructor core algebra of Section 2.3 that `deriv` and `pderiv` operate on. No real regular-expression user writes that way. A Snort rule, a SpamAssassin filter, or an ordinary call to `cargo run - match` arrives as an ordinary PCRE-style pattern *string*, `"[a-c]+\d{2}"` rather than `Seq(Star(...), ...)`. `src/frontend/` is the module that closes this gap: it parses that string into its own surface-syntax AST, then lowers that AST into the `Regex` every parser in Chapters 5 and 6 already understands.

It is modeled throughout on the `ExtPattern`, `Parse` and `Translate` modules of `Text.Regex.PDeriv`, from luzhuomi/`regex-pderiv`, Kenny Lu's own reference implementation of Sulzmann and Lu [2012]'s partial-derivative submatching. That is a second, independent reference alongside Sulzmann and Lu [2014]'s own teaching-page material (Section 9.2.1), used here specifically because the flops14-extended paper itself has nothing to say about surface syntax: POSIX parsing is defined directly over the same six-constructor `Regex` algebra this thesis's core inherits, with no PCRE-facing frontend of its own.

Section 8.3's real-world corpus depends on everything described here, surface-syntax parsing, desugaring, and the substring-search padding scheme, to turn raw Suricata, SpamAssassin and RegexLib rule text into `Regex` values in the first place. The chapter closes (Section 7.5) with the price that translation charges: how large a `Regex` an ordinary-looking pattern actually becomes, which is what the four parsers then have to walk once per input character.

7.1 Two ASTs, One Lowering Step

`src/frontend/` is built around a deliberate separation into two abstract syntax trees connected by a single function, rather than parsing pattern text directly into `Regex`. Figure 7.1 gives the whole module in one view: a common path that every entry point follows, and two optional stages that only some of them switch on.

`ExtPat` (`ext_pattern.rs`) is the *external*, surface-syntax AST: marking and non-marking groups, `{m,n}` bounds, `\d/\w/\s` shorthands, `^/$` anchors, and a `greedy: bool` flag on every quantifier (`a*` against `a*?`). It holds everything a PCRE-style string can express, kept exactly as written, with nothing desugared yet. `Regex` (Section 2.3) is the *internal*, six-constructor core every parser derives or partial-derives directly: `Phi` | `Eps` | `Lit` | `Seq` | `Alt` | `Star`, with no groups, no bounds, no anchors and no laziness flag. Anything

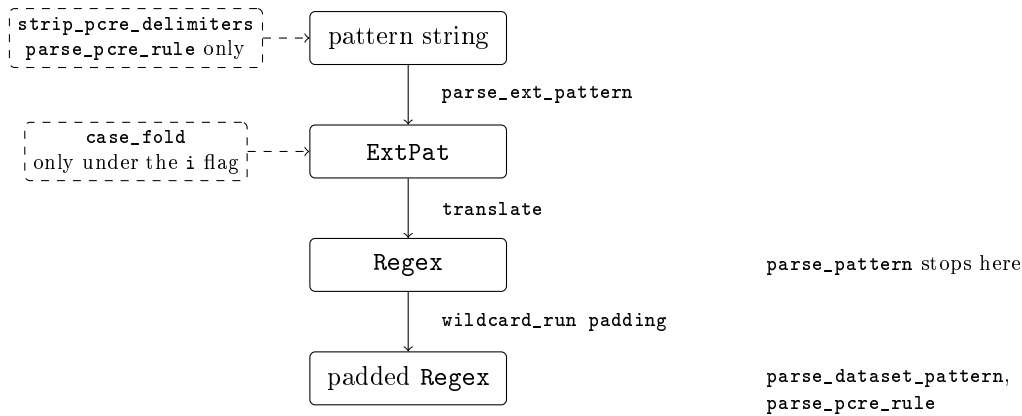


Figure 7.1: The frontend as a pipeline. The solid path is common to all three entry points; the dashed stages apply only where marked. `parse_pattern` stops at `Regex` and means full-string equality, while `parse_dataset_pattern` and `parse_pcre_rule` continue into the padding of Section 7.4.

`ExtPat` can say that `Regex` cannot represent is desugared away by `translate` (Section 7.3) or, for `^` and `$` specifically, erased into `Eps` and handled one level up instead (Section 7.4).

Keeping `ExtPat` as an honest AST in its own right, rather than routing straight from the parser into `Regex` in a single pass, is what makes `translate` independently testable. The unit tests in `translate.rs` construct `ExtPat` values by hand and check their `Regex` output directly, with no dependency on the recursive-descent logic in `parse.rs` at all. It is the same separation-of-concerns argument that Section 5.6.1 makes for keeping `ParseTree` construction separate from `flatten` and `decode`, applied one layer further out, at the point where a PCRE-style string first becomes structured data at all.

The separation earns its keep a second time at the entry points. `mod.rs` exposes three of them, all sharing one parser and one `translate`, and differing only in what they do to the `ExtPat` in between:

Entry point	Behaviour
<code>parse_pattern</code>	<code>translate</code> alone: full-string match
<code>parse_dataset_pattern</code>	plus anchor-aware padding (Section 7.4)
<code>parse_pcre_rule</code>	plus <code>/.../i</code> delimiter stripping and case folding

Every example in Chapters 5 and 6, and every invocation of `cargo run - parse`, goes through `parse_pattern`, which is why `"(a|ab)(b|)"` against `"ab"` means the whole input and not a substring of it. The corpus of Section 8.3 goes through the other two, which is where real rule text is handled.

7.2 Parsing Surface Syntax

`frontend/parse.rs` is an ordinary recursive-descent parser over a `Chars` iterator, structured directly after the grammar of `Text.Regex.PDeriv.Parse`. Its productions correspond one for one, each handling exactly the surface-syntax layer its reference counterpart does:

Reference	This frontend	Layer handled
p_ere	parse_or	alternation
p_branch	parse_concat	concatenation
p_exp	parse_postfixed	postfix quantifiers
p_atom	parse_atom	a single atom
p_group	parse_group	(...) forms
p_charclass	parse_charclass	[...]
p_enum	parse_class_enum	class members
p_bound	parse_bound	{m,n}

The table is a list of names; Figure 7.2 is what those names mean operationally. Each production consumes exactly one syntactic layer and then calls the next one down, so the order of the calls *is* the precedence order, and the single back-edge from `parse_group` is what makes the grammar recursive at all.

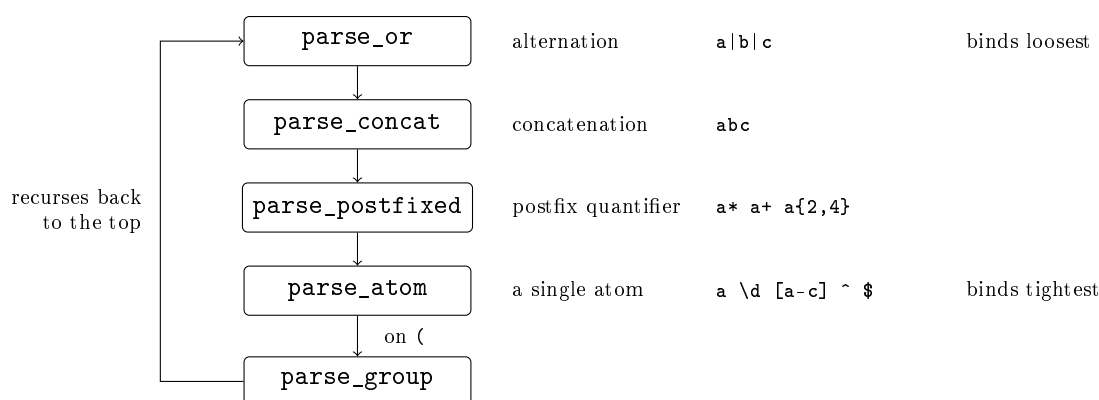


Figure 7.2: The recursive-descent ladder of `frontend/parse.rs`. Each production strips one layer and delegates the rest downward, so precedence falls out of the call order. `parse_group` is the only production that climbs back to the top, which is why parentheses override precedence and why nesting is unbounded.

Four deliberate departures from the reference:

- **Class shorthands are PCRE character classes, not literal escapes.** The reference’s `Translate.hs` reads a bare `EEscape c` as the literal character `c` always; this frontend special-cases `d/D/w/W/s/S` into character classes (Section 7.3’s `alphabet.rs`) instead, since real datasets use them that way throughout.
- **Backreferences and lookahead are rejected outright.** The backreferences `\1` to `\9`, and the four lookahead openers `(?=`, `(?!`, `(?<=` and `(?<!`, all produce a descriptive parse error rather than the reference’s literal-digit or three-digit octal-ASCII escape reading. Both describe languages that are not regular in the formal sense this thesis’s entire construction depends on, so silently mis-parsing them into *some* `Regex` would be a worse outcome than refusing them, and Section 8.3’s extraction pipeline depends on this refusal being loud and categorized rather than silent.
- **Named groups are accepted.** `(?<name>...)` and `(?P<name>...)` parse successfully, and the name is dropped during `translate`. A name carries no meaning once lowered to `Regex`, since this project has no capture-group semantics at all, but

rejecting a construct this common in real Suricata and RegexLib rules purely because the reference parser predates it would reject working patterns for no benefit. `(?P=name)`, a named *backreference*, is rejected along with the numeric ones.

- **`\xHH` hex escapes are accepted**, another addition absent from the reference, for the same reason.

Two small extensions exist for this thesis’s own sake rather than for the datasets. A literal ε character in the pattern text parses to `ExtPat::Eps`, and a literal `∅` to `ExtPat::Never`, so that Section 2.1’s own notation for ε and `∅` can be written directly on the command line. PCRE syntax has no way to spell either, and the alternative used throughout Chapters 5 and 6 is an empty alternative: `(b|)` is $b + \varepsilon$, because `parse_concat` returns `ExtPat::Empty` when a branch is empty rather than failing.

Bounds are parsed speculatively. `parse_bound` is the one place where the parser backtracks. A `{` in a pattern is only sometimes the start of a repetition bound, so `parse_bound` clones the iterator, tries to read a complete `{m}`, `{m,}` or `{m,n}` specification from the clone, and commits the clone back only on success. On failure the `{` is left unconsumed, and since `{` is in the parser’s `SPECIALS` set, the next `parse_atom` call reports it as unexpected.

This has a consequence worth recording, because it makes one check in `translate` unreachable. An inverted bound such as `a{4,2}` is rejected by `try_parse_bound_spec`, which returns `None` when the upper bound is below the lower one, so the error the user sees comes from the *atom* parser:

```
$ cargo run -- parse "a{4,2}" "aa"
Regex parse error: unexpected special character '{' here
```

rather than from `desugar_bound`’s own "upper bound is less than lower bound" message, which no input reaching it through `parse_pattern` can trigger. The message is less specific than the one that exists for the condition, which is worth knowing when reading extraction-failure categories in Section 8.3.

`parse_bound` additionally accepts a lazy-quantifier marker (`a*?`, `a+?`, `a{2,4}?`) and records it as `ExtPat`’s `greedy`: `bool` flag: correctly read, but, as Section 7.3 explains, not honored by `translate`. Grouping, alternation and concatenation otherwise follow the reference’s precedence exactly: alternation binds loosest, then concatenation, then postfix quantifiers, then a single atom, which may be a literal, an escape, a character class, a group or an anchor.

7.3 Translating ExtPat into Regex

`translate` (`frontend/translate.rs`) is a single structural recursion over `ExtPat`, one match arm per constructor, each either a direct one-to-one mapping or an explicit desugaring:

ExtPat	Regex
Empty, Eps	Eps
Never	Phi
Char(c)	Lit(c)
Group/GroupNonMarking(r)	translate(r) (grouping is precedence-only)
Or([r1, ..., rn])	left-folded Alt chain
Concat([r1, ..., rn])	left-folded Seq chain
Opt(r)	Alt(translate(r), Eps)
Plus(r)	Seq(translate(r), Star(translate(r)))
Star(r)	Star(translate(r))
Carat, Dollar	Eps (see Section 7.4)
Dot, Any(cs), NoneOf(cs)	sorted, deduplicated Alt of Lit
Escape(c)	class shorthand, or Lit(c)
Bound(r, lo, hi)	unrolled exact/optional repetition, below

The left-folding of `Or` and `Concat` is what makes the associativity convention of Section 2.1 concrete: `abc` becomes `Seq(Seq(a, b), c)`, and it is that tree, not the string, that every derivative in this thesis is taken of.

Character classes are ordinary alternation, not a primitive. `Any`, `NoneOf`, `Dot` and the class-shorthand escapes all route through `alt_of_chars`, which sorts and deduplicates the character set and left-folds it into a plain `Alt` chain of `Lit` nodes. `alphabet.rs` supplies the underlying sets: `alphabet()`, which is 98 characters (printable ASCII plus tab, newline and carriage return), `digit_chars/word_chars/space_chars` for `\d/\w/\s`, and `complement` (the alphabet minus a given set) for their negated and uppercase counterparts and for `[^...]`.

There is no dedicated character-class `Regex` variant anywhere in this project, and Section 9.5.2 returns to this as an unexploited optimization. Every one of the four parsers only ever needs to understand `Regex`'s original six constructors, at the cost of a `\d` inside a real pattern becoming a ten-way `Alt` that the parser walks one branch at a time on every character. Section 7.5 measures how large that cost actually is.

Two consequences of drawing the alphabet this way are worth stating, since both are departures from PCRE that a rule author would not expect. The alphabet is ASCII only, so a negated class `[^a]` denotes 97 specific characters rather than "every character except a", and a non-ASCII input character matches no class at all. And `alphabet()` *includes* newline, so `.` here matches a newline where PCRE's `.` by default does not.

Bounds unroll literally, capped against blowup. `desugar_bound` builds `r` repeated exactly `lo` times, via `repeat_exact`, a left-folded `Seq` chain seeded with `Eps`, then appends either `Star(r)` for an unbounded upper end (`r{2,}`) or `hi - lo` further nested optional copies for a bounded one (`r{2,4}`), each layer of `optional_extra` being `Alt(Eps, Seq(r, ...))`. An exact bound `r{2}` takes a third path, returning `repeat_exact` alone with nothing appended. The `Eps` seed stays in the result rather than being smart-constructed away, so `a{2}` lowers to `Seq(Seq(Eps, Lit('a')), Lit('a'))` rather than `Seq(Lit('a'), Lit('a'))`. That costs one node and one nullable-test per derivative step, and it keeps `repeat_exact` a single unconditional fold with `Eps` as the natural identity for `lo == 0`.

Unlike every other desugaring in this table, this one can make a pattern arbitrarily large

from a short piece of source text. `r{1586,}`, which is Section 8.3's own example of a rejected real-world bound, would otherwise unroll into 1586 nested `Seq` nodes before the `Star` even starts. `MAX_BOUND`, at 1000 repetitions, is this project's own safety limit rather than something carried over from the reference, and `translate` refuses any bound past it with a descriptive error rather than silently building an oversized `Regex`:

```
$ cargo run -- parse "a{1586,}" "aa"
Regex parse error: bound {1586,} exceeds this frontend's cap of
1000 repetitions (literal unrolling would blow up the regex size)
```

Greedy and lazy quantifier syntax is parsed but not honored. Every `Opt`, `Plus`, `Star` and `Bound` match arm above ignores the `greedy` flag that `parse_bound` (Section 7.2) recorded, because `Regex` has no per-operator laziness concept at all. POSIX-versus-Greedy disambiguation in this thesis is a whole-parser policy, `deriv_std/deriv_bc` against `pderiv_std/pderiv_bc` (Section 4.3), rather than a per-quantifier one. `a*` and `a*?` therefore produce identical `Regex` values.

A pattern mixing lazy and greedy quantifiers, expecting the per-operator control PCRE engines provide, will parse successfully but not behave as its author intended. This is a genuine expressiveness gap between the surface syntax this frontend accepts and what `Regex` can distinguish, noted here rather than left to be discovered. The flag is nonetheless carried on `ExtPat` rather than discarded at parse time, so that the information survives to the one layer that could act on it if per-quantifier policy were ever added.

Case folding. `parse_pcre_rule` handles a `/PATTERN/FLAGS` wrapper, the form Suricata and SpamAssassin rules are written in. `strip_pcre_delimiters` splits the body from the flags, passing a pattern through unchanged if it is not `/-`delimited, and the `i` flag triggers `case_fold` on the `ExtPat` before `translate` runs. `case_fold` rewrites each literal letter into a two-way `Any`, so `a` becomes `[aA]`, and leaves digits, punctuation and class shorthands untouched, since those are already case-agnostic. Folding at the `ExtPat` layer rather than after lowering is what keeps this to a single recursion: at the `Regex` layer the same change would have to distinguish literals that came from a class from literals that came from source text, a distinction `translate` has by then already erased.

7.4 Substring Search

No parser in Chapters 5 and 6 answers any question but "does this entire input derive to nullable". There is no "found somewhere inside a longer string" mode anywhere below the frontend, and neither Sulzmann and Lu [2014] nor Sulzmann's teaching-page material defines one: POSIX and Antimirov derivatives are inherently full-match operations. Real rules, though, are *written* as search patterns. A Suricata rule fires on seeing its pattern anywhere in a packet payload, not on the payload being exactly that pattern.

`frontend/mod.rs` therefore implements substring search entirely at this layer, before a `Regex` ever reaches a parser, by padding the translated core with `wildcard_run()`, a `Star` over a 98-way `Alt` of the full alphabet meaning "any run of characters at all", on whichever side lacks an explicit anchor. Figure 7.3 gives the four cases, with the size of each lowered result alongside.

<code>^abc\$</code>	<table><tr><td>abc</td></tr></table>	abc	9 nodes, no padding		
abc					
<code>^abc</code>	<table><tr><td>abc</td><td>Σ^*</td></tr></table>	abc	Σ^*	204 nodes, end padded	
abc	Σ^*				
<code>abc\$</code>	<table><tr><td>Σ^*</td><td>abc</td></tr></table>	Σ^*	abc	204 nodes, start padded	
Σ^*	abc				
<code>abc</code>	<table><tr><td>Σ^*</td><td>abc</td><td>Σ^*</td></tr></table>	Σ^*	abc	Σ^*	399 nodes, both sides padded
Σ^*	abc	Σ^*			

Figure 7.3: Anchor-aware padding under the search entry points. Each Σ^* is one `wildcard_run()`, 196 nodes on its own, so the padding dominates the lowered expression whenever the pattern is short. The fully anchored case still costs 9 rather than 5 nodes, because the erased `^` and `$` leave an `Eps` apiece behind in the concatenation.

`starts_with_carat` and `ends_with_dollar` (`ext_pattern.rs`) decide which sides need padding by walking into `Concat` and `Group` to find a leading `^` or trailing `$` even when nested, before `Carat` and `Dollar` are themselves translated away to `Eps`. Under the search-oriented entry points `parse_dataset_pattern` and `parse_pcre_rule`, as against `parse_pattern`’s plain full-string equality used everywhere else in this thesis’s own examples, deciding which side of this padding is skipped is the anchors’ only real effect.

Because the anchors are recognized structurally rather than positionally, a `^` that is not at the start, as in `a^b`, is not an anchor at all. It still parses to `Carat` and still translates to `Eps`, so it contributes a node to the lowered expression but nothing to the language:

```
$ cargo run -- parse "a^b" "ab" --diag 1
Regex:  a^b
Input:  "ab"
Match:  true
Tree:   ((a, ()), b)
```

The `()` in the middle of that tree is the erased anchor. PCRE would treat the same pattern as unmatchable outside multiline mode. This is a silent difference rather than an error, and it is the one place in the frontend where a mis-written pattern is neither rejected nor faithfully represented.

Edge case. An unanchored pattern already nullable on empty input, with `"a*"` the simplest example, gets padded on both sides regardless of its own content. The padding’s own empty match, combined with either `Star`, is then enough by itself to match *any* string at all, not just runs of `a`. This is correct substring-search semantics, since "some substring matches" is satisfied by the empty substring that every string contains, and `mod.rs`’s own test `unanchored_nullable_pattern_matches_anything` asserts exactly this behaviour rather than treating it as a defect. It is worth flagging nonetheless, because such a pattern carries no information in a corpus: it matches every input and so contributes nothing to Section 8.3’s measurements beyond its own parse cost.

7.5 The Cost of Translation

Every desugaring in this chapter is exact, and several of them are expensive. Since the object that reaches Chapters 5 and 6 is the lowered `Regex` rather than the pattern string, the size of that object, and not the length of the pattern, is what the parsers’

per-character work scales with. Counting nodes in the lowered result for a few patterns makes the translation’s real price visible:

The purely structural desugarings stay small, and their shapes are worth seeing written out, since each is exactly what Section 7.3’s table prescribes:

Pattern	Lowers to	Nodes
<code>a*</code>	<code>Star(Lit)</code>	2
<code>a?</code>	<code>Alt(Lit, Eps)</code>	3
<code>a+</code>	<code>Seq(Lit, Star(Lit))</code>	4
<code>a{2}</code>	<code>Seq(Seq(Eps, Lit), Lit)</code>	5
<code>a{2,}</code>	exact copies then <code>Star</code>	8
<code>a{2,4}</code>	exact copies then nested options	15

Character classes and search padding are what change the order of magnitude. A `\d` is a 10-way `Alt` chain at 19 nodes, `\w` a 63-way one at 125, `[^a]` a 97-way one at 193, and `.` a 98-way one at 195. An unanchored search for the three-character literal `abc` reaches 399.

That last figure is the one that matters most for Section 8.3. Searching for a three-character literal builds a 399-node expression, because each `wildcard_run()` is a 195-node alternation under a `Star` and two of them are needed. Every derivative step then walks that structure.

Figure 7.4 puts these on one logarithmic axis, together with two patterns closer to what the corpus actually contains. The span is the point: the same six constructors carry expressions differing by more than two orders of magnitude, and nothing about the pattern’s source length predicts where in that range it lands.

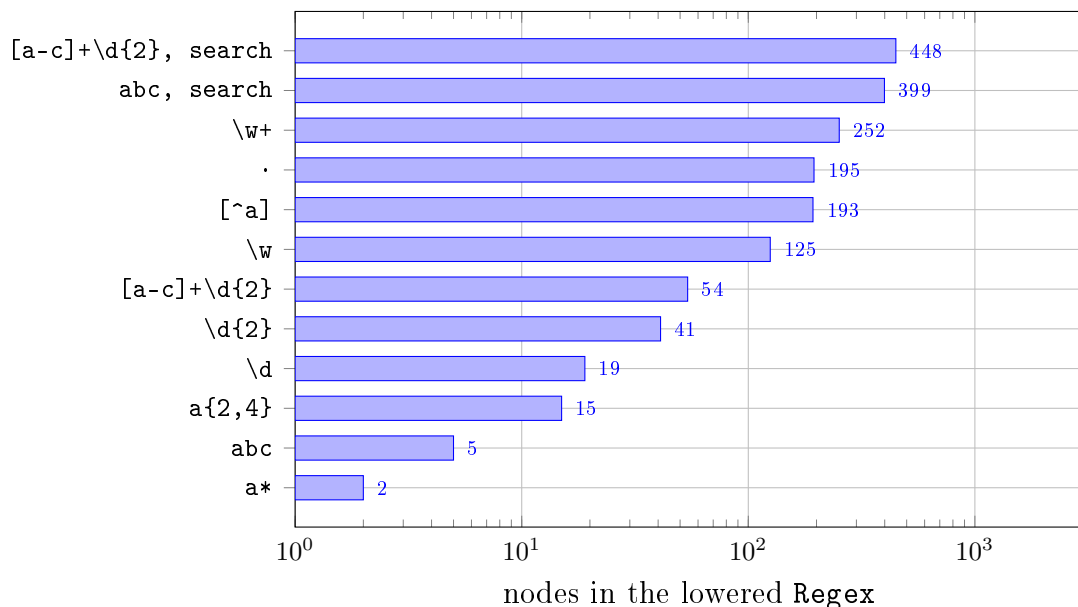


Figure 7.4: Lowered node counts, logarithmic horizontal axis. The three smallest are plain structural desugarings, with no character class involved. Every entry from `\d` upward expands one, and the two `search` entries additionally carry the padding of Figure 7.3. A realistic corpus pattern therefore starts around 450 nodes before any input is read. Regenerate with `cargo run --release --example thesis_figures -- lowered`.

This is the single largest constant factor separating this implementation from an engine

with a native character-class representation, and it applies to the corpus patterns uniformly, since essentially all of them are unanchored searches containing at least one class shorthand.

The expansion is also visible in the diagnostics, in a way that is unhelpful rather than merely large. Because the expected-set reported on a failure (Section 9.3.3) is read off the first-literal set of the expression at the point of failure, and a lowered class *is* that set, a single mismatched character against `.` reports all 98 alternatives individually:

```
$ cargo run -- parse "." "ä" --diag 1
Regex:  .
Input:  "ä"
Match:  false
Error:  position 1: found 'ä', expected '\t', '\n', '\r', ' ',
        '!', '"', '#', '$', '%', '&', ... , '}', or '~'
```

(abbreviated here; the tool prints the full list). The message is accurate, and it is exactly what the pattern denotes after lowering, but it no longer resembles the pattern the user wrote. A native class representation would let the diagnostics layer report **expected: any character** instead, which is one more argument for Section 9.5.2 beyond the performance one, and Section 9.3.4 treats it as an instance of the general limit on what a tracing layer can recover once an abstraction has been compiled away.

The effect compounds rather than merely adding, which is why the chart climbs faster than the patterns lengthen. `\d{2}` lowers to two copies of the 19-node digit chain, reaching 41 nodes, and `\w+` to a 63-way alternation appearing twice, once for the required copy and once inside the **Star**, reaching 252. Stacking both effects, `[a-c]+\d{2}` is 54 nodes as a full-string pattern and 448 once padded for search, from twelve characters of source text.

Case folding is the one remaining multiplier, and it is comparatively cheap: `case_fold` turns each literal *letter* into a two-way **Alt** and leaves everything else alone, so `/abc/i` lowers to 405 nodes against `/abc/`'s 399, six more for three folded letters. Unlike the class expansion, this one scales with the number of letters in the pattern rather than with the size of the alphabet.

None of this changes which language is denoted, and none of it changes which parse tree any of the four parsers selects. It changes only how much structure each of them traverses per input character, which is exactly the quantity Section 8.5 measures and Section 9.5.2 proposes to reduce.

Evaluation

Chapters 5 and 6 developed four parser combinations – `deriv_std`, `deriv_bc`, `pderiv_std`, `pderiv_bc` – each claimed to compute either the POSIX or the Greedy $\geq_r$ -ordered parse tree, by four structurally different constructions. This chapter verifies those claims empirically: Section 8.2 describes how the test suite checks the four parsers, and the three Boolean matchers of Chapter 3, against each other and against the paper’s own worked examples; Section 8.3 describes how a corpus of real-world regular expressions was assembled to test and benchmark against, alongside the synthetic patterns used elsewhere in this thesis; Section 8.4 reports the results of that testing; and Section 8.5 measures where the four parsers’ costs actually diverge from each other, on both the synthetic and real-world workloads.

8.1 Compared Engines

The primary object of comparison in this chapter is the four parser combinations against *each other* – two POSIX, two Greedy, one tree-building and one bit-coded within each pair (Chapters 5-6) – since that internal comparison is what this thesis’s own theoretical claims (POSIX-correctness, Greedy/POSIX agreement on membership, the tree-vs-bit-coded performance trade-off) are actually about. A second, external comparison against established regular expression engines is motivated in Chapter 1.1 and positions these results relative to the wider field. It is implemented as an optional, feature-gated addition to this codebase (Rust’s `regex` crate as an ordinary optional dependency; Google’s RE2 via a small C++ shim over its own API, since RE2 offers no C API of its own) rather than a default one, specifically so that the default build stays free of any external regular-expression engine dependency – `cargo build/cargo test/cargo bench` never touch either engine unless the `external-engines` feature is explicitly enabled. Section 8.6 reports and discusses the resulting numbers; what follows briefly positions both engines conceptually, since that section refers back to this framing.

8.1.1 Rust’s `regex` Crate

Rust’s `regex` crate compiles a pattern to a Thompson-construction NFA and simulates it without backtracking, guaranteeing linear-time matching in the input length independent of the pattern – the same automaton-based guarantee Chapter 1.1 attributes to finite-automata approaches generally. Its disambiguation policy is leftmost-first, matching Section 4.3.1’s Greedy order rather than POSIX leftmost-longest; it offers no POSIX mode. This makes it a natural comparison point for this thesis’s Greedy parsers (`pderiv_std`/`pderiv_bc`) specifically, on both performance and, since both claim the same disambiguation policy,

parse-tree agreement – an even more direct check than the membership-only agreement this thesis’s own Greedy/POSIX comparison already establishes (Section 8.4).

8.1.2 RE2

Google’s RE2 is likewise automaton-based (Thompson NFA compiled lazily to a DFA on demand), explicitly designed to avoid the exponential backtracking blowup of backreference-supporting engines [Google LLC, 2024, Cox, 2007]. Unlike Rust’s `regex` crate, RE2 supports an explicit POSIX leftmost-longest mode alongside its default Perl-style leftmost-first mode, making it a comparison point for *both* disambiguation policies studied in this thesis, in one engine, depending on which mode is selected.

8.2 Testing Strategy

The test suite is split, following ordinary Rust convention, into unit tests (`#[cfg(test)] mod tests` blocks inside the `src/` files they exercise, testing private helper functions alongside public ones) and integration tests (`tests/`, exercising only the public API each parser or matcher module exposes). `tests/common/mod.rs` factors out what is shared across the integration suites: the two worked examples from the paper, `paper_r1` and `paper_r2` (Section 4.2), and two assertion helpers, `assert_round_trip` (checking `flatten(parse(r,w)) == w`, i.e. that a produced parse tree is actually evidence for the word it was built from) and `assert_parsers_agree` (comparing two `Option<ParseTree>` results with a labeled failure message identifying which two implementations disagreed).

8.3 Dataset Preparation

The synthetic patterns used throughout this thesis – `paper_r1`, `paper_r2`, the pathological $(a^*)^*$ -style expressions of Section 3.1 – are deliberately small and hand-constructed, chosen to isolate one specific algorithmic property at a time. They say little about how any of the four parsers behaves on the regular expressions people actually write. Mamouras et al.’s disambiguation-robustness study [Mamouras et al., 2024] motivates closing that gap: it studies exactly the POSIX-versus-Greedy question this thesis studies, via real Snort/Suricata intrusion-detection signatures, and finds concrete real-world patterns where the two policies visibly diverge. That paper provides no public dataset artifact of its own, so this thesis’s corpus was independently assembled from three families of real-world regular expression sources chosen to cover different "shapes" of real regex use, not three samples of the same thing:

- **Snort/Suricata** network intrusion-detection signatures (`pcre:"..."` rule options, sourced from the Emerging Threats Open ruleset) – literal-heavy, moderate complexity, the closest of the three to "everyday" regex use.
- **SpamAssassin** email spam-filtering rules (`body/header` Perl-regex rules) – content matching over unstructured text, a different shape than network signatures.
- **RegexLib** crowd-sourced general-purpose patterns (validators, extractors) – the opposite end, where chained lookahead assertions are idiomatic, stress-testing the frontend’s rejection path hardest of the three.

Extraction. Each source embeds its patterns in a different surrounding syntax, so each needs its own extractor (`examples/extract_dataset.rs`): Suricata’s rule lines carry a `pcre:"/PATTERN/FLAGS"`; option; SpamAssassin’s carry a bare `body/header NAME = /PATTERN/FLAGS` without the Suricata’s trailing-quote convention; RegexLib’s sample file lists plain patterns directly, one (possibly multi-line) entry per blank-line-delimited block. All three converge on the same `/PATTERN/FLAGS` PCRE delimiter convention once extracted, parsed by `frontend::parse_pcre_rule` – including the case-insensitive `i` flag, applied by case-folding every literal character in the translated `ExtPat` to an `Any` of both cases (Chapter 7’s frontend) rather than as a runtime matching option, since `Regex` itself has no case-insensitivity concept.

Filtering. Real rules routinely use constructs no derivative-based engine over the six-constructor `Regex` of Section 2.3 can express at all – backreferences and lookahead assertions describe languages that are not regular in the formal sense this thesis’s entire construction depends on. `parse_pcre_rule` rejects both outright with a descriptive error rather than silently mis-parsing them, and `extract_dataset` tallies every rejection by cause (`backreference`, `lookaround`, or `other` – chiefly bound expressions such as `{1586,}` that would unroll past this frontend’s repetition cap, and a handful of unsupported `(?...)` group syntaxes). Re-running this pipeline against the tracked raw sources gives the current triage:

Format	Extracted	Accepted	Backreference	Lookaround	Other
Suricata	231	209 (90.5%)	5	11	6
SpamAssassin	82	69 (84.1%)	1	11	1
RegexLib	499	407 (81.6%)	8	55	29
Total	812	685 (84.4%)	14	77	36

Lookaround dominates the rejections and is concentrated almost entirely in RegexLib, matching that family’s idiomatic use of chained assertions; Suricata and SpamAssassin reject mostly on the same handful of negative-lookahead idioms (guarding against one specific following word). The accepted 685 patterns, deduplicated across all three formats into `data/corpus_patterns.txt` (665 lines after deduplication), are what Section 8.5.5’s dataset benchmark actually runs against.

Unanchored matching. Real rules are *written* as search patterns – a Suricata rule fires on seeing a match anywhere in a packet payload, not on the payload being exactly that pattern – so they are parsed for substring search via `frontend::parse_dataset_pattern/parse_pcre_rule` rather than the full-string-equality `parse_pattern` used elsewhere in this thesis’s own examples. Section 7.4 covers the mechanism in full – an unanchored core pattern is padded with an ordinary `Star` over a 98-way alternation on whichever side lacks an explicit `^/$` anchor, with no special status in the parser layer – and the one consequence worth restating here: real dataset patterns, run unanchored the way they have to be, are measurably bigger and expose scaling behavior the small synthetic patterns of Section 8.5.4 do not by themselves.

What this dataset does not (yet) give. The corpus was triaged for "does it parse at all," not for "does POSIX visibly diverge from Greedy on this pattern the way `paper_r1/paper_r2` do" – unlike Mamouras et al.’s own disambiguation-robustness analysis [Mamouras et al.,

2024], this thesis does not (yet) identify which, if any, of the 685 accepted real-world patterns are themselves disambiguation-non-robust in their sense. Combining this dataset with their static-analysis technique to find such patterns automatically, or constructing inputs that provoke visible POSIX/Greedy divergence on the existing corpus directly, is a natural extension of this evaluation left to future work (Section 9.5.2).

8.4 Correctness Verification

8.4.1 Paper Examples Reproduced

Both worked examples from Sulzmann and Lu’s paper are reproduced as executable tests rather than only as the manual derivations of Sections 4.3 and 4.3.3: `parse_recursive_paper_r1_on_ab` and `parse_recursive_paper_r2_on_ab` (`tests/test_deriv_std.rs`) assert the exact POSIX tree shape derived by hand in Section 4.3.3; `paper_r1_computes_greedy_not_posix` and `paper_r2_computes_greedy_not_posix` (`src/parsers/pderiv_bc/parse.rs`) assert both the Greedy tree shape of Section 4.3.1 *and* that it disagrees with the POSIX answer on the identical input – encoding the divergence itself, not merely each answer separately, as the property under test. Every one of these four assertions currently passes, confirming this implementation reproduces both documented policies exactly on the paper’s own examples.

8.4.2 Cross-Implementation Equivalence

Following the agreement-testing strategy of Section 8.2, the full test suite checks, and at the time of writing passes, agreement at every layer this thesis introduces more than one implementation of the same specification:

- `match_naive = match_deriv = match_pderiv` on membership, across the hand-picked cases of `tests/test_matchers.rs`.
- `parse_recursive` and `parse_loop` produce identical parse trees on every shared test case (Section 5.6.4), including `recursive_traced_and_loop_traced_agree_on_all_paper_c` comparing full derivation traces, not just final answers.
- `deriv_bc` agrees with `deriv_std` (via `parse_recursive`) on every shared test case (`bitcoded_agrees_with_recursive`, `tests/test_deriv_bc.rs`), the empirical counterpart of Section 5’s claim that both compute the identical POSIX tree by different means.
- `pderiv_std` and `pderiv_bc` agree exactly, not just on membership, on every hand-picked and fuzzed case (Section 8.2).
- All four parser combinations agree on membership wherever POSIX and Greedy are not known to diverge, and the two POSIX parsers (`deriv_std`, `deriv_bc`) and two Greedy parsers (`pderiv_std`, `pderiv_bc`) each agree exactly with their own policy-mate on tree shape – exercised directly by the CLI’s `--parser all` comparison mode (`src/cli/parser.rs`), which reports all three checks side by side for a given regex/input pair on demand.

No disagreement has been observed in any of these checks against the current implementation; a regression in any one parser would show up first as a failure here, in the specific

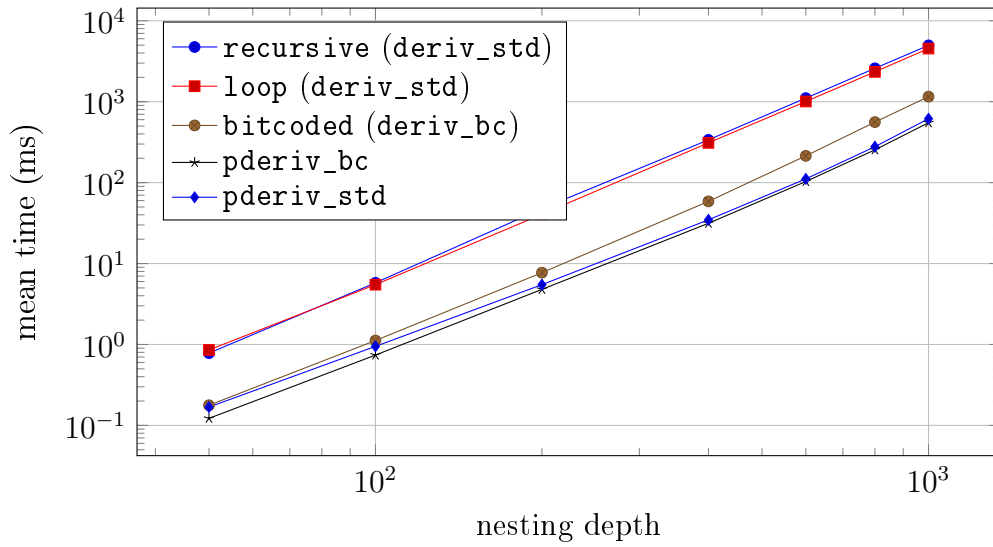
pairing that parser participates in, before it could show up as an incorrect answer with no diagnosis of which implementation is at fault.

8.5 Performance Analysis

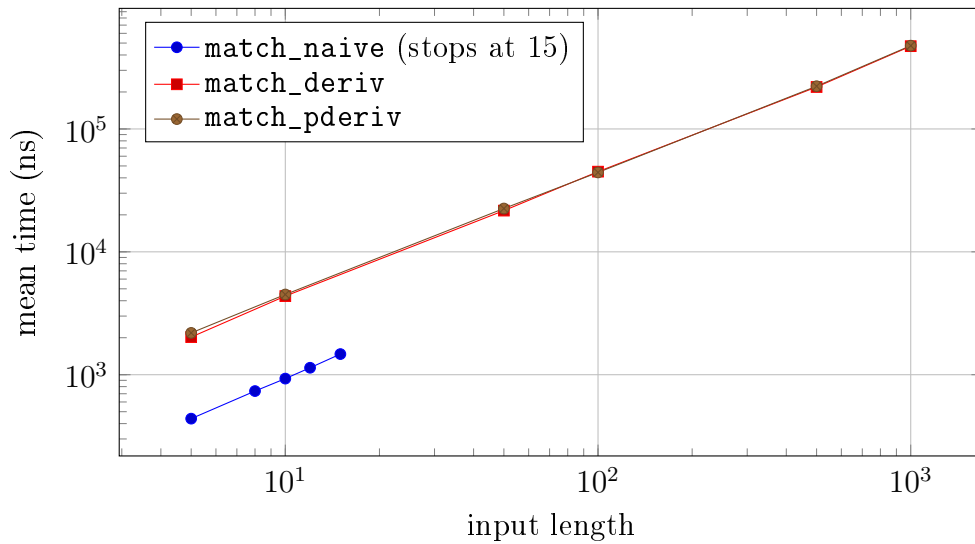
8.5.1 Stack Depth: Recursive vs. Iterative Construction

8.5.2 Heap Allocation: deriv_std vs. deriv_bc

8.5.3 Derivative-Based vs. Partial-Derivative-Based



8.5.4 Pathological Inputs



8.5.5 Benchmark Results

Parser	$n=5$	$n=10$	$n=15$	$n=18$
recursive (deriv_std)	8.0 μ s	110.7 μ s	1.427 ms	6.052 ms
bitcoded (deriv_bc)	11.4 μ s	183.8 μ s	4.175 ms	49.491 ms
pderiv_bc	13.4 μ s	210.6 μ s	2.207 ms	9.236 ms
pderiv_std	22.4 μ s	323.9 μ s	3.374 ms	14.263 ms

Parser	Mean time / sweep
deriv_std_rec	1.311 s
deriv_std_loop	1.091 s
deriv_bc	643.5 ms
pderiv_bc	159.2 ms
pderiv_std	158.6 ms

Parser	Mean time
deriv_std_rec	1.000 s
deriv_bc	81.1 ms

8.6 Comparison with Existing Engines

8.6.1 Performance

Engine	Mean time / sweep	vs. regex
deriv_std_rec (POSIX, recursive)	1.503 s	$\times 1,415,254$
deriv_std_loop (POSIX, loop)	1.218 s	$\times 1,146,893$
deriv_bc (POSIX, bit-coded)	690.74 ms	$\times 650,417$
pderiv_bc (Greedy, bit-coded)	171.85 ms	$\times 161,815$
pderiv_std (Greedy, tree-injection)	173.28 ms	$\times 163,167$
regex (Rust crate)	1.06 μ s	$\times 1$
re2_perl (leftmost-first)	6.61 μ s	$\times 6.2$
re2_posix (leftmost-longest)	2.75 μ s	$\times 2.6$

Engine	Mean time / sweep	vs. regex
match_deriv	273.07 ms	$\times 259,815$
match_pderiv	134.95 ms	$\times 128,397$
regex (Rust crate)	1.05 μ s	$\times 1$
re2_perl (leftmost-first)	6.63 μ s	$\times 6.3$
re2_posix (leftmost-longest)	2.72 μ s	$\times 2.6$

8.6.2 Standardized Benchmarking via rebar

Discussion

9.1 Discussion of Evaluation Results

9.2 Rust versus Haskell

9.2.1 Language Characteristics of the Reference Implementation

The reference material this thesis translates from – both flops14-extended’s own presentation [Sulzmann and Lu, 2014] and Sulzmann’s fuller teaching-page Haskell transcript [Sulzmann, 2026] – leans on several Haskell features with no direct Rust counterpart, each already surfaced individually at the point in this thesis where it first mattered; this section collects them.

Algebraic data types with blanket deriving. The teaching page’s own `regex` type,

```
data R = Phi | Eps | L Char | Seq R R | Choice R R | Star R
    deriving (Show, Eq, Ord)
```

derives structural equality, ordering, *and* printing in one clause, whether or not a given piece of code actually uses all three. Section 2.3’s `Regex` derives only `Debug`, `Clone`, `PartialEq`, `Eq`, and `Hash` – no `Ord` – because nothing in this thesis’s Rust port ever compares two `Regex` values for ordering; Rust’s per-trait, opt-in `derive` makes that omission a visible, deliberate fact about the codebase, where the reference’s blanket `deriving` leaves it implicit.

GADT syntax used without a type index. The reference’s parse tree type is written as a GADT,

```
data U where
  Nil :: U
  Empty :: U
  Letter :: Char -> U
  LeftU :: U -> U
  RightU :: U -> U
  Pair :: (U,U) -> U
  List :: [U] -> U
```

but no constructor’s return type is ever refined to anything other than plain `U` – this is an ordinary sum type written in GADT syntax, not a use of GADTs’ actual type-level

power. Section 5.6.1’s `ParseTree` enum needs nothing beyond an ordinary Rust `enum` to match it.

9.2.2 What the Reference Provides vs. What It Leaves Open

9.2.3 Translation Strategy

9.2.4 Translation Insights

9.3 Mismatch Analysis via Derivative Tracing

9.3.1 The Gap

9.3.2 Deriving Diagnostics for Free from the Algorithm Itself

9.3.3 Design

`DiagLevel` (`src/diagnostics/mod.rs`) has four values – `Off` (0), `Basic` (1, `regex/input/-match/tree/error caret`), `Verbose` (2, `Basic` plus timing, expression count, and construction steps), and `Debug` (3, the full derivation trace, written to a file if `REGEX_DIAG_REPORT` is set) – selected via the CLI’s `-diag` flag or the `REGEX_DIAG` environment variable, and applies only to `parse`, not `match`: a Boolean matcher has no parse tree, and hence nothing at levels 1–3 to report beyond what the trace itself would show, which level 3 already provides directly. Each of the four parser variants renders its own trace independently rather than sharing one generic renderer – `diag_2/{deriv_std,deriv_bc,pderiv_std,pderiv_bc}.rs` and the analogous four files under `diag_3/` – since what is worth showing at each level differs by construction (a bit-coded parser’s trace is bit-strings and `ARegex` states; the tree-injection parsers’ traces are `Regex` derivative sequences and, at level 3, the backward injection steps that reconstruct a tree from them). `docs/DIAG.md` documents concrete output at every level, for every parser, on both successful and failing matches.

9.3.4 Scope and Limits

9.4 Optimization Challenges and Insights

9.5 Future Work

9.5.1 Paper-Suggested Extensions

Tagged NFA for Submatching

Space-Efficient Parsing

DFA/NFA Hybrid Abstraction

9.5.2 Further Performance Optimizations

Character Classes in Place of Literal Alternation

Structural Sharing via Rc

Memoizing `deriv/nullable`

Benchmarking the Matcher, Not the Parser, for Membership

9.5.3 A Full Mismatch Diagnostics System

Related Work and Conclusion

Bibliography

- Valentin Antimirov. Partial derivatives of regular expressions and finite automaton constructions. *Theoretical Computer Science*, 155(2):291–319, 1996. doi: 10.1016/0304-3975(95)00182-4.
- Oren Ben-Kiki, Clark Evans, and Ingy döt Net. YAML ain’t markup language (YAML) version 1.1, 2005. URL <https://yaml.org/spec/1.1/>. Accessed: 2026-09-12.
- Janusz A. Brzozowski. Derivatives of regular expressions. *Journal of the ACM*, 11(4):481–494, 1964. doi: 10.1145/321239.321249.
- Russ Cox. Regular expression matching can be simple and fast (but is slow in Java, Perl, PHP, Python, Ruby, ...). <https://swtch.com/~rsc/regexp/regexp1.html>, 2007. Accessed: 2026-08-05.
- Alain Frisch and Luca Cardelli. Greedy regular expression matching. In *Automata, Languages and Programming (ICALP 2004)*, volume 3142, pages 618–629. Springer, 2004. doi: 10.1007/978-3-540-27836-8_53.
- Google LLC. RE2: A fast, safe, POSIX-compatible regular expression library. <https://github.com/google/re2>, 2024. Accessed: 2026-08-05.
- HitchDev. The norway problem: Why StrictYAML refuses to do implicit typing (and so should you), 2026. URL <https://hitchdev.com/strictyaml/why/implicit-typing-removed/>. Accessed: 2026-09-12.
- IEEE and The Open Group. Posix.1-2017: Base definitions, chapter 9 (regular expressions), 2017. IEEE Std 1003.1-2017.
- Konstantinos Mamouras, Alexis Le Glaunec, Wu Angela Li, and Agnishom Chattopadhyay. Static analysis for checking the disambiguation robustness of regular expressions. *Proceedings of the ACM on Programming Languages*, 8(PLDI):231, 2024. doi: 10.1145/3656461.
- Martin Sulzmann. Programming paradigms: Regular expressions. <https://sulzmann.github.io/ProgrammingParadigms/pp-regular-expressions.html>, 2026. Accessed: 2026-08-08.
- Martin Sulzmann and Kenny Zhuo Ming Lu. Regular expression sub-matching using partial derivatives. In *PPDP*, pages 79–90, 2012. doi: 10.1145/2370776.2370788.

- Martin Sulzmann and Kenny Zhuo Ming Lu. Posix regular expression parsing with derivatives. In *Functional and Logic Programming (FLOPS 2014)*, volume 8475. Springer, 2014.
- The Flex Project. Patterns. Lexical Analysis with Flex, 2026. URL <https://westes.github.io/flex/manual/Patterns.html>. Accessed: 2026-09-12.
- The Rust Project Developers. Using `Box<T>` to point to data on the heap. The Rust Programming Language, 2026a. URL <https://doc.rust-lang.org/book/ch15-01-box.html>. Accessed: 2026-09-11.
- The Rust Project Developers. Derivable traits. The Rust Programming Language, 2026b. URL <https://doc.rust-lang.org/book/appendix-03-derivable-traits.html>. Accessed: 2026-09-11.
- The Rust Project Developers. Enums and pattern matching. The Rust Programming Language, 2026c. URL <https://doc.rust-lang.org/book/ch06-00-enums.html>. Accessed: 2026-09-11.
- The Rust Project Developers. The `match` control flow construct. The Rust Programming Language, 2026d. URL <https://doc.rust-lang.org/stable/book/ch06-02-match.html>. Accessed: 2026-09-11.
- The Rust Project Developers. `char` primitive type. Rust Standard Library, 2026e. URL <https://doc.rust-lang.org/std/primitive.char.html>. Accessed: 2026-09-11.
- The Rust Project Developers. `std::hash`. Rust Standard Library, 2026f. URL <https://doc.rust-lang.org/std/hash/index.html>. Accessed: 2026-09-11.
- The Rust Project Developers. `HashMap` in `std::collections`. Rust Standard Library, 2026g. URL <https://doc.rust-lang.org/std/collections/struct.HashMap.html>. Accessed: 2026-09-11.
- The Rust Project Developers. `HashSet` in `std::collections`. Rust Standard Library, 2026h. URL <https://doc.rust-lang.org/std/collections/struct.HashSet.html>. Accessed: 2026-09-11.
- The Rust Project Developers. Enumerated types. The Rust Reference, 2026i. URL <https://doc.rust-lang.org/reference/types/enum.html>. Accessed: 2026-09-11.
- The Rust Project Developers. Match expressions. The Rust Reference, 2026j. URL <https://doc.rust-lang.org/reference/expressions/match-expr.html>. Accessed: 2026-09-11.
- The Rust Project Developers. Types. The Rust Reference, 2026k. URL <https://doc.rust-lang.org/reference/types.html>. Accessed: 2026-09-11.

Code Listings

Full Benchmark Results

Test Case Catalogue

CLI and ENV Reference